Chapter 06

Async Programming and Data Integration

6 lessons • 25 hr

elevify

CHAPTER 06 CONTENTS

Chapter 06

6 lessons • 25 hr

LESSON 01	**Reading and Writing Local Files**	1 hr 6 min	**4**
	Summary	8 min	**6**
	Lesson transcript	8 min	**7**
	Further study materials	57 min	**10**
	Exercises		**43**
LESSON 02	**Fetching Data with httpx**	55 min	**53**
	Summary	7 min	**55**
	Lesson transcript	7 min	**56**
	Further study materials	48 min	**59**
	Exercises		**84**
LESSON 03	**Connecting to a SQLite Database**	1 hr 2 min	**91**
	Summary	10 min	**93**
	Lesson transcript	10 min	**94**
	Further study materials	52 min	**98**
	Exercises		**125**
LESSON 04	**Async Handlers in Flet**	18 min	**137**
	Summary	8 min	**139**
	Lesson transcript	8 min	**140**
	Further study materials	10 min	**141**
	Exercises		**147**

Chapter 06

6 lessons • 25 hr

LESSON 05	**Displaying Dynamic Lists from APIs**	36 min	**152**
	Summary	8 min	**154**
	Lesson transcript	8 min	**155**
	Further study materials	28 min	**158**
	Exercises		**171**
CONCLUSION	**Practical project of the chapter**		**178**
	Context and Provocation		**180**
	Development Guide		**184**
	Self-Assessment Rubric		**184**
	Model Response & Assessment Reference		**185**
	Notes		**191**

Chapter 06 • Async Programming and Data Integration

Lesson 01 • Reading and Writing Local Files

47 pages • 1 hr 6 min

elevify

Lesson 01 • Reading and Writing Local Files

1 hr 6 min • 47 pages

◎ What you will learn

Integrates Python file I/O with Flet's FilePicker control for reading and saving data. Enables offline-capable desktop applications.

Summary	8 min	**6**
Transcript	8 min	**7**
Further study materials		
Text • Reading JSON Files in Python: Context Managers, Streaming, and Error Handling Techniques	29 min	**10**
Text • Mastering Python File I/O: Text, CSV, and JSON Operations	14 min	**26**
Text • Reading Text, CSV, JSON, XML, and Excel Files in Python	14 min	**33**
Exercises		**43**
Answer key		**51**

Reading and Writing Local Files

Summary

Why Local File I/O Matters in Flet

Most real apps need to save or load data. Flet gives two tools: FilePicker for choosing files and Python file I/O for reading/writing. FilePicker opens a native dialog and returns a file path. Python I/O uses that path. They work in sequence: first pick, then read/write. This lesson covers step by step how to let users pick files, read text or JSON, write data back, and display results.

Setting Up the FilePicker

FilePicker sits in page.overlay, not as a visible widget. Create an instance, attach on_result callback, add to overlay, then call pick_files() from a button. Most common mistake is forgetting to add to overlay. Use allowed_extensions to restrict file types like ['txt', 'json']. The callback fires after user confirms selection.

Reading Text and JSON Files

The result event gives e.files list with path property. Always check e.files is not None to avoid crash if user cancels. For text, open file with UTF-8 encoding, read(), assign to Text control value, update page. For JSON, use json.load() to get dict/list, then loop to build Flet controls like ListView. No special async needed for small files.

Writing Data to Disk

Use open with mode 'w'. For plain text, write string with f.write(). For JSON, use json.dump() with optional indent for readability. To save, use FilePicker's save_file() which opens dialog for user to name file and choose folder. Result comes back in e.path (single string), not e.files. Handle both in callback if reusing picker.

Displaying, Async, and Common Pitfalls

Choose appropriate control: Text for short, TextField for editing, ListView for arrays. For large files, use asyncio.to_thread() to avoid freezing UI. Common pitfalls: forgetting page.overlay, not checking None on cancel, mixing e.path and e.files, blocking main thread. Complete flow: add picker to overlay, trigger dialog, get path, do file operation, update UI.

NOTES

Reading and Writing Local Files

Transcript

Reading and Writing Local Files in Flet

Welcome to this lesson on reading and writing local files in Flet. In this part of the Flet Python Course, you'll learn step by step to let users pick files from their own computer. Then you'll read the text or JSON from those files. After that, you'll write data back to disk. Finally, you'll display the results inside your Flet controls.

Why Local File I/O Matters

Most real apps need to persist data or load it from somewhere. A config file, a saved list, an exported report — those all live on the user's device. Flet gives you two tools: the FilePicker control lets users choose files through a native dialog, and standard Python file I/O handles the actual reading and writing. Together they cover almost any local data scenario you'll run into.

FilePicker opens the native dialog and returns a file path — that's the user interaction. Python file I/O takes that path and reads or writes the file content — that's the data handling. Two distinct jobs, working in sequence. FilePicker comes first, then Python does the I/O. The slide shows each on its own card: a file dialog on the left, Python code on the right. Keep that simple mental model in mind as we go through every step.

Setting Up the FilePicker

FilePicker isn't a visible widget you drop onto the page. Instead, it sits in `page.overlay` — Flet's container for invisible or floating controls. You create an instance, attach an `on_result` callback, add it to the overlay, and then call `pick_files()` from a button. That last step is what opens the native dialog. The trick: if you skip adding it to the overlay, nothing happens when you click the button. That's the most common setup mistake, and it's easy to miss.

You'll follow this same pattern each time. Create a FilePicker with your callback assigned to `on_result`. Append it to `page.overlay`. Then wire a button to call `pick_files()`. To restrict file types, pass `allowed_extensions` — for example, `['txt', 'json']` limits the picker to those. The callback fires automatically after the user confirms their selection.

Handling the Result Event

The result event gives you `e.files` — a list of selected file objects. Each object has a `path` property — the full file system path you'll pass to Python's `open` function. Always check that `e.files` is not None before accessing it. If the user closes the dialog without selecting anything, `e.files` will be `None`, and accessing it without that check will crash your app. One line of defensive code prevents that crash entirely.

NOTES

Reading and Writing Local Files

Transcript

Reading Text and JSON Files

Once you have the path, reading a text file is just standard Python. Open it in read mode with UTF-8 encoding, then call `f.read()` to get the entire content as a string. Assign that string to a Text control's `value`, and call `page.update()`. The content appears on the screen instantly. For typical text files, this whole process is synchronous and fast, so no async handling is necessary.

When reading a JSON file, swap out `f.read()` for `json.load(f)`. The `json` module handles all the parsing for you, returning a Python dictionary or list directly — no manual work required. Then loop over the data and build Flet controls on the fly. For example, add a Text item to a ListView for each entry. This pattern is common when loading saved settings, user data, or any structured content from disk. It's as straightforward as that.

Writing Data to Disk

Writing uses the same `open` pattern, but with mode `'w'`. Pass a string to `f.write()` for plain text. For JSON, call `json.dump()` with your dictionary or list. The `indent` parameter is optional; I'd recommend using it — it adds line breaks and spaces, making the file readable in any text editor. If you need to append to an existing file instead of overwriting it, switch to mode `'a'`.

When writing files, let users choose where to save, not just pick an existing file. FilePicker's `save_file()` opens a native dialog. The user names the file and picks the destination folder. The result comes back in `e.path` as a single string, not a list. If you reuse the same picker for opening and saving, your callback must handle both `e.path` from saving and `e.files` from picking. That's the key difference from `pick_files()`.

Displaying Content in Flet Controls

How you display file content depends on what the content is. Short string or single value? Text control does the job. Longer text, maybe read or edit? Multiline TextField works well. Structured data like a JSON array — use a ListView, loop over items, add one control per entry. Pick the container that matches your data's shape. Don't just reach for the first one that comes to mind.

Handling Large Files with Async

Synchronous reading works for small files. But a large file will freeze the UI if read directly. Use `asyncio.to_thread()` to offload the file operation to a thread. That keeps Flet's event loop running while the file loads in the background. Same principle as fetching from an API — keep heavy work off the main thread.

Common Pitfalls to Avoid

The blocking read you just saw is one of four common pitfalls. First,

NOTES

Reading and Writing Local Files

Transcript

forgetting `page.overlay` keeps the picker invisible. Second, skipping the `None` check on cancel crashes your app. Third, mixing up `e.path` with `e.files` breaks a save. Fourth, that blocking read. Avoid these, and next we walk through the complete open, read, display, and save flow.

The Complete Flow

So here's the full flow. First, you add the FilePicker to the overlay. Then you trigger the dialog from a button. Next, you grab the path from the result event. Then you perform the file operation with Python. Finally, you update the UI. That's the pattern – simple and repeatable. Every local file feature you build in Flet follows this exact sequence. Once you've got it in muscle memory, adding file support to any screen takes just a few minutes.

Real-World Example: Note-Taking App

Imagine a simple note-taking app for jotting down ideas. Without file I/O, any text the user types disappears instantly the second the app closes. But add a FilePicker and Python file I/O — you can now include a Load button that reads a JSON file into the TextField, and a Save button that writes the current note back to disk. And that's a complete, useful feature, built from exactly the tools covered in this lesson.

More Practical Applications

With FilePicker and Python file I/O, you can build a config loader that reads a JSON settings file on startup, applying user preferences. A data exporter that lets users save app data as a text or JSON file to their chosen folder. A log viewer that opens any text log file and shows its contents in a scrollable TextField. These are real, shippable features. You have the tools. Go build something that persists.

NOTES

Further study materials Lesson 1

Content type: Text

Reading JSON Files in Python: Context Managers, Streaming, and Error Handling Techniques

Available at https://python.plainenglish.io/how-to-read-json-file-in-python-with-examples-in-2026-9877d0cdca71

Full Content

Summary

JSON and Standard File Reading

JSON is the most common data format for web APIs, used by 87% of all web services. Python's built-in json module provides json.load() to read JSON files. Always use a context manager to open the file with UTF-8 encoding. Then call json.load(file) to convert JSON into Python dictionaries or lists. Handle errors like FileNotFoundError or JSONDecodeError with try-except blocks. This method is reliable for most files.

Reading JSON from Strings

When JSON data is already in memory as a string, use json.loads() instead of json.load(). This is common when processing API responses or data from databases. The function takes a string argument and returns Python objects. For example, parse an API response string into a dictionary and access its fields. Ensure you use the correct function to avoid type errors. This method is essential for working with in-memory data.

Modern Best Practices with pathlib

Modern Python development recommends using the pathlib module for file paths instead of plain strings. Path objects handle cross-platform differences automatically and provide useful methods like .exists() and .open(). Combine pathlib with json.load() for cleaner, more maintainable code. For example, create a Path object, check if the file exists, then open it with the .open() method. This approach reduces bugs and improves readability.

Handling Large JSON Files

For JSON files larger than 100 MB, loading the entire file into memory can cause performance issues. Use streaming parsers like ijson to process data iteratively. Install ijson with pip, then use ijson.items() to parse objects one at a time. This keeps memory usage constant regardless of file size. Standard parsers use about 136 MB for a 25 MB file, while streaming uses much less. This technique is crucial for big data applications.

Common Errors and Advanced Techniques

Common JSON parsing errors include FileNotFoundError,

NOTES

Further study materials Lesson 1

Summary

JSONDecodeError, and UnicodeDecodeError. Validate your JSON syntax and use try-except blocks to handle them gracefully. For production code, validate data structure after parsing, safely access nested fields, and cache frequently loaded files. Use binary mode for better performance with large files. Process data incrementally to avoid memory buildup. These practices ensure robust and efficient code.

NOTES

Further study materials Lesson 1

Transcript

How to Read JSON File in Python with Examples in 2026

Working with JSON data has become essential for modern developers. Whether you're building APIs, processing configuration files, or analyzing data from web services, knowing how to read JSON files in Python efficiently is a fundamental skill. This comprehensive guide walks you through every method, from basic file reading to advanced performance optimization techniques used by professional developers.

*[Image: https://miro.medium.com/1*M3b2zJxaU7r6Q1qayv1VEg.png]*

Understanding JSON and Why It Dominates Data Exchange

JSON currently accounts for 87% of all web API responses, making it the de facto standard for data interchange in 2025. Its simplicity, flexibility, and broad compatibility make it the preferred choice for developers building APIs, whether you are an in-house engineer or working with a dedicated mobile app development company in New York. Unlike XML, JSON's lightweight structure reduces payload sizes, leading to faster data transmission and lower bandwidth usage.

The format maps naturally to Python's data structures. JSON objects become Python dictionaries, arrays become lists, and the built-in `json` module handles all conversions seamlessly. This native compatibility is a key reason REST APIs continue to power 83% of all web services, with JSON as their primary data format.

The Standard Method: Using json.load() for File Reading

The most reliable way to read a JSON file in Python is through the built-in `json` module's `json.load()` function. This method reads a file object, parses the JSON content, and converts it into native Python data structures.

Step 1: Import the JSON Module

Python's `json` module comes pre-installed with every Python distribution, requiring no additional setup. Simply import it at the beginning of your script:

python

```
import json
```

This single import provides access to all JSON functionality, including `json.load()` for files and `json.loads()` for strings.

Step 2: Open the File Using Context Managers

NOTES

Further study materials Lesson 1

Transcript

Always use Python's `with` statement when working with files. This context manager automatically handles file closure, even if errors occur during processing. Open files in read mode with UTF-8 encoding, which is the standard for JSON:

python

```
with open('data.json', 'r', encoding='utf-8') as file:
# File operations here
pass
```

The `with` statement prevents resource leaks and makes your code more reliable by ensuring proper cleanup.

Step 3: Parse JSON with json.load()

Inside the context manager, pass the file object to `json.load()`. The function reads the entire file, parses the JSON structure, and returns a Python object:

python

```
with open('data.json', 'r', encoding='utf-8') as file:
data = json.load(file)
```

If the JSON root is an object (with curly braces), you receive a dictionary. If it's an array (with square brackets), you get a list.

Complete Working Example

Here's a full implementation that demonstrates proper error handling and data access:

python

```
import json

try:
with open('data.json', 'r', encoding='utf-8') as file:
data = json.load(file)

# Access data like a normal Python dictionary
print(f"Name: {data['name']}")
print(f"City: {data['city']}")
print(f"Courses: {', '.join(data['courses'])}")

except FileNotFoundError:
print("Error: The file 'data.json' was not found.")
except json.JSONDecodeError as e:
print(f"Error: Invalid JSON format - {e}")
except KeyError as e:
print(f"Error: Expected key {e} not found in JSON.")
```

For a sample `data.json` file:

json

```
{
```

NOTES

Further study materials Lesson 1

Transcript

```
"name": "John Doe",
"city": "New York",
"isAdmin": false,
"courses": ["History", "Math", "Science"]
}
```

This pattern forms the foundation for almost any script that loads configuration or data from JSON sources.

Reading JSON from Strings with json.loads()

When JSON data exists as a string variable rather than a file, use `json.loads()` (note the 's' for string). This commonly occurs when receiving data from API responses, database fields, or command-line arguments.

When to Choose json.loads() Over json.load()

Use `json.loads()` when your JSON data is already in memory as a string. The function expects a string argument, while `json.load()` expects a file object. Attempting to use the wrong function will cause type errors.

Common scenarios include processing HTTP responses, reading from message queues, or handling JSON embedded in other data structures.

Practical Example: Parsing API Response Data

Here's how to parse JSON data received from a web service:

python

```
import json

# Simulated API response stored as a string
api_response_string = '{"userId": 1, "id": 1, "title":
"Complete Python tutorial", "completed": false}'

# Parse the string into a Python dictionary
task_data = json.loads(api_response_string)

# Work with the parsed data
print(f"Task: {task_data['title']}")
print(f"Status: {'Completed' if task_data['completed']
else 'Pending'}")
print(f"User ID: {task_data['userId']}")
```

The distinction between `json.load()` and `json.loads()` is crucial for writing flexible code that handles data from various sources efficiently.

Modern Python Best Practice: Using pathlib for File Paths

As of 2025, modern Python development strongly favors the `pathlib` module over plain string paths. This object-oriented

NOTES

Further study materials Lesson 1

Transcript

approach to file system operations eliminates cross-platform compatibility issues and makes code more maintainable.

Why pathlib Improves Your Code

The `Path` object from `pathlib` offers several advantages:

- Cross-platform path handling (Windows, macOS, Linux)
- Built-in methods for file existence checks
- Cleaner path joining without manual string concatenation
- More readable and intuitive code
- Type safety and better IDE support

This approach removes the common bugs caused by forward slash versus backslash differences between operating systems.

Implementing pathlib with JSON Operations

Here's the recommended pattern combining `pathlib` with `json.load()`:

python

```
import json
from pathlib import Path

# Create a Path object for your JSON file
json_file_path = Path('data.json')

# Check existence before attempting to open
if json_file_path.exists():
with json_file_path.open('r', encoding='utf-8') as file:
data = json.load(file)
print(f"Loaded data from: {json_file_path.absolute()}")
print(f"City: {data['city']}")
else:
print(f"File not found at path:
{json_file_path.absolute()}")
```

For complex project structures, `pathlib` makes working with relative paths much cleaner:

python

```
from pathlib import Path

# Get the script's directory
script_dir = Path(__file__).parent

# Build path to config file in a subdirectory
config_path = script_dir / 'config' / 'settings.json'

if config_path.is_file():
with config_path.open('r', encoding='utf-8') as file:
config = json.load(file)
```

This modern approach is highly recommended for all new Python projects and demonstrates professional coding practices.

NOTES

Further study materials Lesson 1

Transcript

Handling Large JSON Files: Performance Optimization

When working with JSON files larger than 100 MB, loading the entire file into memory can become problematic. Traditional methods that read complete JSON files into Python often consume excessive memory, leading to performance slowdowns or crashes. In such cases, streaming parsers offer an effective solution.

Using ijson for Memory-Efficient Parsing

The ijson library allows you to parse and extract data from JSON files iteratively, processing JSON data in smaller chunks. This makes it possible to work with files that exceed available memory.

First, install ijson using pip:

bash

```
pip install ijson
```

Here's how to stream through a large JSON array:

python

```
import ijson

# Process large JSON file without loading it entirely
with open('large_data.json', 'rb') as file:
# Parse items iteratively
objects = ijson.items(file, 'item')

for obj in objects:
# Process each object individually
print(f"Processing: {obj['name']}")
# Your processing logic here
```

With proper configuration, **ijson** can achieve **up to 656× faster JSON parsing** compared to naive implementations. The key is using the `items()` method with an appropriate prefix to efficiently target your data structure.

Performance Comparison: Standard vs Streaming

When parsing a 25 MB JSON file, the standard library uses approximately 136 MB of RAM and takes about 0.42 seconds, whereas optimized parsers like orjson use only 113 MB and complete in 0.28 seconds (source: "Faster, more memory-efficient Python JSON parsing with msgspec").

For larger files in the gigabyte range, streaming parsers maintain constant memory usage, regardless of file size.

Common Errors and Their Solutions

Understanding typical JSON parsing errors helps you write more robust code and debug issues quickly.

NOTES

Further study materials Lesson 1

Transcript

Error 1: FileNotFoundError

This occurs when Python cannot locate the specified file. Common causes include:

- Typos in the filename
- Incorrect file path
- Script running from a different directory than expected

Solution:

python

```
from pathlib import Path
import json

json_file = Path('data.json')

if not json_file.exists():
print(f"Error: File not found at {json_file.absolute()}")
print(f"Current directory: {Path.cwd()}")
else:
with json_file.open('r', encoding='utf-8') as file:
data = json.load(file)
```

Error 2: json.decoder.JSONDecodeError

This error indicates invalid JSON syntax. Common issues include:

- Trailing commas after the last array or object element
- Single quotes instead of double quotes for strings
- Missing commas between elements
- Unescaped special characters in strings

Solution: Validate your JSON file using built-in Python tools or online validators:

python

```
import json

try:
with open('data.json', 'r', encoding='utf-8') as file:
data = json.load(file)
except json.JSONDecodeError as e:
print(f"JSON syntax error at line {e.lineno}, column
{e.colno}")
print(f"Error message: {e.msg}")
print("Please check your JSON file for syntax errors")
```

Common syntax mistakes to avoid:

python

```
# INCORRECT - trailing comma
{"name": "Test", "value": 123,}

# CORRECT
```

NOTES

Further study materials Lesson 1

Transcript

```
{"name": "Test", "value": 123}

# INCORRECT - single quotes
{'name': 'Test'}

# CORRECT - double quotes
{"name": "Test"}
```

Error 3: UnicodeDecodeError

This happens when the file uses a different character encoding than UTF-8. While UTF-8 is the standard for JSON files Working With JSON Data in Python - Real Python, you may occasionally encounter files with alternative encodings.

Solution:

python

```
import json

# Try UTF-8 first (standard)
try:
with open('data.json', 'r', encoding='utf-8') as file:
data = json.load(file)
except UnicodeDecodeError:
# Fallback to alternative encoding if needed
print("UTF-8 failed, trying latin-1...")
with open('data.json', 'r', encoding='latin-1') as file:
data = json.load(file)
```

Always explicitly specify `encoding='utf-8'` to make your code's expectations clear and prevent platform-dependent behavior.

Advanced Techniques for Production Code

Professional applications require robust error handling and data validation beyond basic parsing.

Validating JSON Structure

Use Python's type checking and validation patterns to ensure JSON contains expected fields:

python

```
import json
from typing import Dict, List, Any

def validate_user_data(data: Dict[str, Any]) -> bool:
"""Validate that JSON contains required user fields."""
required_fields = ['name', 'email', 'id']

for field in required_fields:
if field not in data:
print(f"Missing required field: {field}")
return False
```

NOTES

Further study materials Lesson 1

Transcript

```
if not isinstance(data['id'], int):
print("Field 'id' must be an integer")
return False

return True

# Usage
with open('user.json', 'r', encoding='utf-8') as file:
user_data = json.load(file)

if validate_user_data(user_data):
print(f"Processing user: {user_data['name']}")
else:
print("Invalid user data structure")
```

Working with Nested JSON Structures

Complex JSON often contains deeply nested objects and arrays. Here's how to safely access nested data:

python

```
import json

def safe_get(data: dict, *keys, default=None):
"""Safely retrieve nested values from dictionaries."""
for key in keys:
if isinstance(data, dict):
data = data.get(key, default)
else:
return default
return data

# Example nested JSON
nested_json = '''
{
"user": {
"profile": {
"address": {
"city": "New York",
"zipcode": "10001"
}
}
}
}
'''

data = json.loads(nested_json)

# Safe access with default values
city = safe_get(data, 'user', 'profile', 'address',
'city', default='Unknown')
country = safe_get(data, 'user', 'profile', 'address',
'country', default='Unknown')

print(f"City: {city}") # Output: City: New York
print(f"Country: {country}") # Output: Country: Unknown
```

NOTES

Further study materials Lesson 1

Transcript

Handling Multiple JSON Objects in One File

Some applications store multiple JSON objects in a single file (JSON Lines format). Each line contains a complete JSON object:

python

```
import json

# Process JSON Lines file
with open('events.jsonl', 'r', encoding='utf-8') as file:
for line_number, line in enumerate(file, 1):
try:
event = json.loads(line)
print(f"Processing event {line_number}: {event['type']}")
except json.JSONDecodeError as e:
print(f"Error on line {line_number}: {e}")
continue
```

This format is popular for log files, data streams, and large datasets because each line can be processed independently.

Performance Best Practices for 2025

Follow these guidelines to ensure optimal performance when reading JSON files:

1. Use Binary Mode for Better Performance

When working with large files, opening them in binary mode can improve parsing speed:

python

```
import json

# Better performance for large files
with open('large_data.json', 'rb') as file:
data = json.load(file)
```

2. Implement Caching for Frequently Accessed Files

If your application reads the same JSON file multiple times, cache the parsed data:

python

```
import json
from pathlib import Path
from functools import lru_cache

@lru_cache(maxsize=32)
def load_config(config_path: str) -> dict:
"""Load and cache configuration file."""
with open(config_path, 'r', encoding='utf-8') as file:
return json.load(file)

# First call reads from file
config = load_config('config.json')
```

NOTES

Further study materials Lesson 1

Transcript

```
# Subsequent calls use cached data
config = load_config('config.json') # Instant, no file
I/O
```

3. Process Data Incrementally

For large datasets, process and discard data as you go rather than accumulating results:

python

```
import json

def process_large_dataset(filename):
"""Process large JSON array incrementally."""
results = []

with open(filename, 'r', encoding='utf-8') as file:
data = json.load(file)

for item in data:
# Process and store only what you need
processed = {
'id': item['id'],
'summary': item['description'][:100] # Keep only first
100 chars
}
results.append(processed)

# Periodically write results and clear memory
if len(results) >= 1000:
write_results_to_database(results)
results.clear()
```

Frequently Asked Questions

What's the core difference between json.load() and json.loads()?

The distinction is in their input types. `json.load()` accepts a file object and reads directly from files, while `json.loads()` accepts a string and parses in-memory data. Choose based on whether your JSON data is in a file or already loaded as a string.

Can Python handle very large JSON files?

Yes, but the approach matters. Standard `json.load()` works well for files up to a few hundred megabytes. For larger files, use libraries like ijson that parse JSON iteratively, allowing you to work with files that exceed available memory Processing large JSON files in Python without running out of memory.

What Python data types does JSON convert to?

JSON objects (key-value pairs) become Python dictionaries. JSON arrays become Python lists. JSON strings remain strings, numbers become int or float, true/false become True/False, and null

NOTES

Further study materials Lesson 1

Transcript

becomes None. This direct mapping makes working with JSON in Python intuitive.

Do I need to install external packages for JSON?

No. The `json` module is part of Python's standard library and requires no installation. However, for specialized use cases like streaming large files (ijson), faster parsing (orjson), or schema validation (jsonschema), you may choose to install third-party packages.

Why explicitly specify encoding='utf-8'?

The JSON standard recommends UTF-8 encoding Working With JSON Data in Python Real Python, but some systems may default to other encodings. Explicitly setting `encoding='utf-8'` ensures consistent behavior across different computers and operating systems, preventing unexpected decoding errors.

How do I handle malformed JSON from external sources?

Always wrap JSON parsing in try-except blocks to catch `JSONDecodeError`. Validate the structure after parsing and consider using schema validation libraries like `jsonschema` for production applications that consume untrusted JSON data.

What's the best way to debug JSON parsing issues?

Use Python's built-in JSON validation to identify specific syntax errors:

python

```
import json

try:
with open('data.json', 'r', encoding='utf-8') as file:
data = json.load(file)
except json.JSONDecodeError as e:
print(f"Error at line {e.lineno}, column {e.colno}:
{e.msg}")
```

For complex files, use online JSON validators or tools like `jq` to inspect and format JSON before parsing.

Real-World Use Cases and Examples

Use Case 1: Processing API Configuration Files

Many applications store configuration in JSON format. Here's a robust configuration loader:

python

```
import json
from pathlib import Path
```

NOTES

Further study materials Lesson 1

Transcript

```
from typing import Dict, Any

class ConfigLoader:
"""Load and validate application configuration."""

def __init__(self, config_path: str):
self.config_path = Path(config_path)
self.config: Dict[str, Any] = {}

def load(self) -> Dict[str, Any]:
"""Load configuration with error handling."""
if not self.config_path.exists():
raise FileNotFoundError(f"Config file not found:
{self.config_path}")

try:
with self.config_path.open('r', encoding='utf-8') as
file:
self.config = json.load(file)

self._validate_config()
return self.config

except json.JSONDecodeError as e:
raise ValueError(f"Invalid JSON in config file: {e}")

def _validate_config(self):
"""Ensure required configuration keys exist."""
required_keys = ['api_key', 'database_url', 'debug_mode']
missing = [key for key in required_keys if key not in
self.config]

if missing:
raise ValueError(f"Missing required config keys:
{missing}")

# Usage
config_loader = ConfigLoader('config.json')
config = config_loader.load()
print(f"API Key: {config['api_key']}")
```

Use Case 2: Processing Data from REST APIs

When consuming REST APIs, you'll frequently parse JSON responses:

python

```
import json
import urllib.request
from typing import List, Dict

def fetch_github_repos(username: str) -> List[Dict]:
"""Fetch and parse GitHub user repositories."""
url = f"https://api.github.com/users/{username}/repos"

try:
```

NOTES

Further study materials Lesson 1

Transcript

```
with urllib.request.urlopen(url) as response:
data = response.read()
repos = json.loads(data)

# Extract relevant information
repo_info = [
{
'name': repo['name'],
'stars': repo['stargazers_count'],
'language': repo['language']
}
for repo in repos
]

return repo_info

except urllib.error.URLError as e:
print(f"Error fetching data: {e}")
return []
except json.JSONDecodeError as e:
print(f"Error parsing JSON response: {e}")
return []

# Usage
repos = fetch_github_repos('python')
for repo in repos[:5]: # Show first 5
print(f"{repo['name']}: {repo['stars']} stars
({repo['language']})")
```

Use Case 3: Batch Processing JSON Data Files

For data analysis tasks, you often need to process multiple JSON files:

python

```
import json
from pathlib import Path
from typing import Iterator, Dict

def process_json_directory(directory: str) ->
Iterator[Dict]:
"""Process all JSON files in a directory."""
json_dir = Path(directory)

for json_file in json_dir.glob('*.json'):
try:
with json_file.open('r', encoding='utf-8') as file:
data = json.load(file)
yield {
'filename': json_file.name,
'data': data,
'size': json_file.stat().st_size
}
except (json.JSONDecodeError, IOError) as e:
print(f"Error processing {json_file.name}: {e}")
continue
```

NOTES

Further study materials Lesson 1

Transcript

```
# Usage: Process all JSON files and aggregate statistics
for item in process_json_directory('./data'):
print(f"Processed {item['filename']} ({item['size']}
bytes)")
```

Final Thoughts

Mastering JSON file reading in Python is essential for modern development. The `json.load()` function combined with context managers provides the standard, reliable method for parsing file-based data. For string-based JSON, `json.loads()` handles in-memory data efficiently.

By adopting modern practices like the pathlib module, implementing proper error handling, and understanding performance optimization techniques, you can write production-ready code that handles any JSON data source reliably. Whether you're building APIs, processing configuration files, or analyzing data from web services, these techniques form the foundation of professional Python development, often supported by experts in mobile app development

For applications dealing with large files, remember that streaming parsers like ijson provide memory-efficient alternatives to standard loading methods. Always validate your data, handle errors gracefully, and choose the right tool for your specific use case.

NOTES

Further study materials Lesson 1

Content type: Text

Mastering Python File I/O: Text, CSV, and JSON Operations

Full Content

Available at https://medium.com/@azizulraihan/python-file-saga-a-comprehensive-guide-to-reading-writing-appending-text-csv-and-json-ab9e8208753d

Summary

Introduction and Basic File Modes

This part explains the basics of file I/O in Python. The open() function is used to open a file with modes 'r' for reading, 'w' for writing, and 'a' for appending. The 'w' mode creates a new file or truncates an existing one, while 'a' adds data to the end. The article aims to be a one-stop guide for common file operations.

Reading and Writing Text Files

Text files store human-readable text without complex formatting. Python offers methods like .read() to read the whole file and .readlines() to read line by line. For writing, use 'w' mode and .write() to add strings; other data types must be converted using str(). The with statement ensures the file is closed automatically. Example: writing 'Hello\n' then 'World' to output.txt.

Reading and Writing CSV Files

CSV files store tabular data with columns separated by commas. Use csv.reader to read rows as lists, or csv.DictReader to read rows as dictionaries with headers as keys. For writing, use csv.writer to write rows from a 2D list with .writerows(), or csv.DictWriter to write from a list of dictionaries, first writing headers with .writeheader(). Append data by opening in 'a' mode.

Reading and Writing JSON Files

JSON is a text-based format for structured data. Use json.load() to read a JSON file into a Python dictionary or list. For writing, use json.dump() to serialize data to a file, or json.dumps() for a string, often with indent for readability. To append to a JSON file, first read existing data into a list, add new data, then write the updated list back. The json module simplifies these tasks.

Appending Data to Files

Appending adds new data without deleting existing content. For text files, open in 'a' mode and use .write() to add a string, often with a newline. For CSV files, use csv.writer in 'a' mode to append rows. For JSON files, read existing data, convert to a list if needed, append new data, then rewrite the entire file. This ensures the new data is added correctly while preserving the original format.

NOTES

Further study materials Lesson 1

Transcript

Python File Saga: A Comprehensive Guide to Reading, Writing, Appending, Text, CSV, and JSON

[Image: Image from Pexels]

We've all been down that rabbit hole, haven't we? Scouring the web for code snippets to handle file read/write operations whenever the need arises. It was this recurring search that inspired me to create a go-to spot for all the file I/O solutions. So, welcome aboard as we embark on this journey into the world of file handling in Python.

Understanding Python's File I/O Basics

To perform any operation on a file, the first step is to open it. You can use the `open()` function to open a file and specify the mode in which you want to open it. The most commonly used modes are:

- `'r'`: Read mode - Opens the file for reading.
- `'w'`: Write mode - Opens the file for writing (creates a new file if it doesn't exist or truncates the file if it does).
- `'a'`: Append mode - Opens the file for appending data to the end.

There are other advanced modes like `r+`, `w+`, `a+` but we will focus on the above three for now.

Text files

Text files, in their plainest form, are a fundamental means of storing data. They consist of human-readable text devoid of any intricate formatting. Python offers straightforward ways to work with these files, whether your goal is to extract information from them or add new data. In this context, we'll explore the contents of a file housing a collection of renowned quotes.

```
The future belongs to those who believe in the beauty of
their dreams." - Eleanor Roosevelt
"Success is not final, failure is not fatal: It is the
courage to continue that counts." - Winston Churchill
"In the middle of every difficulty lies opportunity." -
Albert Einstein
"Life is really simple, but we insist on making it
complicated." - Confucius
"The only limit to our realization of tomorrow will be
our doubts of today." - Franklin D. Roosevelt
"Don't watch the clock; do what it does. Keep going." -
Sam Levenson
```

Reading the entire file

When it comes to reading the entire content of a file in one fell swoop, Python provides a convenient tool: the `.read()` method. In the following code snippet, we open the file

NOTES

Further study materials Lesson 1

Transcript

`'my-text-file.txt'` in `'r'` mode, signifying that it's for reading.

```
with open("my-text-file.txt", "r") as file:
quotes= file.read()
print(quotes)
file.close()
```

Also, we use a `with` statement to ensure that the file is properly closed after reading its contents.

Read by line

Reading a text file line by line in Python is a straightforward task thanks to the `.readlines()` method. In the code snippet below, we open the file `'my-text-file.txt'` in `'r'` mode for reading.

```
with open("my-text-file.txt", "r") as file:
quotes = file.readlines()
print(quotes)
```

Within this context, we utilize `.readlines()` to read the file line by line, which results in a list called 'quotes,' where each element represents a line from the text file. This approach offers a convenient way to process text files, especially when you need to work with individual lines of content.

[Image: read text file line by line]

Write

When it comes to creating and adding data to text files in Python, the 'w' mode comes into play. In the following code snippet, we open a file named `'output.txt'` with `'w'` mode, indicating that it's intended for writing.

```
with open('output.txt', 'w') as file:
file.write("Hello
")
file.write("World
")
file.write(str([12, 234]))
```

We use `' '` to get to the next line as necessary. And remember you can only write `string` data types to the file. It's essential to note that text files can only store string data types. Therefore, we convert the other data types (i.e., list, dictionary, numbers) to a string using `str()` before writing it to the file.

Append

To append data to a text file in Python, you can open the file in append mode (`'a'`) and then use the `write()` method to add content to the end of the existing file contents. Here's an example:

```
new_data = 'Catch 22'
with open('output.txt', 'a') as file:
```

NOTES

Further study materials Lesson 1

Transcript

```
file.write('
' + new_data)
```

The `output.txt` file now looks like below.

```
Hello
World
[12, 234]
Catch 22
```

CSV files

Working with CSV (Comma-Separated Values) files is a common task in data processing and analysis using Python. CSV files are plain text files used to store tabular data, where each row represents a record, and columns are separated by a delimiter, typically a comma (`,`). Here's a guide on how to read, write, and manipulate CSV files in Python.

```
Name, Age, Country
Alice, 28, USA
Bob, 32, Canada
Carol, 25, UK
```

Read CSV by row

To read a CSV file row by row in Python, you can use the `csv.reader` class from the `csv` module. Each row in the CSV file will be treated as a list of values. Here's how you can read a CSV file row by row:

```
import csv

with open("my-csv-file.csv", 'r') as file:
csv_reader = csv.reader(file, delimiter=',')
for row in csv_reader:
print(row)
```

Read CSV as a dictionary

Reading a CSV file as a dictionary in Python is a common requirement, especially when dealing with structured data where you want to access columns by their header names. You can use the `csv.DictReader` class from the `csv` module to achieve this. Here's how to read a CSV file as a dictionary:

```
from csv import DictReader
with open("my-csv-file.csv", 'r') as file:
dict_reader = DictReader(file)
data_dict = list(dict_reader)
print(data_dict)
```

Write into CSV

To write data into a CSV file in Python, you can create a writer object with the `csv.writer` class from the `csv` module. And then you can write from a 2D list of data with the `.writerows()`

NOTES

Further study materials Lesson 1

Transcript

method. Here's how to write data into a CSV file:

```
import csv

# Data to be written to the CSV file
data = [
['Name', 'Age', 'Country'],
['Alice', 28, 'USA'],
['Bob', 32, 'Canada'],
['Carol', 25, 'UK']
]

with open('output.csv', 'w', newline='') as csv_file:
# Create a CSV writer object
csv_writer = csv.writer(csv_file,delimiter=',')

# Write the data to the CSV file
csv_writer.writerows(data)
```

Write from dictionary

To write data from a dictionary into a CSV file in Python, you can use the `csv.DictWriter` class from the `csv` module. This class allows you to write dictionaries into CSV files while specifying which keys (dictionary keys) correspond to which columns in the CSV. Here's how you can do it:

```
import csv

# Data as dictionaries
data = [
{'Name': 'Alice', 'Age': 28, 'Country': 'USA'},
{'Name': 'Bob', 'Age': 32, 'Country': 'Canada'},
{'Name': 'Carol', 'Age': 25, 'Country': 'UK'}
]

# Specify the CSV file header
fieldnames = ['Name', 'Age', 'Country']

with open('output.csv', 'w', newline='') as csv_file:
# Create a DictWriter object and specify the fieldnames
csv_writer = csv.DictWriter(csv_file,
fieldnames=fieldnames)

# Write the header row
csv_writer.writeheader()

# Write the data from the list of dictionaries
csv_writer.writerows(data)
```

Append to CSV

To append data to an existing CSV file in Python, you can use the `csv.writer` class with the 'a' (append) mode when opening the file. Here's how you can append data to an existing CSV file:

```
import csv
```

NOTES

Further study materials Lesson 1

Transcript

```
# Data to be appended to the CSV file
new_data = [
['David', 30, 'Australia'],
['Eva', 28, 'Germany']
]

# Open the existing CSV file for appending
with open('existing.csv', 'a', newline='') as csv_file:
# Create a CSV writer object
csv_writer = csv.writer(csv_file)

# Write the new data to the CSV file
csv_writer.writerows(new_data)
```

JSON files

JSON is a text-based data interchange format that is easy for both humans and machines to read and write. It is widely used for storing and transmitting structured data, such as configuration settings, web API responses, and more.

```
{
"Name": "Alice",
"Age": "28",
"Country": "USA"
}
```

Read JSON

In Python, the `json` module makes it easy to work with JSON data. We can open the JSON file using the `open()` function and use the `json.load()` function to parse the JSON data. Here is an example below:

```
import json

# Open the JSON file for reading
with open('data.json', 'r') as json_file:
# Parse the JSON data
data = json.load(json_file)

# Now you can work with the data as a Python dictionary
print(data)
```

Write JSON

```
import json

# Data to write into JSON
data = [
{'Name': 'Alice', 'Age': ' 28', 'Country': 'USA'},
{'Name': 'Bob', 'Age': ' 32', 'Country': 'Canada'},
{'Name': 'Carol', 'Age': ' 25', 'Country': 'UK'}
]

# Open JSON file for writing
```

NOTES

Further study materials Lesson 1

Transcript

```
with open('output.json', 'w') as file:
# Write JSON data
file.write(json.dumps(data, indent=4))
```

Append into JSON

Appending data to an existing JSON file in Python involves several steps:

1. Open the Existing JSON File for Reading.
2. If the existing data is not in a List structure, then create a List and Append Existing Data.
3. Append New Data to the List.
4. Open and write to the same JSON file.

```
import json

# Open the existing JSON file for reading
with open('my-json-file.json', 'r') as json_file:
# Load the existing data from the file
existing_data = json.load(json_file)

# Declare empty List
data = []
data.append(existing_data)

# New data to be appended (as a Python dictionary)
new_data = {
"name": "Eva",
"age": 30,
"city": "Berlin"
}

# Append the new data to the newly created list
data.append(new_data)

# Open the same JSON file for writing (this will
overwrite the file)
with open('my-json-file.json', 'w') as json_file:
# Write the updated data structure back to the JSON file
json.dump(data, json_file, indent=4)
```

References

NOTES

Further study materials Lesson 1

Content type: Text

Reading Text, CSV, JSON, XML, and Excel Files in Python

Available at https://medium.com/@nutanbhogendrasharma/python-read-different-type-of-files-4090a6e12c4c

Full Content

Summary

Introduction and Installation

This blog teaches how to read different file types in Python: text, CSV, TSV, JSON, XML, and Excel. It uses built-in modules like csv, json, and xml.etree.ElementTree, which require no installation, and xlrd for Excel files. To install xlrd, run 'pip install xlrd'. Sample files are provided for practice.

Reading Text Files

To read a text file, declare the file path, open it with open(), and use read() to print all content. To read line by line, iterate over the file object. The readlines() method returns a list of all lines. Example code shows how to print each line. This is useful for processing text data line by line.

Reading CSV and TSV Files

Use the csv module to read CSV and TSV files. csv.reader returns each row as a list. csv.DictReader maps each row to a dictionary using column headers. For TSV files, specify delimiter='\t' or use dialect='excel-tab'. Then iterate over the reader to print rows or access specific fields.

Reading JSON and XML Files

JSON files are read using the json module. Use json.load() to parse the file into a Python dictionary or list. For multiple records, iterate over the list. XML files are parsed with xml.etree.ElementTree. Parse the file, get the root, then iterate over child elements or use findall() to find specific tags. Access text and attributes.

Reading Excel Files

Excel files in .xls format are read using the xlrd library. Open the workbook with xlrd.open_workbook(), then select a sheet by index. Use sheet.ncols and sheet.nrows to get dimensions. Print column names with cell_value(0, i). Iterate over rows with row_values(i). Access specific row values as needed.

NOTES

Further study materials Lesson 1

Transcript

Python Read Different Types of Files

In this blog, we will read different types of files like text, CSV, TSV, JSON, XML, and Excel sheets. For that, we will use the `csv`, `json`, `xml.etree.ElementTree`, and `xlrd` modules.

[Image: Photo by Hitesh Choudhary on Unsplash]

Installation

Installation for reading CSV, TSV, JSON, and XML files

CSV is part of Python's standard library, so you don't need to install it with pip.

Python comes with a built-in package called `json` for encoding and decoding JSON data. So no need to install it with pip.

Python's standard library contains the `xml` package. The `xml.etree.ElementTree` module implements a simple and efficient API for parsing and creating XML data. So no installation is required.

Installation for reading Excel sheets

`xlrd` is a library for reading data and formatting information from Excel files in the historical `.xls` format.

```
pip install xlrd
```

Sample files

We have created sample files for this blog. If you want to download, click on the following links:

- sample.txt
- sample.csv
- sample.json
- sample1.json
- sample.xml
- sample.xlsx
- sample.tsv

Read text file

Declare the text file path and open the file. After that, print.

```
txtFile = "input/sample.txt"

with open(txtFile) as f:
lines = f.read()
print(lines)
pass
```

Output:

*[Image: https://miro.medium.com/1*HjXU3xsG983CQnn3V_5lPw.png]*

NOTES

Further study materials Lesson 1

Transcript

Read each line from text file

```
with open(txtFile) as f:
for line in f:
print(line)
pass
pass
```

Output:

*[Image: https://miro.medium.com/1*AFkyZ24NJuzJzUWoyUWlng.png]*

Read file with `readlines()`

`readlines()` returns all lines in the file as a list.

```
with open(txtFile) as f:
lines = f.readlines()
print(lines)
pass
```

Output:

*[Image: https://miro.medium.com/1*vkWifSsUvBQMGganc8B4tQ.png]*

Print each line

```
with open(txtFile) as f:
lines = f.readlines()
#print(lines)
for line in lines:
print(line)
pass
pass
```

Output:

*[Image: https://miro.medium.com/1*2LFyaB53idfOdSWhUCNFvw.png]*

Read CSV file

Import the `csv` module.

```
import csv
```

Help on built-in function `reader` in module `_csv`:

reader(...) csv_reader = reader(iterable [, dialect='excel'] [optional keyword args]) for row in csv_reader: process(row)

The "iterable" argument can be any object that returns a line of input for each iteration, such as a file object or a list. The optional "dialect" parameter is discussed below. The function also accepts optional keyword arguments which override settings provided by the dialect.

The returned object is an iterator. Each iteration returns a row of the CSV file (which can span multiple input lines).

NOTES

Further study materials Lesson 1

Transcript

Declare CSV file path, open, and then read

```
csvFile = "input/sample.csv"

with open(csvFile, newline='') as cf:
csv_reader = csv.reader(cf)
print(csv_reader)
pass
```

_Output: *<csv.reader object at 0x7f0838395cd0>*

Print each line

```
with open(csvFile, newline='') as cf:
csv_reader = csv.reader(cf)
#print(csv_reader)
#read each line and print
for each_row in csv_reader:
print(each_row)
pass
pass
```

Output:

*[Image: https://miro.medium.com/1*f__30tXs7fI7gHHnyjetkA.png]*

Read CSV with `DictReader`

`csv.DictReader(f, fieldnames=None, restkey=None, restval=None, dialect='excel', *args, **kwds)`

Creates an object that operates like a regular reader but maps the information in each row to a dict whose keys are given by the optional `fieldnames` parameter.

```
with open(csvFile, 'r') as cf:
reader = csv.DictReader(cf)
print(reader)
pass
```

Output: <csv.DictReader object at 0x7f0838351290>

```
with open(csvFile, 'r') as cf:
reader = csv.DictReader(cf)
#print(reader)
#print each row
for row in reader:
print(row)
pass
pass
```

Output:

*[Image: https://miro.medium.com/1*ogbsqGeKPvKWeS1VqyX8Qg.png]*

```
with open(csvFile, 'r') as cf:
reader = csv.DictReader(cf)
#print(reader)
#print each row
```

NOTES

Further study materials Lesson 1

Transcript

```
for row in reader:
print(f'First name: {row["Firstname"]} and Last name is
{row["Lastname"]}')
pass
pass
```

Output:

*[Image: https://miro.medium.com/1*QxqZPrQ6RZKMO_DjkLIn8A.png]*

Read JSON file

Import the `json` module.

```
import json
```

Open the JSON file and read.

```
jsonFile = 'input/sample.json'
```

```
with open(jsonFile) as jf:
jsonData = json.load(jf)
print(jsonData)
pass
```

Output:

*[Image: https://miro.medium.com/1*UuUQp0Cp7o62HeHsPg5kpA.png]*

```
with open(jsonFile) as jf:
jsonData = json.load(jf)
#print(jsonData)
print('id: ' + str(jsonData['id']))
print('name: ' + str(jsonData['name']))
print('subject: ' + str(jsonData['subject']))
pass
```

Output:

*[Image: https://miro.medium.com/1*qFlQ8AMJ6TgAUjHIxABr6w.png]*

Read another JSON file with multiple records

```
jsonFile = 'input/sample1.json'
```

```
with open(jsonFile) as jf1:
jsonData = json.load(jf1)
print(jsonData)
pass
```

Output:

*[Image: https://miro.medium.com/1*nZ7hKZ36JzoQJuHAl4fDKg.png]*

```
with open(jsonFile) as jf1:
jsonData = json.load(jf1)
#print(jsonData)
for s in jsonData:
print(f'
id: {s["id"]}')
```

NOTES

Further study materials Lesson 1

Transcript

```
print(f'name: {s["name"]}')
print(f'subject: {s["subject"]}')
pass
pass
```

Output:

*[Image: https://miro.medium.com/1*J8L4-T3AJIRKtraIYZHKUA.png]*

Read XML

```
import xml.etree.ElementTree as ET
```

Parse XML file

```
xmlFile = 'input/sample.xml'

tree = ET.parse(xmlFile)

#get the root of xml file
root = tree.getroot()
print(root)
```

Output: <Element 'data' at 0x7f08382efa70>

Print child tag and its attributes

```
tree = ET.parse(xmlFile)
#get the root of xml file
root = tree.getroot()
#print(root)
#print child tag and it's attributes
for child in root:
print(child.tag)
print(child.attrib)
print("
")
pass
```

Output:

*[Image: https://miro.medium.com/1*7qOHHXwEJXLZVPitlVYFUg.png]*

Print the children 'neighbor' of root node

```
tree = ET.parse(xmlFile)
#get the root of xml file
root = tree.getroot()
#print(root)
#print child tag and it's attributes
for child in root:
print(child.tag)
print(child.attrib)
print("
")
pass
#print the children 'neighbor' of root node
for neighbor in root.iter('neighbor'):
print(neighbor.attrib)
```

NOTES

Further study materials Lesson 1

Transcript

```
print("
")
pass
```

Output:

*[Image: https://miro.medium.com/1*U0zW2kQ3RE3sYhXaF7TsyQ.png]*

Print the children 'year' of root node

```
tree = ET.parse(xmlFile)
#get the root of xml file
root = tree.getroot()
#print(root)
#print child tag and it's attributes
for child in root:
print(child.tag)
print(child.attrib)
print("
")
pass
#print the children 'neighbor' of root node
for neighbor in root.iter('neighbor'):
print(neighbor.attrib)
print("
")
pass
#print the children 'year' of root node
for year in root.iter('year'):
print(int(year.text))
print("
")
pass
```

Output:

*[Image: https://miro.medium.com/1*8_Bg9uzQrub03kji-aO-8Q.png]*

Find all countries

```
tree = ET.parse(xmlFile)
#get the root of xml file
root = tree.getroot()
#print(root)
#print child tag and it's attributes
for child in root:
print(child.tag)
print(child.attrib)
print("
")
pass
#print the children 'neighbor' of root node
for neighbor in root.iter('neighbor'):
print(neighbor.attrib)
print("
")
pass
```

NOTES

Further study materials Lesson 1

Transcript

```
#print the children 'year' of root node
for year in root.iter('year'):
print(int(year.text))
print("
")
pass
#find all the countries
for country in root.findall('country'):
rank = int(country.find('rank').text)
print("Country ranks are: ")
print(rank)
print(country.attrib)
pass
```

Output:

*[Image: https://miro.medium.com/1*ALuKEnD0UmxQYuBE9pv-aw.png]*

Read Excel file

```
import xlrd
```

Open workbook and take first sheet.

```
excelFile = "input/sample.xlsx"

wb = xlrd.open_workbook(excelFile)

sheet = wb.sheet_by_index(0)
print(sheet)
```

Output: <xlrd.sheet.Sheet object at 0x7f0809e62b10>

Print the number of columns in the Excel sheet

```
print(sheet.ncols)
```

Output: 4

Print the column names

```
for i in range(sheet.ncols):
print(sheet.cell_value(0, i))
pass
```

Output:

*[Image: https://miro.medium.com/1*nPR5S5FlTM_j1bQ1_O_TRA.png]*

Print the number of rows in the Excel sheet

```
print(sheet.nrows)
```

Output: 6

Print the row values

```
for i in range(sheet.nrows):
print(sheet.row_values(i))
pass
```

NOTES

Further study materials Lesson 1

Transcript

Output:

*[Image: https://miro.medium.com/1*SfgEW_mzAVCB3M3pr6M1yA.png]*

Print specific row values

```
print(sheet.row_values(1))
```

Output:

*[Image: https://miro.medium.com/1*uY169vg9nLshglZ_aXbN2w.png]*

Read TSV file

```
import csv
```

Open the TSV file and read.

```
tsvFile = "input/sample.tsv"
```

```
with open(tsvFile) as tf:
tsv_reader = csv.reader(tf, delimiter="\t")
#print each row
for row in tsv_reader:
print(row)
pass
pass
```

Output:

*[Image: https://miro.medium.com/1*z-xxlj8CklAZLrvjdGNrvA.png]*

Read TSV file with `DictReader()`

```
with open(tsvFile) as tf:
#read tsv file with DictReader()
tsv_reader = csv.DictReader(tf, dialect='excel-tab')
#iterate tsv_reader
for row in tsv_reader:
#print each row
print(row)
pass
pass
```

Output:

*[Image: https://miro.medium.com/1*zzdTuBX5CYaVywmc8MoGDA.png]*

```
with open(tsvFile) as tf:
#read tsv file with DictReader()
tsv_reader = csv.DictReader(tf, dialect='excel-tab')
#iterate tsv_reader
for row in tsv_reader:
#print each row
#print(row)
#print row with key
print(row['SrNo'], row['Username'])
pass
pass
```

NOTES

Further study materials Lesson 1

Transcript

Output:

*[Image: https://miro.medium.com/1*VLvVF61_ZRyVZo_HbeviGQ.png]*

That's it. Thanks for reading.

NOTES

Exercises • Lesson 1

EXERCISE 1 • True or false

CONTEXT

Flet provides the FilePicker control for selecting files and standard Python file I/O for reading and writing data.

QUESTION • Indicate whether the information is true or false

To read a JSON file in Flet, you must use json.load() after opening the file.

◯ TRUE ◯ FALSE

NOTES

Exercises • Lesson 1

EXERCISE 2 • Multiple choice

CONTEXT

In Flet, when you want to let users select a file from their device, you need to set up a FilePicker control properly.

QUESTION • Select the correct option

What is a required step before a FilePicker can open a native dialog?

A. Use the pick_files() method immediately after creating the FilePicker.

B. Add the FilePicker to page.overlay.

C. Call page.update() after creating the FilePicker.

D. Define an on_result callback for the FilePicker.

NOTES

Exercises • Lesson 1

EXERCISE 3 • True or false

CONTEXT

Python provides built-in functions to parse JSON data from various sources.

QUESTION • Indicate whether the information is true or false

The json.load() function is used to parse JSON data from a string.

◯ TRUE ◯ FALSE

NOTES

Exercises • Lesson 1

EXERCISE 4 • Multiple choice

CONTEXT

When working with JSON data in Python, you encounter two main parsing functions: one for file objects and one for strings. Choosing the correct function is essential for proper data handling.

QUESTION • Select the correct option

Which function should you use to parse JSON data that is already stored in a string variable?

A json.loads()

B json.dump()

C json.load()

D json.dumps()

NOTES

Exercises • Lesson 1

EXERCISE 5 • True or false

CONTEXT

Python provides different modes for file operations. The 'w' mode is used for writing to a text file.

QUESTION • Indicate whether the information is true or false

When writing to a text file in Python using the 'w' mode, you can directly write integers without converting them to strings.

◯ TRUE ◯ FALSE

NOTES

Exercises • Lesson 1

EXERCISE 6 • Multiple choice

CONTEXT

In Python file handling, reading a text file can be done in different ways, such as reading the entire content at once or reading line by line.

QUESTION • Select the correct option

What does the .readlines() method return when applied to a file object opened in read mode?

A. A generator object that yields lines one by one

B. A tuple of strings, each representing a line of the file

C. A list of strings, each representing a line of the file

D. A single string containing the entire file content

NOTES

Exercises • Lesson 1

EXERCISE 7 • True or false

CONTEXT

In Python, you can read different types of files using various modules. Some modules are included in the standard library, while others need to be installed separately.

QUESTION • Indicate whether the information is true or false

The xlrd module is part of Python's standard library and does not require installation with pip.

◯ TRUE ◯ FALSE

NOTES

Exercises • Lesson 1

EXERCISE 8 • Multiple choice

CONTEXT

The csv module provides two ways to read CSV files: csv.reader and csv.DictReader. Understanding their output formats is important for data processing.

QUESTION • Select the correct option

When using csv.DictReader to read a CSV file, what data type does each row represent?

A) Each row is a tuple of values.

B) Each row is a set of column values.

C) Each row is a dictionary with column names as keys.

D) Each row is a list of strings.

NOTES

Exercise Answers

LESSON 1

EX.01 • True or false

True.

json.load() parses the file content into a Python dictionary or list, which is the correct way to read JSON files in Flet.

I got it right I got it wrong

EX.02 • Multiple choice

Option B — "Add the FilePicker to page.overlay."

The lesson states that if you skip adding the FilePicker to page.overlay, nothing happens when you click the button. This is the most common setup mistake, and it is essential for the dialog to appear.

I got it right I got it wrong

EX.03 • True or false

False.

json.load() is designed for reading JSON from a file object, not from a string. To parse JSON from a string, you should use json.loads().

I got it right I got it wrong

EX.04 • Multiple choice

Option A — "json.loads()"

The json.loads() function (with 's' for string) is designed to parse JSON from a string. It accepts a string argument and returns a Python object.

I got it right I got it wrong

EX.05 • True or false

False.

Text files can only store string data types. Integers must be converted using str() before writing, as shown in the lesson.

I got it right I got it wrong

EX.06 • Multiple choice

Option C — "A list of strings, each representing a line of the file"

The .readlines() method reads all lines from the file and returns them as a list, where each element is a string containing one line, including the newline character.

I got it right I got it wrong

Exercise Answers

LESSON 1

EX.07 • True or false

False.

xlrd is a third-party library for reading Excel .xls files. Unlike csv, json, and xml.etree.ElementTree, xlrd is not in the standard library and must be installed using pip.

I got it right I got it wrong

EX.08 • Multiple choice

Option C — "Each row is a dictionary with column names as keys."

csv.DictReader maps each row to a dictionary where keys are the column headers from the first row, and values are the cell values. This allows accessing fields by name.

I got it right I got it wrong

Chapter 06 • Async Programming and Data Integration

Lesson 02 • Fetching Data with httpx

36 pages • 55 min

elevify

Lesson 02 • Fetching Data with httpx

55 min • 36 pages

◎ What you will learn

Uses the httpx library to make async HTTP GET and POST requests. Fetched data is then rendered into Flet controls in subsequent sections.

Summary	7 min	**55**
Transcript	7 min	**56**
Further study materials		
Text • Mastering HTTPX for Asynchronous HTTP with HTTP/2 and Superior Performance	12 min	**59**
Text • Building Robust HTTP Retry Policies with httpx and Pydantic Validation	35 min	**66**
Exercises		**84**
Answer key		**90**

Fetching Data with httpx

Summary

Introduction to Async HTTP in Flet

In this lesson, you will learn how to fetch live data from the internet and display it in a Flet app. We use httpx, an async-friendly HTTP client for Python. Synchronous HTTP calls freeze the entire interface, making buttons unresponsive. Async requests solve this by keeping the event loop active, allowing the app to remain responsive while data is fetched in the background. This is essential for any app needing up-to-date information without freezing the UI.

Installing and Setting Up httpx

Installing httpx is simple: run 'pip install httpx' in the terminal. Unlike the older requests library, httpx has built-in async support, so no need for extra wrappers like aiohttp. After installation, import both flet and httpx. The key component is httpx.AsyncClient, which manages connection pooling automatically, saving resources and improving speed. Use it inside async functions for efficient HTTP handling in Flet's async environment.

Async GET Request Pattern

Every async GET request follows a four-step pattern. First, define the function with 'async def'. Second, open an AsyncClient using 'async with' context manager. Third, await the GET call, which suspends the function until the server responds without blocking anything else. Fourth, use the response to parse data and update Flet controls. A minimal example: the function is async def, inside it open AsyncClient, await a GET request, and the rest of the app runs normally.

Parsing JSON and Updating UI

Once the response arrives, parsing JSON is done with response.json(), which returns a Python dictionary or list. Access fields using dict keys, assign values to Flet controls, then call page.update() to refresh the screen. Before the request, controls show placeholder text. After await completes, update control values and call page.update(). Flet re-renders only changed elements, giving a smooth transition without flicker or full reload.

Error Handling and Common Pitfalls

Handle errors gracefully by calling raise_for_status on the response. This raises an HTTPStatusError for 4xx or 5xx codes. Catch it with try-except and show a friendly error message in the UI. Also handle RequestError for network failures like timeouts. A common mistake is forgetting 'await', which returns a coroutine object instead of a response. Always await async calls. To improve user experience, show a loading spinner (ProgressRing) before the request and hide it after completion.

NOTES

Fetching Data with httpx

Transcript

Introduction

In this lesson, you'll learn how to pull live data from the internet and show it inside a Flet app. We'll use httpx, an async-friendly HTTP client for Python. You'll make GET requests, parse JSON responses, and handle errors — all without freezing your UI. This is essential for any app that needs up-to-date information. I'll guide you through each step, from making the request to displaying the data in your app.

A synchronous HTTP call halts Python until the reply arrives. In Flet, that freezes your whole interface — buttons go dead, the window locks up, and users are left staring at a blank screen. Async requests fix that: the event loop keeps ticking, fetching data in the background while your app stays responsive. That’s exactly why we use async here.

Installing httpx

Installing httpx is a single command. Just open your terminal, type `pip install httpx`, hit enter, and you're done. Unlike the older requests library, httpx comes with async support built right in — no need for a separate async wrapper like aiohttp. No extra configuration steps. After that, you're all set to import it.

Importing and Setting Up

Start your file by importing flet and httpx. The critical component is `httpx.AsyncClient`, the async-compatible session object. Use it inside async functions. It manages connection pooling automatically — no new socket per request, saving resources and speed. This is the efficient way to handle HTTP in Flet's async environment. Make sure you import both at the top.

The Async GET Request Pattern

Every async GET request in httpx follows a simple four-step pattern. First, declare your function with `async def` — that's what enables async behavior. Second, open an `AsyncClient` using the `async with` context manager. It handles connection pooling and ensures cleanup. Third, `await` the GET call. The function suspends until the server responds, blocking nothing else. Fourth, use the response to parse the data and update your Flet controls.

Here's a minimal working example. The function is an `async def`, so Flet can await it when a button is pressed. Inside, we open an `AsyncClient` and await a GET request to a URL. The response comes back as soon as the server replies. The rest of the app kept running the whole time. That's the foundation for more complex requests.

Parsing JSON and Updating the UI

Once the response arrives, parsing the JSON takes a single method

NOTES

Fetching Data with httpx

Transcript

call. `response.json()` decodes the body and returns a Python dictionary or list, depending on the API. Then access the fields using regular dict keys -- `data["title"]` or `data["id"]`. Assign those values to your Flet controls and call `page.update()` to update the screen. Three steps, no extra libraries needed.

Before your request is sent, the controls display placeholder text. After the `await` operation finishes, you update the control values with the parsed data and then call `page.update()`. Flet re-renders only the elements that actually changed—the transition is smooth, no flicker, no full page reload. The user sees the real content appear seamlessly.

Handling Errors Gracefully

Not every request succeeds. The cleanest way to handle failures is to call `raise_for_status` on the response. If the server returns a 4xx or 5xx status code, that method raises an `HTTPStatusError`. You catch it in a `try-except` block and then show a friendly error message in your UI, maybe a red Flet `Text` control or a `SnackBar`. This prevents your app from crashing silently and provides the user with useful feedback.

You need to handle two distinct error categories. `HTTPStatusError`: the server replied, but the status code was bad. `RequestError`: the request never completed – a timeout or a DNS failure, for example. Catching each separately lets you give the user a precise message: either the server had a problem, or the network itself was unreachable.

In production, use this full pattern. The `try` block sends the request and calls `raise_for_status`. The first `except` catches HTTP status errors and shows the status code to the user. The second `except` catches network errors like timeouts. Both branches update a status `Text` control and call `page.update()`. This single pattern covers most real-world failures.

Common Pitfall: Missing `await`

Most common beginner mistake: forgetting `await`. Python doesn't error—it returns a coroutine object, not a response. Your code runs on garbage data. The app looks like it ran but nothing updates. Always `await` every async call. When something's broken, check for a missing `await` first.

Wiring the request to a button is straightforward. Simply pass your async function directly to the `on_click` parameter of an `ElevatedButton`. Flet detects that the handler is a coroutine and automatically awaits it. No threads, executors, or any special setup needed. After the `await` finishes, you update your controls, call `page.update()`, and the data appears on screen. It's that simple.

NOTES

Fetching Data with httpx

Transcript

Adding a Loading Spinner

Always let users know something's happening. Before you `await` the request, show a `ProgressRing` and call `page.update()`. After the `await` completes, hide the spinner, show the data, and call `page.update()` again. Those two extra lines separate a polished app from a broken one.

Summary and Next Steps

You installed httpx and learned why `AsyncClient` fits Flet. You built async GET requests using the four-step pattern. Parsed JSON with one method call, then pushed data into controls. And you added error handling for server errors and network failures. Four skills — everything to connect any Flet app to a REST API.

You can now fetch a single resource and display it in your Flet app. Next, you'll go further—fetching arrays of records from an API and rendering them dynamically inside a Flet `ListView`. You'll build each row programmatically and handle pagination. The async and error handling patterns you just learned apply directly, so you're already most of the way there. All you need is a loop and a `ListView` to display them.

NOTES

Further study materials Lesson 2

Content type: Text

Mastering HTTPX for Asynchronous HTTP with HTTP/2 and Superior Performance

Available at https://medium.com/@think-data/have-you-heard-of-this-powerful-alternative-to-requests-in-python-2f74cfdf6551

Full Content

Summary

Introducing HTTPX: A New Python HTTP Library

The text introduces HTTPX as a powerful alternative to the popular Requests library in Python. It explains that while Requests has been the go-to for HTTP requests due to its simplicity and wide support, HTTPX offers modern features like async support and HTTP/2. The article aims to break down the differences and help readers choose the right tool for their needs, with practical examples and use cases.

Requests: The Synchronous Classic

Requests is a classic synchronous HTTP library known for its simplicity. It handles cookies, sessions, authentication, and JSON easily. Basic usage is straightforward, as shown with GET and POST examples. However, its main limitation is blocking execution: each request waits for the previous one to finish, causing delays when making multiple calls. For instance, fetching three APIs sequentially takes the sum of their response times.

HTTPX: Async and Modern Features

HTTPX is a modern alternative that supports both synchronous and asynchronous requests, making it faster by allowing parallel execution. It also supports HTTP/2 and has efficient connection pooling. Synchronous usage is similar to Requests, but the real power is async mode. Using asyncio, multiple requests can be sent simultaneously, reducing total time. An example shows how fetching three APIs in parallel completes in the time of the slowest response.

Requests vs HTTPX: Feature Comparison

The article compares key features of Requests and HTTPX in a table. HTTPX supports both sync and async, while Requests is only sync. HTTPX offers better performance with async and connection pooling, plus HTTP/2 support. Both support streaming and retries. Async parallel requests are only possible with HTTPX. A rule of thumb: use Requests for simple sync tasks, and HTTPX for async or high-concurrency applications.

NOTES

Further study materials Lesson 2

Summary

Performance Test and Final Recommendations

A performance test measures time to fetch five URLs: Requests takes about 5 seconds (synchronous), while HTTPX async finishes in around 1 second. The final thoughts recommend Requests for quick scripts and HTTPX for high performance and concurrency. The article encourages trying HTTPX's AsyncClient and engaging with the author via comments. It also promotes following for more data engineering content.

NOTES

Further study materials Lesson 2

Transcript

Have You Heard of This Powerful Alternative to Requests in Python?

If you've been working with Python for a while, you've probably used the **Requests** library to fetch data from an API or send an HTTP request. It's been the go-to library for HTTP requests in Python for years. But recently, a newer, more powerful alternative has emerged: **HTTPX**.

[Image: Photo by Denley Photography on Unsplash]

So, what's the difference? When should you use `requests`, and when should you switch to `httpx`? If you're confused, don't worry—I've got you covered. In this blog, we'll break it all down with clear explanations, practical examples, and real-world use cases.

By the end, you'll not only understand the key differences between `requests` and `httpx`, but you'll also be able to **choose the right tool for the job** and even explain it to others! Let's dive in. ?

1?❑ Requests: The Classic HTTP Library

Before we talk about `httpx`, let's first understand why `requests` became so popular in the first place.

Why is Requests So Popular?

Python's `requests` library is well-loved because:

- ? It's simple and easy to use.
- ? It handles cookies, sessions, authentication, and headers automatically.
- ? It has built-in support for JSON handling.
- ? It's widely used, meaning tons of documentation and community support.

Basic Usage of Requests

Here's how you can use `requests` to make a **GET** request:

```
import requests

response =
requests.get("https://jsonplaceholder.typicode.com/posts/
1")
print(response.json()) # Print the JSON response
```

And here's how you send a **POST** request:

```
import requests

data = {"title": "New Post", "body": "This is the body.",
```

NOTES

Further study materials Lesson 2

Transcript

```
"userId": 1}
response =
requests.post("https://jsonplaceholder.typicode.com/posts
", json=data)
print(response.status_code) # 201 (Created)
print(response.json()) # Response body
```

? **Simple, right?** That's why `requests` is so widely used—it just works!

However, there's a catch...

2?❑ The Problem with Requests: Blocking Execution

One major limitation of `requests` is that it's **synchronous**.

This means **each request blocks the execution of your program** until it gets a response. If you're making multiple requests (e.g., fetching data from multiple APIs), they will run **one after another**, causing delays.

Imagine you need to fetch data from three different APIs:

```
import requests

urls = [
"https://jsonplaceholder.typicode.com/posts/1",
"https://jsonplaceholder.typicode.com/posts/2",
"https://jsonplaceholder.typicode.com/posts/3"
]
for url in urls:
response = requests.get(url)
print(response.json())
```

This approach **blocks execution**, meaning each request **waits** for the previous one to finish before starting. If each API takes **2 seconds** to respond, the total time is **6 seconds**. ?

This is where `httpx` comes in.

3?❑ HTTPX: The Modern Alternative

Why Use HTTPX?

The `httpx` library is like `requests` but with **superpowers**.

- ? **It supports both synchronous and asynchronous requests.**
- ⚡ **It's faster because it can send requests in parallel.**
- ? **It supports HTTP/2 for better performance.**
- ? **It has built-in connection pooling for efficient networking.**

NOTES

Further study materials Lesson 2

Transcript

In short, if you need better performance and modern features, `httpx` is the way to go!

Basic Usage of HTTPX (Synchronous Mode)

If you want to use `httpx` just like `requests`, you can!

```
import httpx

response =
httpx.get("https://jsonplaceholder.typicode.com/posts/1")
print(response.json()) # Print the JSON response
```

Pretty much identical, right? But the real power of `httpx` comes when you **go async**.

4?❑ The Game Changer: Asynchronous Requests in HTTPX

With `httpx`, you can send requests asynchronously using **asyncio**. This allows multiple requests to be sent at the same time, instead of waiting for each one to finish before sending the next.

Example: Making Asynchronous Requests with HTTPX

```
import httpx
import asyncio

async def fetch_data(url):
async with httpx.AsyncClient() as client:
response = await client.get(url)
return response.json()

async def main():
urls = [
"https://jsonplaceholder.typicode.com/posts/1",
"https://jsonplaceholder.typicode.com/posts/2",
"https://jsonplaceholder.typicode.com/posts/3"
]

tasks = [fetch_data(url) for url in urls]
responses = await asyncio.gather(*tasks)

for res in responses:
print(res)

asyncio.run(main())
```

? **What's happening here?** Instead of making requests one by one, `asyncio.gather()` sends them **all at once**, drastically reducing the total execution time!

? **If each request takes 2 seconds, this will complete in just 2 seconds instead of 6!**

NOTES

Further study materials Lesson 2

Transcript

5?❑ Key Differences: Requests vs HTTPX

```
-------------------------------------------------------
-----------------------------------------
| Feature | requests | httpx |
|-------------------------|-----------------------------
----|-----------------------------------------|
| Sync & Async Support | ? Only synchronous | ? Supports
both |
| Performance | ?♂? Slower (blocking I/O) | ? Faster
(async & connection pooling) |
| HTTP/2 Support | ? No | ? Yes |
| Streaming Responses | ? Yes | ? Yes |
| Connection Pooling | ? Yes (via Session) | ? More
efficient |
| Async Parallel Requests | ? No | ? Yes |
| Retries | ? Yes (via `requests.adapters`) | ? Yes
(built-in) |
| Python Versions | ? 3.x | ? 3.6+ |
-------------------------------------------------------
-----------------------------------------
```

6?❑ When Should You Use Requests vs HTTPX?

```
-------------------------------------------------------
---------------------
| Scenario | Use requests | Use httpx |
|--------------------------------------------------|-----
---------|-----------|
| Simple API calls | ? | ? |
| Asynchronous (non-blocking) requests | ? | ? |
| Fetching data from multiple sources concurrently | ? |
? |
| Need HTTP/2 support | ? | ? |
| Simple authentication & headers | ? | ? |
| Scraping websites | ? | ? |
| Large-scale applications with high concurrency | ? | ?
|
-------------------------------------------------------
---------------------
```

Rule of Thumb

- Use `requests` if you need a **simple, synchronous** HTTP client.
- Use `httpx` if you need **asynchronous capabilities** or **better performance** in large-scale applications.

" Measure the time taken to fetch data from multiple URLs using both `requests` *and* `httpx` *(async).*

```
import time
import requests
import httpx
```

NOTES

Further study materials Lesson 2

Transcript

```
URLS = ["https://jsonplaceholder.typicode.com/posts/1"] *
5

# Using requests (Synchronous)
start = time.time()
for url in URLS:
requests.get(url)
print("Requests Time:", time.time() - start)

# Using httpx (Asynchronous)
import asyncio

async def fetch_all():
async with httpx.AsyncClient() as client:
tasks = [client.get(url) for url in URLS]
await asyncio.gather(*tasks)

start = time.time()
asyncio.run(fetch_all())
print("HTTPX (Async) Time:", time.time() - start)
```

7?❑ Final Thoughts

- If you're building a **quick script**, `requests` is fine.
- If you need **high performance & concurrency**, `httpx` is the better choice.
- Async programming is the future, and `httpx` prepares you for that shift.

? **Want to level up?** Try using `httpx.AsyncClient()` in your next project!

Let me know in the comments if you have any questions or need more examples! ?

> *" Have you tried* `httpx`*? What's your experience? Drop your thoughts in the comments!*

If you are an aspiring Data Engineer or a Data Engineer trying to add more weight to your skill bag, or even if you are interested in topics like this, please do hit the Follow ☞ and Clap ? show your support. It might not be much, but it definitely boosts my confidence to pump out more use case–based content on different Data Engineering tools.

Thank You ? for Reading!

NOTES

Further study materials Lesson 2

Content type: Text

Building Robust HTTP Retry Policies with httpx and Pydantic Validation

Full Content

Available at https://medium.com/@raell.dottin/handling-http-retries-with-httpx-in-python-091e022732a4

Summary

Why Retry Logic Matters in HTTP Clients

Many students write simple HTTP client functions that work fine in scripts but fail silently in production. When a function is reused in scheduled jobs or workers, failures like timeouts, 502, 503, or 429 become common. Not all errors should be retried. Bad payloads or wrong credentials won't improve with retries. But temporary failures like timeout or rate limit can be handled by waiting and trying again. The key is to carefully choose which failures to retry.

Starting with a Basic Async Client

A simple async function using httpx opens a client, sends a GET request, checks status, and returns JSON. For a one-off script, this is fine. But problems start when nobody watches it. A single timeout can fail the whole job. Also, a new AsyncClient is created each time, which is inefficient for repeated calls. The function does not handle retries, backoff, or which errors to retry. It leaves important choices implicit.

Selecting Which Errors to Retry and Safe Methods

Start with a small set of temporary status codes: 408, 429, 500, 502, 503, 504. Only retry these if they are likely to be temporary. Avoid retrying 400, 401, or most 404s. Also, limit retries to safe HTTP methods like GET, HEAD, OPTIONS. POST requests can cause duplicate work if the server received the request but the response was lost. Retry unsafe methods only if the API supports idempotency keys.

Backoff, Retry-After Header, and Settings

Use exponential backoff to avoid overwhelming an already struggling service. For example, wait 0.5s, then 1s, then 2s, up to a max delay. Always respect the Retry-After header when the server sends it. Use asyncio.sleep in async code to avoid blocking the event loop. Validate settings with Pydantic to reject bad values like negative delays or zero attempts early.

Full Client Example with Validation and Testing

A complete ApiClient wraps the retry logic, manages the httpx client lifetime, and validates responses with Pydantic models. The

NOTES

Further study materials Lesson 2

Summary

client uses async context managers for proper cleanup. Testing is easier because delay calculations and retry behavior can be tested with a fake sleep function and MockTransport. This pattern makes failure behavior explicit, reviewable, and maintainable.

NOTES

Further study materials Lesson 2

Transcript

Handling HTTP Retries with httpx in Python

[Image: Photo by Anton Savinov on Unsplash]

> *" **Read the full article on Medium: ***

Introduction

I've written the small version of this code plenty of times.

Open an HTTP client, send a request, check the response, return the JSON. The function is short enough to understand at a glance, and that feels good. Short code has a way of making the problem look smaller than it is.

Then the function gets reused.

It moves from a script into a scheduled job. It ends up in a worker. Someone runs it from an internal tool and quietly starts depending on it. That's usually when the dull failures begin to matter. One request times out. A gateway returns `502`. The API gives a few `503` responses during a deploy. A batch job runs too eagerly and gets a `429`.

When I first started writing clients like this, I treated those failures too evenly. An error was an error. The code either failed immediately or retried in a way that was more hopeful than careful. After living with enough little API clients, I've come to prefer a slower first move: write down what kind of failure you're actually dealing with.

Some failures are worth retrying. Some are just your code being wrong twice.

A malformed payload won't improve because you send it again. Bad credentials won't become valid after a second attempt. A missing resource usually stays missing, unless the API has delayed creation or replication lag that you've actually seen and chosen to handle. A timeout, a rate-limit response, or a short run of gateway errors is different. Those are the cases where waiting briefly and trying again can be reasonable.

Start with the small version

Here's the kind of function people often write first:

```
import httpx


async def fetch_user(user_id: int) -> dict:
async with httpx.AsyncClient() as client:
response = await client.get(
f"https://api.example.com/users/{user_id}",
timeout=10,
)
```

NOTES

Further study materials Lesson 2

Transcript

```
response.raise_for_status()
return response.json()
```

This is a perfectly decent first version. The shape is obvious. The function opens an async client, makes the request, checks the HTTP status, and returns the decoded JSON.

For a throwaway script, I'd probably stop there.

The problems start when the function has to run without someone watching it. A single timeout can fail the whole job. A short upstream deploy can look like your program broke. A rate-limit response may be asking the client to slow down, while this function only knows how to raise and leave.

There's another quiet issue too: this function creates a new `AsyncClient` every time. That's fine for a tiny example, but repeated API work usually deserves a client with a visible lifetime. The client owns connection pooling and shared configuration, so the caller should know when it gets created and when it gets closed.

That doesn't mean you need a large client library. It means the code needs answers that the small version doesn't provide. Which failures should get another attempt? Which HTTP methods are safe to repeat? How many attempts are allowed? How long can one attempt wait? What happens if the response says `200`, but the JSON is shaped wrong?

Those questions are already sitting inside the program. The short version just leaves them implicit.

Pick the failures you'll retry

I like starting with a small list, because a small list forces you to think.

```
TEMPORARY_STATUS_CODES = {
408, # Request Timeout
429, # Too Many Requests
500, # Internal Server Error
502, # Bad Gateway
503, # Service Unavailable
504, # Gateway Timeout
}

def is_temporary_status(status_code: int) -> bool:
return status_code in TEMPORARY_STATUS_CODES
```

This isn't a list to paste into every program without looking at the API in front of you. It's a starting policy.

429 often means the client is moving too fast. `503` often means the upstream service is unavailable for a while. `502` and `504` are

NOTES

Further study materials Lesson 2

Transcript

often gateway-shaped failures, and another attempt may work once the path clears.

A `400` is usually your request being bad. A `401` usually points to credentials or authentication setup. Retrying either one tends to add noise. A `404` is the annoying one because it depends on the service. In most APIs, missing means missing. In some APIs, a resource may show up a few seconds after creation. I wouldn't retry `404` unless I had a concrete reason.

The retry helper should retry because the failure matches a choice you made, not because the code got scared by an exception.

Retry only methods that are safe to repeat

Status codes are only part of the decision. The HTTP method matters too.

Retrying a `GET` is usually safer than retrying a `POST`, because `GET` is supposed to read data. A request that creates a charge, sends a message, opens a ticket, or starts a job can cause duplicate work if the first attempt reached the server but the response was lost.

For a conservative client, start with read-like methods:

```
RETRYABLE_METHODS = frozenset({"GET", "HEAD", "OPTIONS"})
```

This doesn't mean `POST` can never be retried. It means you shouldn't retry it casually. If an API supports idempotency keys, or if the operation is explicitly safe to repeat, then retrying a mutating request may be correct. The code should show that choice.

This is one place where retry examples often get too loose. Retrying "the request" sounds harmless until the request changes state.

Give retry settings a checked shape

Retry settings are configuration, and configuration has a habit of getting changed under pressure.

Someone lowers the delay for a local test. Someone copies settings into another script. Someone sets `max_attempts` to `0` without thinking through what that means. You don't need a large configuration system for this, but you do want obvious bad values rejected early.

Pydantic is useful here.

```
from pydantic import BaseModel, Field, field_validator


class RetrySettings(BaseModel):
max_attempts: int = Field(default=3, ge=1, le=10)
base_delay: float = Field(default=0.5, ge=0)
```

NOTES

Further study materials Lesson 2

Transcript

```
max_delay: float = Field(default=8.0, ge=0)
retryable_status_codes: frozenset[int] = Field(
default_factory=lambda: TEMPORARY_STATUS_CODES
)
retryable_methods: frozenset[str] = Field(
default_factory=lambda: RETRYABLE_METHODS
)

@field_validator("retryable_methods", mode="before")
@classmethod
def normalize_retryable_methods(cls, value: object) ->
frozenset[str]:
if isinstance(value, str):
return frozenset({value.upper()})

return frozenset(str(method).upper() for method in value)
```

Now the retry helper can rely on a few basic facts. There's at least one attempt. There aren't a hundred attempts because someone typed an extra zero. The delay values aren't negative. The allowed methods are normalized to uppercase.

That's not a lot of structure. It's just enough to keep retry behavior from becoming loose numbers scattered through a few call sites.

```
settings = RetrySettings(
max_attempts=3,
base_delay=0.5,
max_delay=8.0,
)
```

Wait a little longer each time

A retry loop that fires immediately can be worse than no retry loop at all.

If the upstream service is struggling, tight retries add pressure. If the server has rate-limited you, immediate retries can keep you stuck. Backoff is the ordinary fix: wait a little, then a little more, and stop the delay from growing past a limit you can live with.

```
def calculate_backoff_delay(
attempt: int,
base_delay: float,
max_delay: float,
) -> float:
delay = base_delay * (2 ** attempt)
return min(delay, max_delay)
```

With `base_delay=0.5`, the waits go roughly `0.5`, `1.0`, `2.0`, and `4.0` seconds until the cap gets involved.

The cap is important because a retry delay shouldn't wander off forever by accident. A background worker may be allowed to wait longer. A command-line tool probably shouldn't sit there for several minutes while the person running it wonders if it froze.

NOTES

Further study materials Lesson 2

Transcript

In async Python, the pause should use `asyncio.sleep()`.

```
import asyncio

await asyncio.sleep(1.0)
```

`time.sleep()` blocks the thread. `asyncio.sleep()` pauses the current coroutine and lets the event loop keep doing other work. That detail is easy to overlook in a tiny example, and then it shows up later as unrelated async work backing up for no obvious reason.

Read Retry-After when the server gives it to you

Some APIs tell you how long to wait before trying again. The usual place is the `Retry-After` header.

The value may be a number of seconds, or it may be an HTTP date. Supporting both isn't much code.

```
from datetime import datetime, timezone
from email.utils import parsedate_to_datetime
```

Now the retry delay can use the server's hint when it exists.

```
def calculate_retry_delay(
attempt: int,
settings: RetrySettings,
response: httpx.Response | None = None,
) -> float:
if response is not None:
retry_after_delay = get_retry_after_delay(response)

if retry_after_delay is not None:
return min(retry_after_delay, settings.max_delay)
return calculate_backoff_delay(
attempt=attempt,
base_delay=settings.base_delay,
max_delay=settings.max_delay,
)
```

The cap still matters. If a server asks the client to wait a very long time, that may be fine for a background job and miserable for a CLI command. The code should show which choice you made.

Keep the retry helper small enough to test

The first `fetch_user()` function was readable because it had a small job. If you pour retries, parsing, validation, and error handling into that same function, it becomes the place where every concern gets dumped.

A helper keeps the retry behavior in one place. It should send the request, check whether the response is retryable, catch request-level failures like timeouts and connection errors, wait before the next attempt, and eventually give up.

NOTES

Further study materials Lesson 2

Transcript

It should also accept its sleep function as a dependency. That may feel fussy, but it makes the retry behavior easier to test. Tests can pass a no-op sleep function instead of waiting in real time.

```
from collections.abc import Awaitable, Callable
from typing import Any

import httpx

Sleep = Callable[[float], Awaitable[None]]

async def request_with_retries(
client: httpx.AsyncClient,
method: str,
url: str,
settings: RetrySettings,
*,
sleep: Sleep = asyncio.sleep,
**request_kwargs: Any,
) -> httpx.Response:
method = method.upper()
last_error: Exception | None = None
for attempt in range(settings.max_attempts):
response: httpx.Response | None = None
try:
response = await client.request(
method,
url,
**request_kwargs,
)
should_retry_status = (
method in settings.retryable_methods
and response.status_code in
settings.retryable_status_codes
)
if not should_retry_status:
response.raise_for_status()
return response
last_error = httpx.HTTPStatusError(
message=f"Temporary HTTP failure:
{response.status_code}",
request=response.request,
response=response,
)
except httpx.RequestError as error:
if method not in settings.retryable_methods:
raise
last_error = error
if attempt < settings.max_attempts - 1:
delay = calculate_retry_delay(attempt, settings,
response)
await sleep(delay)
if last_error is not None:
raise last_error
raise RuntimeError("Retry loop exited without response or
error.")
```

NOTES

Further study materials Lesson 2

Transcript

There are a few choices here worth noticing.

Non-temporary HTTP errors leave through `response.raise_for_status()`. The helper doesn't retry `400` or `401` just because they are failures.

Request-level errors, such as timeouts and connection failures, are only retried for methods the policy allows. That keeps the helper from repeating a mutating operation without permission.

The loop has a hard end. After the final attempt, it raises the last error. A retry loop without a firm stop can turn a clear failure into a hanging program.

Keep timeouts separate from retries

A timeout and a retry policy sit close together in your head, but they do different jobs.

A timeout limits one attempt. The retry policy decides whether the client should make another attempt after failure.

You usually want both.

```
import httpx

timeout = httpx.Timeout(
timeout=10.0,
connect=3.0,
)
client = httpx.AsyncClient(
base_url="https://api.example.com",
timeout=timeout,
)
```

I prefer putting the timeout on the client and keeping retry rules in the helper. It makes tuning less annoying later. If connection setup is slow, you can change the connect timeout. If a job is too impatient with temporary failures, you can adjust the retry settings. Those two changes shouldn't require surgery in the same function.

This is the kind of plain separation that feels boring until you have five copied request functions and each one behaves a little differently.

Own the HTTP client locally

Creating a fresh `AsyncClient` for every request is fine for the first small example. It's not the shape I'd use once the code is making repeated calls.

A small local API client is enough. The program creates it, passes it to functions that need API access, and closes it when the work is done.

```
from __future__ import annotations
```

NOTES

Further study materials Lesson 2

Transcript

```
from pydantic import BaseModel, Field, ValidationError

class ApiClientConfig(BaseModel):
base_url: str
timeout: float = Field(default=10.0, gt=0)
connect_timeout: float = Field(default=3.0, gt=0)
retry_settings: RetrySettings =
Field(default_factory=RetrySettings)

class UserOut(BaseModel):
id: int
name: str
email: str

class UpstreamPayloadError(Exception):
pass

class ApiClient:
def __init__(self, config: ApiClientConfig) -> None:
self._config = config
self._client = httpx.AsyncClient(
base_url=config.base_url.rstrip("/"),
timeout=httpx.Timeout(
timeout=config.timeout,
connect=min(config.timeout, config.connect_timeout),
),
)
async def __aenter__(self) -> "ApiClient":
return self
async def __aexit__(self, *args: object) -> None:
await self.close()
async def close(self) -> None:
await self._client.aclose()
async def get_user(self, user_id: int) -> UserOut:
response = await request_with_retries(
client=self._client,
method="GET",
url=f"/users/{user_id}",
settings=self._config.retry_settings,
)
try:
data = response.json()
except ValueError as error:
raise UpstreamPayloadError("Upstream returned invalid
JSON.") from error
try:
return UserOut.model_validate(data)
except ValidationError as error:
raise UpstreamPayloadError("Upstream returned unexpected
user data.") from error
```

The caller owns the client's lifetime.

```
async def main() -> None:
config = ApiClientConfig(
```

NOTES

Further study materials Lesson 2

Transcript

```
base_url="https://api.example.com",
timeout=10.0,
)

async with ApiClient(config) as client:
user = await client.get_user(123)
print(user)

if __name__ == "__main__":
asyncio.run(main())
```

This keeps ownership visible. The program creates the client. The client owns the `AsyncClient`. The caller closes it by leaving the async context manager. Tests can create a fresh client. A different caller can use different retry settings.

That is a better habit than hiding HTTP clients in module-level state and hoping every future caller wants the same behavior.

Validate the response before passing it around

A successful HTTP status only says the HTTP request succeeded. It doesn't promise that the body is useful.

The API can omit a field. It can return a string where you expected an integer. It can send an HTML error page with a `200` because some proxy behaved badly. If you call enough APIs, you eventually see something ugly enough that you stop trusting status codes alone.

A Pydantic model gives the expected shape a name.

```
from pydantic import BaseModel

class UserOut(BaseModel):
id: int
name: str
email: str
```

The client validates the response near the boundary.

```
try:
return UserOut.model_validate(data)
except ValidationError as error:
raise UpstreamPayloadError("Upstream returned unexpected
user data.") from error
```

This can feel strict in a tiny example. In code that depends on another service, I'd rather fail near the HTTP call than let bad data wander through the program and become harder to trace.

The full example

Here's the version with the pieces together.

```
from __future__ import annotations
```

NOTES

Further study materials Lesson 2

Transcript

```
import asyncio
from collections.abc import Awaitable, Callable
from datetime import datetime, timezone
from email.utils import parsedate_to_datetime
from typing import Any
import httpx
from pydantic import BaseModel, Field, ValidationError,
field_validator

TEMPORARY_STATUS_CODES = frozenset({
408,
429,
500,
502,
503,
504,
})
RETRYABLE_METHODS = frozenset({"GET", "HEAD", "OPTIONS"})
Sleep = Callable[[float], Awaitable[None]]

class RetrySettings(BaseModel):
max_attempts: int = Field(default=3, ge=1, le=10)
base_delay: float = Field(default=0.5, ge=0)
max_delay: float = Field(default=8.0, ge=0)
retryable_status_codes: frozenset[int] = Field(
default_factory=lambda: TEMPORARY_STATUS_CODES
)
retryable_methods: frozenset[str] = Field(
default_factory=lambda: RETRYABLE_METHODS
)
@field_validator("retryable_methods", mode="before")
@classmethod
def normalize_retryable_methods(cls, value: object) ->
frozenset[str]:
if isinstance(value, str):
return frozenset({value.upper()})
return frozenset(str(method).upper() for method in value)

class ApiClientConfig(BaseModel):
base_url: str
timeout: float = Field(default=10.0, gt=0)
connect_timeout: float = Field(default=3.0, gt=0)
retry_settings: RetrySettings =
Field(default_factory=RetrySettings)

class UserOut(BaseModel):
id: int
name: str
email: str

class UpstreamPayloadError(Exception):
pass

def parse_retry_after(value: str) -> float | None:
try:
return max(0.0, float(value))
```

NOTES

Further study materials Lesson 2

Transcript

```
except ValueError:
pass
try:
retry_time = parsedate_to_datetime(value)
if retry_time.tzinfo is None:
retry_time = retry_time.replace(tzinfo=timezone.utc)
delay = (retry_time -
datetime.now(timezone.utc)).total_seconds()
return max(0.0, delay)
except (TypeError, ValueError, OverflowError):
return None

def get_retry_after_delay(response: httpx.Response) ->
float | None:
retry_after = response.headers.get("Retry-After")
if retry_after is None:
return None
return parse_retry_after(retry_after)

def calculate_backoff_delay(
attempt: int,
base_delay: float,
max_delay: float,
) -> float:
delay = base_delay * (2 ** attempt)
return min(delay, max_delay)

def calculate_retry_delay(
attempt: int,
settings: RetrySettings,
response: httpx.Response | None = None,
) -> float:
if response is not None:
retry_after_delay = get_retry_after_delay(response)
if retry_after_delay is not None:
return min(retry_after_delay, settings.max_delay)
return calculate_backoff_delay(
attempt=attempt,
base_delay=settings.base_delay,
max_delay=settings.max_delay,
)

async def request_with_retries(
client: httpx.AsyncClient,
method: str,
url: str,
settings: RetrySettings,
*,
sleep: Sleep = asyncio.sleep,
**request_kwargs: Any,
) -> httpx.Response:
method = method.upper()
last_error: Exception | None = None
for attempt in range(settings.max_attempts):
response: httpx.Response | None = None
try:
```

NOTES

Further study materials Lesson 2

Transcript

```
response = await client.request(
method,
url,
**request_kwargs,
)
should_retry_status = (
method in settings.retryable_methods
and response.status_code in
settings.retryable_status_codes
)
if not should_retry_status:
response.raise_for_status()
return response
last_error = httpx.HTTPStatusError(
message=f"Temporary HTTP failure:
{response.status_code}",
request=response.request,
response=response,
)
except httpx.RequestError as error:
if method not in settings.retryable_methods:
raise
last_error = error
if attempt < settings.max_attempts - 1:
delay = calculate_retry_delay(attempt, settings,
response)
await sleep(delay)
if last_error is not None:
raise last_error
raise RuntimeError("Retry loop exited without response or
error.")

class ApiClient:
def __init__(self, config: ApiClientConfig) -> None:
self._config = config
self._client = httpx.AsyncClient(
base_url=config.base_url.rstrip("/"),
timeout=httpx.Timeout(
timeout=config.timeout,
connect=min(config.timeout, config.connect_timeout),
),
)
async def __aenter__(self) -> "ApiClient":
return self
async def __aexit__(self, *args: object) -> None:
await self.close()
async def close(self) -> None:
await self._client.aclose()
async def get_user(self, user_id: int) -> UserOut:
response = await request_with_retries(
client=self._client,
method="GET",
url=f"/users/{user_id}",
settings=self._config.retry_settings,
)
try:
```

NOTES

Further study materials Lesson 2

Transcript

```
data = response.json()
except ValueError as error:
raise UpstreamPayloadError("Upstream returned invalid
JSON.") from error
try:
return UserOut.model_validate(data)
except ValidationError as error:
raise UpstreamPayloadError("Upstream returned unexpected
user data.") from error

async def main() -> None:
config = ApiClientConfig(
base_url="https://api.example.com",
timeout=10.0,
)
async with ApiClient(config) as client:
user = await client.get_user(123)
print(user)

if __name__ == "__main__":
asyncio.run(main())
```

The longer version costs more code, but the extra code names the choices the small version left buried.

How long can one request wait? Which failures get another attempt? Which methods are safe to repeat? How long should the client pause? Should the client listen when the server sends `Retry-After`? What happens when the response body is JSON, but not the JSON your program expected?

For a one-off script, those questions may not deserve this much structure. For a job or tool that people rely on, they become real maintenance concerns.

How I would test it

The useful parts of this design can be tested without making real HTTP calls.

The delay calculation is just a function.

```
def test_calculate_backoff_delay_caps_value() -> None:
assert calculate_backoff_delay(
attempt=10,
base_delay=0.5,
max_delay=8.0,
) == 8.0
```

`Retry-After` parsing can be tested directly.

```
def test_parse_retry_after_seconds() -> None:
assert parse_retry_after("2") == 2.0
```

The retry behavior can be tested with `httpx.MockTransport` and a fake sleep function.

NOTES

Further study materials Lesson 2

Transcript

```
async def no_sleep(delay: float) -> None:
return None

async def test_retries_temporary_status_then_succeeds()
-> None:
calls = 0
async def handler(request: httpx.Request) ->
httpx.Response:
nonlocal calls
calls += 1
if calls == 1:
return httpx.Response(503, request=request)
return httpx.Response(
200,
json={
"id": 123,
"name": "Ada",
"email": "ada@example.com",
},
request=request,
)
transport = httpx.MockTransport(handler)
async with httpx.AsyncClient(
transport=transport,
base_url="https://api.example.com",
) as client:
response = await request_with_retries(
client=client,
method="GET",
url="/users/123",
settings=RetrySettings(max_attempts=2),
sleep=no_sleep,
)
assert calls == 2
assert response.status_code == 200
```

I'd also test that a `POST` is not retried by default, that a `400` leaves immediately, that invalid JSON becomes `UpstreamPayloadError`, and that a malformed response fails validation.

Those tests protect the behavior that tends to drift when retry code gets copied around.

What this example does not try to solve

This client is not a full SDK.

It does not handle authentication refresh, pagination, structured logging, metrics, distributed rate limits, circuit breaking, or schema evolution across API versions. Those may matter in a larger client, but adding all of them here would bury the retry lesson.

It also does not add jitter to backoff. In a large fleet of clients, jitter is often a good idea because it keeps many clients from retrying at the same time after a shared outage. For one small tool, fixed backoff may be enough. For a service with many workers, I'd add

NOTES

Further study materials Lesson 2

Transcript

jitter deliberately and test the bounds.

The point is not to pretend the example is production complete. The point is to make the retry behavior explicit enough that it can be reviewed, tested, and changed without hunting through five copied request functions.

Conclusion

Keep the first version when it's enough. A small script doesn't need to pretend it's a service client.

Once the same HTTP call supports a job, a worker, or a tool other people rely on, give the failure behavior somewhere obvious to live. Put timeouts on the client. Retry only failures that might improve after a wait. Use backoff so retries don't pile onto an already unhappy service. Read `Retry-After` when the server sends it. Use `asyncio.sleep()` in async code. Avoid retrying unsafe methods by default. Validate the response before the rest of the program trusts it.

HTTP calls still fail. The practical improvement is that the failure behavior becomes easier to read, easier to test, and less likely to be copied badly into the next function.

Frequently Asked Questions

Should every failed httpx request be retried?

No. Retry failures that are likely to be temporary. Bad payloads, bad credentials, and most missing resources usually won't improve because the client repeats the request.

Which HTTP status codes are common retry candidates?

Common examples include `408`, `429`, `500`, `502`, `503`, and `504`. Treat that as a starting policy, not a list to paste into every client without thinking.

Why avoid retrying POST by default?

A `POST` may create or change something. If the first attempt reached the server but the client never received the response, retrying can duplicate the operation. Retry mutating requests only when the API makes that safe, often through idempotency keys.

Why use exponential backoff?

Immediate retries can add pressure to a service that's already struggling. Backoff gives the upstream service some room before the next attempt.

Why use asyncio.sleep instead of time.sleep?

`asyncio.sleep()` pauses the coroutine without blocking the event loop. Other async work can continue while the retry waits.

NOTES

Further study materials Lesson 2

Transcript

Why use Pydantic for retry settings?

Retry settings are configuration, and configuration should be checked. A value like `max_attempts=0` should fail early instead of quietly changing the behavior of the client.

Why validate the upstream response with Pydantic?

A successful HTTP response can still contain data your program doesn't expect. Validation catches that problem near the boundary instead of letting bad data move deeper into the program.

Why inject the sleep function?

It makes retry behavior easier to test. Tests can pass a fake sleep function and verify retry behavior without waiting in real time.

Is this production-ready?

It is a solid small-client pattern, not a complete SDK. Larger clients may need authentication refresh, pagination, structured logging, metrics, jitter, circuit breaking, distributed rate limiting, and more careful concurrency rules.

NOTES

Exercises • Lesson 2

EXERCISE 1 • True or false

CONTEXT

When building Flet apps that fetch data from the internet, it's important to use asynchronous HTTP requests to keep the UI responsive. The httpx library provides tools for making these requests.

QUESTION • Indicate whether the information is true or false

The correct way to make an asynchronous GET request with httpx is to use the 'httpx.get' function directly, without an 'AsyncClient'.

◯ TRUE ◯ FALSE

NOTES

Exercises • Lesson 2

EXERCISE 2 • Multiple choice

CONTEXT

When making asynchronous HTTP requests in a Flet app using httpx, it's important to follow the correct async pattern to ensure the request works as expected.

QUESTION • Select the correct option

What happens if you forget to use the 'await' keyword when making an async GET request with httpx?

A. The function returns a coroutine object instead of the actual response.

B. The app crashes with a runtime error.

C. The request is sent but the response is ignored.

D. A SyntaxError is raised immediately.

NOTES

Exercises • Lesson 2

EXERCISE 3 • True or false

CONTEXT

Python offers several libraries for making HTTP requests, each with different capabilities for handling synchronous and asynchronous operations.

QUESTION • Indicate whether the information is true or false

The requests library supports asynchronous requests.

◯ TRUE ◯ FALSE

NOTES

Exercises • Lesson 2

EXERCISE 4 • Multiple choice

CONTEXT

In the lesson, we compared the requests library and the modern httpx library. While both can make HTTP requests, httpx offers several advanced features that are not available in requests.

QUESTION • Select the correct option

Which of the following features is supported by HTTPX but not by Requests?

A) Connection pooling

B) Streaming responses

C) HTTP/2 support

D) JSON handling

NOTES

Exercises • Lesson 2

EXERCISE 5 • True or false

CONTEXT

When implementing HTTP retries, it is important to decide which HTTP methods are safe to repeat. The lesson provides a default set of retryable methods.

QUESTION • Indicate whether the information is true or false

According to the lesson, POST requests should always be retried by default.

◯ TRUE ◯ FALSE

NOTES

Exercises • Lesson 2

EXERCISE 6 • Multiple choice

CONTEXT

The lesson discusses HTTP retry strategies and emphasizes that not all methods are safe to repeat automatically. It introduces a set of retryable methods that are considered idempotent and safe to retry by default.

QUESTION • Select the correct option

Which of the following HTTP methods is considered safe to retry by default, according to the lesson?

A GET

B DELETE

C PUT

D POST

NOTES

Exercise Answers

LESSON 2

EX.01 • True or false

False.

The lesson explicitly states that you must use 'httpx.AsyncClient' inside an async function with 'await'. Using 'httpx.get' directly is synchronous and would freeze the Flet UI.

I got it right I got it wrong

EX.02 • Multiple choice

Option A — "The function returns a coroutine object instead of the actual response."

The lesson explicitly states that forgetting 'await' means Python returns a coroutine object, not a response, and the code runs on garbage data without updating anything.

I got it right I got it wrong

EX.03 • True or false

False.

The requests library is synchronous only, meaning each request blocks execution until a response is received. It does not support async operations natively.

I got it right I got it wrong

EX.04 • Multiple choice

Option C — "HTTP/2 support"

The lesson explicitly states that requests does not support HTTP/2, while httpx does. This is a key advantage of httpx for better performance.

I got it right I got it wrong

EX.05 • True or false

False.

The lesson states that POST requests are not retried by default because they may create or change state. The default retryable methods are GET, HEAD, and OPTIONS.

I got it right I got it wrong

EX.06 • Multiple choice

Option A — "GET"

The lesson explicitly includes GET in RETRYABLE_METHODS because it is a read-only operation that does not change state, making it safe to retry without risk of duplication or side effects.

I got it right I got it wrong

Chapter 06 • Async Programming and Data Integration

Lesson 03 • Connecting to a SQLite Database

44 pages • 1 hr 2 min

elevify

Lesson 03 • Connecting to a SQLite Database

1 hr 2 min • 44 pages

◎ What you will learn

Uses Python's sqlite3 module to persist and query structured data from a Flet app. Provides a complete local data layer for CRUD applications.

Summary	10 min	**93**
Transcript	10 min	**94**
Further study materials		
Text • Using Python's sqlite3 Module to Create, Query, and Manage SQLite Databases with In-Memory Option	12 min	**98**
Audio • SQLite CRUD Operations for Creating, Reading, Updating, and Deleting Data	13 min	**104**
Text • Building a Python SQLite CRUD Application with Recursive Infinite Loop Menu	11 min	**108**
Text • How to Create Tables, Insert Data, Query, and Execute Multiple SQL Statements in SQLite	15 min	**114**
Exercises		**125**
Answer key		**135**

Connecting to a SQLite Database

Summary

Setting Up SQLite Database in Flet

SQLite is a lightweight database that comes with Python's built-in sqlite3 module. No extra installation needed. You connect to a single file, like app_data.db, and it survives app restarts. Set up connection and table at startup using IF NOT EXISTS. Commit after every write. Close connection on app exit. This lifecycle keeps your app clean and persistent.

Inserting and Querying Data Safely

Use parameterized queries with ? placeholders to prevent SQL injection. For INSERT, pass values as a tuple and call commit. For SELECT, use fetchall to get rows as tuples. Keep column order consistent. This makes your code safe and predictable. These two operations are the foundation for getting data in and out of your database.

Displaying Data with ListView

After fetching rows, build UI controls dynamically. Use ListView for a scrollable list. Apply the clear-rebuild-update pattern: clear controls, loop through query results, add Text or other controls, then call page.update. Wire text field and button so user input triggers add_task and refresh. This gives instant feedback and keeps UI in sync with database.

Updating and Deleting Records

Update and Delete complete CRUD. Always use the primary key id in WHERE clauses to target the correct row. Write UPDATE or DELETE with parameterized queries, commit, then refresh the UI. Attach buttons or checkboxes to each list item so they know which row to modify. This pattern makes your app fully interactive.

Common Mistakes and Key Takeaways

Avoid forgetting conn.commit() – changes won't save. Never put user input directly in SQL with f-strings; use ? placeholders. Always close connections to prevent locks. Don't refresh UI on every keystroke. With these four operations you can build many apps like task managers or notes. Remember: connect once, use placeholders, commit after writes, and sync UI by clearing, rebuilding, and updating.

NOTES

Connecting to a SQLite Database

Transcript

Introduction

You're about to connect a Flet app to a real SQLite database and build full CRUD functionality. We'll start by setting up the database connection, then add create, read, update, and delete — step by step. All from Python. No separate backend, no extra languages.

You'll end up with a fully functional app that stores data persistently, and you'll see every change reflected live in your Flet UI. That's what this lesson is about.

Why SQLite?

SQLite is the obvious starting point for databases in Flet. It ships with Python's built-in sqlite3 module — zero config, no server, nothing to install. Your data lives in a single file on disk, which means it survives app restarts.

That's a huge step up from keeping state in Python variables. For local desktop and mobile apps, SQLite is the perfect lightweight choice.

CRUD Operations Overview

In this lesson, you'll work through four operations in a logical order. First, you'll create the database and open a connection. Then you'll insert records and query them back.

After that, you'll display those results in a Flet ListView. Finally, you'll add update and delete to make the app fully interactive. Each step builds on the last.

Setting Up the Database Connection

We're in section one: creating and connecting. That means building the database file and opening a connection — the foundation for everything else. Before any query runs, you need both.

Here's the clean way to set them up inside a Flet app. Next we'll go through opening that connection with sqlite3.

Using the sqlite3 Module

The sqlite3 module comes built into Python's standard library. No extra installation — just import it. Then call connect with a filename. In the example, it's 'app_data.db'. If that file doesn't exist, connect creates it. That's one line for setup.

Next, get a cursor from the connection. That cursor is your gateway to the database — every SQL command goes through it. One key tip: maintain a single connection per session; don't open and close for each request. That's the whole routine.

Creating Tables and Committing Changes

The table must exist before any other code runs. With the IF NOT

NOTES

Connecting to a SQLite Database

Transcript

EXISTS clause, you can safely run this CREATE statement on every startup. If the table already exists, nothing changes. If it's missing, it gets created.

After any write operation — INSERT, UPDATE, DELETE — call commit to save the change to disk. No commit means nothing gets written permanently.

Structuring the Database Lifecycle

The key is structure. Define a setup function that opens the connection and creates the table. Call it at the very top of your main function, before any controls are added to the page.

When the app closes, close the connection in a cleanup handler. That way the database lifecycle stays tied to the app lifecycle — predictable and clean.

Inserting and Selecting Data

Now that our database lifecycle is set up, we shift focus to the two fundamental operations: getting data in and pulling it back out. That's INSERT to write new rows and SELECT to query them.

Mastering INSERT and SELECT is where your app comes to life.

Safe Queries with Parameterized Statements

That question mark placeholder is the key safety feature. In the function add_task, we pass the title as a separate tuple — (title,) — instead of formatting it into the SQL string. Sqlite3 escapes it automatically, preventing SQL injection, a real threat whenever user-typed text hits a query.

That's why we always use these parameterized queries. After the insert, call commit to save the row permanently. Next, we'll select those records back.

Reading Data with SELECT and fetchall

After executing a SELECT, call fetchall to retrieve every matching row. It returns a list of tuples — each tuple contains the values for one row in the exact order you listed the columns.

Keep that order consistent. Your code can rely on index positions: row[0] is the id, row[1] is the title, row[2] is the done flag. That predictability keeps your logic clean and avoids surprises every time you query.

Displaying Data in the Flet UI

Fetching rows from the database is only half the job. The other half is showing them in the Flet UI. ListView is the right control for that — it gives you a scrollable list where each item can be built from the tuple data you already have.

NOTES

Connecting to a SQLite Database

Transcript

Now let's move on to building that ListView from query results.

The Clear-Rebuild-Update Pattern

So inside refresh_list, you clear the controls with .clear(), then loop over the fresh query results. For each row, build a Text control with the row number and task name, and append it to the list.

Finally, call page.update to push the new state to the UI. This cycle ensures the UI always mirrors the database — no stale in-memory copy. That's the core pattern: clear, rebuild, update. It's simple but it works every time.

Wiring User Input to Database Actions

Wire the text field and button together so they work in sync. When the user clicks, the handler grabs the current input, calls add_task, clears the field, then calls refresh_list. The new record pops up instantly on the list — no manual reload or page refresh needed.

That instant feedback — seeing your new task appear right there on the list — is exactly what makes the whole app feel polished, responsive, and complete.

Implementing Update and Delete

You've covered Create and Read. Now Update and Delete complete the CRUD cycle. Delete removes a record by its ID, just pass the id. Update changes a field value, specify the new value and the record id.

Both use the same parameterized query pattern you already know. The structure is identical; only the SQL command changes.

Deleting Records Safely

When you need to delete a row, always target the primary key. Using title or any other field that isn't guaranteed unique could delete the wrong record. Pass the id as a parameter to your SQL query, commit the transaction, and then call refresh_list.

This ensures the deleted item vanishes from the UI immediately. Finally, attach the delete button directly to each list item so the id of that specific row is always in scope.

Updating Records

Updating a record follows the same three steps you saw with deletion: write an UPDATE query with a WHERE clause on the id, pass the value as a parameter, commit, then refresh the UI.

Here a checkbox toggles the done column, and the same pattern works for any editable field in your schema. The id stays in scope — just like the delete button, the checkbox is attached directly to each task so it always knows which row to update. This completes the

NOTES

Connecting to a SQLite Database

Transcript

update piece of our CRUD puzzle.

The Four-Step CRUD Pattern

Every SQLite operation in Flet follows that same four-step rhythm. Connect once when the app starts. Write with parameterized queries and commit. Read using fetchall and build your controls.

Then modify by id — update or delete — commit, and refresh the UI. Once this pattern clicks, adding new tables or fields is just more of the same.

Common Mistakes to Avoid

Four mistakes keep tripping people up. Number one: forgetting conn.commit(). The query runs, but nothing gets saved. Two: dropping user input straight into SQL with f-strings - that's a security risk.

Three: not closing the connection, which can lock the database file. Four: refreshing the whole UI on every keystroke. That makes the app sluggish. Avoid those, and your app will behave exactly as expected.

Real-World Applications

With these four operations — Create, Read, Update, Delete — you can build real, useful apps right now. A task manager where you add, complete, and delete tasks. A notes app storing entries per session.

A shopping list tracking items and quantities. They're all different tables and different Flet controls built on the same CRUD pattern. The database layer is identical every time.

Key Takeaways

Here are three things to remember. First, connect once at startup with sqlite3.connect(), create your table with IF NOT EXISTS, close on exit. Second, always use ? placeholders for INSERT, UPDATE, DELETE, and commit after every write.

Third, sync the UI: clear controls, fetch all rows, rebuild, and then call page.update(). That's the complete SQLite workflow for Flet apps.

NOTES

Further study materials Lesson 3

Content type: Text

Using Python's sqlite3 Module to Create, Query, and Manage SQLite Databases with In-Memory Option

Available at https://medium.com/@sunil.gupta676/exploring-sqlite-database-and-table-connection-of-python-99a82d6fb89d

Full Content

Summary

Introduction and Installation of sqlite3

The article introduces SQLite database connection in Python using the sqlite3 module. To use it, install the module via pip. For Python 2.7.3, use 'pip install pysqlite'. For Python 3.0, use 'pip install pysqlite3'. After installation, import the module. The module provides methods like connect, register_converter, register_adapter, and complete_statement. The connect method takes a database name or ':memory:' and optional detect_types.

Database Connection and Cursor Objects

The Connection Object, created by sqlite3.connect(), has methods like cursor(), commit(), rollback(), close(), execute(), and executemany(). The Cursor Object, from connection.cursor(), supports execute() with parameters (qmark or named), executemany() for multiple rows, and executescript() for SQL scripts. It also has fetch methods: fetchone(), fetchmany(size), and fetchall() to retrieve query results.

Example with Database File

The first example shows how to create a connection to 'example.db', get a cursor, create a table 'stocks' with columns date, trans, symbol, qty, price, insert a row, commit changes, and close the connection. Then it reopens the connection and fetches the row using a parameterized query with symbol 'RHAT'. It emphasizes committing changes and closing the connection to avoid data loss.

Example with In-Memory Database

The second example uses an in-memory database (':memory:') for temporary transactions. It creates a table 'people' with name_last and age, inserts data using qmark style (?, ?) and named style (:who, :age), then fetches the row with fetchone() and all rows with fetchall(). It warns that in-memory is not suitable for large data due to performance and storage issues.

Conclusion and Author Note

The article concludes by summarizing the key points:

NOTES

Further study materials Lesson 3

Summary

understanding database connection with sqlite3, using cursor objects, and executing transactions. It invites readers to comment with doubts and subscribe for updates. The author, an Android developer, promotes sharing and provides contact email. The note states the article was originally published on mobologicplus.com on March 16, 2019.

NOTES

Further study materials Lesson 3

Transcript

Exploring SQLite Database and Table Connection with Python

*[Image: https://miro.medium.com/0*xybYDO5T_enTkiaw]*

In our Python learning journey, we continue with some of the other modules that help in learning Python. I have posted many articles related to basic Python concepts, and I hope they help you a lot. Today we will learn about a major module: database connection and its use in Python.

In Python, we can use any database API, like MySQL, SQL Server, and so on. But today we will learn about the sqlite3 module, which is widely used. To use the sqlite3 module, we need to install it using the pip command in the virtual environment. Before installing the SQLite package, please check which Python version is running on your system.

If you are using Python version 2.7.3, use this command to install:

```
pip install pysqlite
```

For Python version 3.0, run this command:

```
pip install pysqlite3
```

Once you have installed the above package, you can import this module in your project and use its features easily. Now let's see the methods available in the SQLite module.

Methods:

- `sqlite3.connect(database)` → Database: either provide db name or `':memory:'`. Detect_types: PARSE_DECLTYPES, PARSE_COLNAMES.
- `sqlite3.register_converter(typename, callable)` → Convert database byte string to Python type.
- `sqlite3.register_adapter(type, callable)` → Convert the Python type to database types.
- `sqlite3.complete_statement(sql)` → Whether string contains one or more SQL statements.

Now we need to understand the Database Connection Object and its methods.

Database: Connection Object

A Connection Object is returned by calling `sqlite3.Connection()`.

Methods:

- `cursor()`: creates a pointer to the database.
- `commit()`: commits the current transaction.

NOTES

Further study materials Lesson 3

Transcript

- `rollback()`: rolls back any changes until the last commit.
- `close()`: closes the connection.
- `execute(sql)`: executes an SQL query.
- `executemany(sql)`: executes multiple SQL statements.

Database: Cursor Object

A Cursor Object is returned by calling `connection.cursor()`.

Statements:

- `execute(sql, parameters)`: Execute SQL statements. Parameters can be passed as `?` or named.
- `executemany(sql, parameters_list)`: Same SQL command with each parameter from a sequence.
- `executescript(sql_script)`: Execute an entire script.

Methods:

- `fetchone()`: fetch one row of query result.
- `fetchmany(size)`: number for rows from query result.
- `fetchall()`: fetches all the rows from query result.

Now let's look at a few examples of SQLite to create a connection between the database and a table. We will create a table, insert some data, and fetch that data with an SQL query. The most important thing is that once you have a connection object that executed properly, you need to close the connection at the end to avoid leaks.

```
import sqlite3
conn = sqlite3.connect('example.db')
print conn
c = conn.cursor()
print c

# Create table
c.execute('''CREATE TABLE stocks
(date text, trans text, symbol text, qty real, price
real)''')

# Insert a row of data
c.execute("INSERT INTO stocks VALUES
('2006-01-05','BUY','RHAT',100,35.14)")

# Save (commit) the changes
conn.commit()

# We can also close the connection if we are done with
it.
# Just be sure any changes have been committed or they
will be lost.
conn.close()

conn = sqlite3.connect('example.db')
```

NOTES

Further study materials Lesson 3

Transcript

```
c = conn.cursor()
t = ('RHAT',)
c.execute('SELECT * FROM stocks WHERE symbol=?', t)
print c.fetchone()
```

In the above example, we can see that we need to import the sqlite3 module to use its features. We need to create a connection with the named database. Then we need to get the cursor object to create the table and insert data into the table. Once you are done, commit the transaction to execute the data insertion, and then close the connection.

If you want to execute more data inserts, fetches, or deletes, you need to get the connection object again and use its cursor object. I hope this is clear. If you still have confusion, let's look at one more example: using the cursor object with memory rather than a database. For a few transactions this is okay, but it is not recommended for huge transactions or data to store in memory, as it will affect performance and storage.

```
import sqlite3
con = sqlite3.connect(":memory:")
cur = con.cursor()
cur.execute("create table people (name_last, age)")

who = "Yeltsin"
age = 72

# This is the qmark style:
cur.execute("insert into people values (?, ?)", (who,
age))

# And this is the named style:
cur.execute("select * from people where name_last=:who
and age=:age",
{"who": who, "age": age})

print cur.fetchone() # fetch the single current row
print cur.fetchall() # fetch all the rows
```

Now it is easy to understand how to use the database cursor object. For advanced usage, we can use the Cursor Adapter to convert from SQL to Python. But I will not go into the details of the Cursor Adapter here. In this article, we have covered the basics of the cursor object statements to execute transactions.

Summary: Now we have a good understanding of the database connection using the sqlite3 module. We have seen a few examples and played around with them to clear up doubts. If you still have issues or doubts, please add a comment, and I will try to reply as much as possible.

Please subscribe to your email to get the newsletter on this blog below. If you like this post, do not forget to share, like, and

NOTES

Further study materials Lesson 3

Transcript

comment in the section below.

Happy Coding

*[Image: https://miro.medium.com/0*9kab_BZhsZKYtYN2]*

I am a very enthusiastic Android developer building solid Android apps. I have a keen interest in developing for Android and have published apps to the Google Play Store. I am always open to learning new technologies. For any help, drop us a line anytime at contact@mobologicplus.com

---

Originally published at mobologicplus.com on March 16, 2019.

NOTES

Further study materials Lesson 3

Content type: Audio

SQLite CRUD Operations for Creating, Reading, Updating, and Deleting Data

Available at https://www.buzzsprout.com/2078987/episodes/11638672-15-2-single-table-sql.mp3

Full Content

NOTES

Summary

Creating a Database in SQLite

We create a database using SQLite Browser. First, open the browser and create a new database on your desktop, naming it 'Py-Forey Fund'. A new file appears. Do not edit this file with a text editor. The browser then asks to create a table. We will create a table called 'users' with two columns: 'name' as text and 'email' as text. Defining columns commits to a contract about the data. The database optimizes storage based on this contract.

Creating the Users Table

The database browser helps us write SQL. The equivalent SQL statement is: CREATE TABLE users (name TEXT, email TEXT);. After clicking OK, the database shows the table structure with these columns. This is the CREATE statement in action, defining the table's schema. We must follow this contract when adding data. The table is now ready for data insertion.

Inserting Data into the Table

Now we add records. Insert a row with name 'Charles' and email 'csev@umich.edu' using INSERT INTO users (name, email) VALUES ('Charles', 'csev@umich.edu');. Execute it in the 'Execute SQL' tab. The table now has one record. We can insert multiple rows at once by separating statements with semicolons. For example, insert Kristen, Ted, and Sarah. Running them shows all rows. In real apps, a program generates this SQL automatically.

Deleting and Updating Rows

To delete data, use DELETE FROM with a WHERE clause. Without WHERE, it deletes all rows. For example, to remove Ted: DELETE FROM users WHERE email = 'ted@umich.edu';. After running, Ted is gone. To update existing rows, use UPDATE. For example, UPDATE users SET name = 'Chuck' WHERE email = 'csev@umich.edu'; changes Charles to Chuck. Without WHERE, it updates every row. These operations allow modifying data.

Retrieving Data and CRUD Summary

The SELECT statement retrieves data. SELECT * FROM users returns all records. Add WHERE to filter, e.g., SELECT * FROM users WHERE

Further study materials Lesson 3

Summary

email = 'csev@umich.edu';. Order results with ORDER BY, e.g., ORDER BY email or ORDER BY name DESC. Databases excel at sorting. We covered CRUD: Create (CREATE TABLE, INSERT), Read (SELECT), Update (UPDATE), Delete (DELETE). SQL is declarative and simple.

NOTES

Further study materials Lesson 3

Transcript

Introduction to Database Creation with SQLite

We're now going to create a database using SQLite Browser. Hopefully you have it downloaded so you can follow along. I have this handout—a basic database handout—that saves you from typing all these commands yourself. Bring it up in your web browser. It contains all the commands I'll type now. You can pull them from the web page, the slides, or the handout.

Creating a New Database

Let's open the database browser. I'll create a new database on my desktop and name it `Py-Forey Fund`. You should see a new file appear. *Do not edit this file with a text editor*—it's meant only for SQLite Browser.

The database browser asks me to create a table. I'll call it `users` with two columns: `name` (text) and `email` (text). When we define columns, we commit to a contract about what data goes in each column. The database optimizes storage based on this contract. We can choose any column names, but once set, we must follow the contract.

Creating the Users Table

The user interface helps us write SQL. Here's the equivalent SQL statement:

```
CREATE TABLE users (name TEXT, email TEXT);
```

You can see that in the slides. After clicking OK, the database shows the table structure. That's the `CREATE` statement in action.

Inserting Data

Now let's add a record. I'll insert a row with `name = 'Charles'` and `email = 'csev@umich.edu'`. This is like a spreadsheet row. The SQL way uses `INSERT`:

```
INSERT INTO users (name, email) VALUES ('Charles',
'csev@umich.edu');
```

I'll paste that into the "Execute SQL" tab and run it. The query executes successfully. Now the table has one record.

Adding Multiple Rows

We can insert multiple rows at once by separating statements with a semicolon:

```
INSERT INTO users (name, email) VALUES ('Kristen',
'kristen@umich.edu');
INSERT INTO users (name, email) VALUES ('Ted',
'ted@umich.edu');
INSERT INTO users (name, email) VALUES ('Sarah',
'sarah@umich.edu');
```

NOTES

Further study materials Lesson 3

Transcript

Run them all together. Now the data view shows all those rows. In real applications, a program (Python, PHP, etc.) generates this SQL automatically.

Deleting Rows

To delete data, use `DELETE FROM` with a `WHERE` clause. Without `WHERE`, it deletes all rows. For example, to remove Ted:

```
DELETE FROM users WHERE email = 'ted@umich.edu';
```

Run it—Ted is gone from the table.

Updating Records

The `UPDATE` statement changes existing rows. Syntax:

```
UPDATE users SET name = 'Chuck' WHERE email =
'csev@umich.edu';
```

This changes Charles to Chuck. Without `WHERE`, it would update every row.

Retrieving Data with SELECT

The `SELECT` statement retrieves data. `SELECT *` means all columns:

```
SELECT * FROM users;
```

This returns all records. You can add a `WHERE` clause to filter:

```
SELECT * FROM users WHERE email = 'csev@umich.edu';
```

Or order results with `ORDER BY`:

```
SELECT * FROM users ORDER BY email;
SELECT * FROM users ORDER BY name DESC;
```

Databases excel at sorting and selecting.

Summary of CRUD Operations

We've covered the four basic database operations: **Create** (CREATE TABLE, INSERT), **Read** (SELECT), **Update** (UPDATE), and **Delete** (DELETE). This is a concise summary of SQL. You might wonder why it took so long to learn such a simple language—it's intentionally straightforward and predictable.

SQL is a declarative language; you don't need to write loops or manage iteration variables. That simplicity is powerful.

NOTES

Further study materials Lesson 3

Content type: Text

Building a Python SQLite CRUD Application with Recursive Infinite Loop Menu

Available at https://medium.com/@damlc/task-8-developing-a-project-on-db-using-update-insert-and-delete-functions-in-sqlite-with-python-3dc31306f6a8

Full Content

Summary

Setting Up SQLite on Kali Linux

The project begins by introducing SQLite and DB Browser for SQLite (DB4S), an open-source tool for creating and editing SQLite databases with a spreadsheet-like interface. Installation steps on Kali Linux are provided using the Snap package manager. Commands include installing snapd, enabling necessary services, and installing sqlitebrowser via snap. This setup allows users to work with a simple student database throughout the project.

Project Overview and User Options

The project involves connecting to an SQLite database with Python to perform operations on student data. Users are prompted to choose from four options: search, update, add, or delete a student. The initial menu displays these choices in Turkish. To avoid repetitive code, the project uses Python functions defined with the 'def' keyword. A class called 'Database' is created to group all database-related functions, making the code modular and reusable.

Database Functions for CRUD Operations

The 'Database' class includes functions for various operations. 'getAllStudents' retrieves all records sorted by name. 'searchStudent' searches by name or surname using a LIKE query. 'searchStudentID' looks up a student by ID. 'updateStudent' modifies a specified column for a given ID. 'addStudents' inserts new name and surname. 'deleteStudent' removes a record by ID. Each function prints results and uses 'Connection.commit()' to save changes permanently.

Implementing User Choices with Conditions

User choices are handled with conditional statements. For search (option 1), the user inputs a name, and the program calls 'searchStudent' and prints results or a not-found message. For update (option 2), it shows all students, asks for an ID, then lets the user choose to update name or surname, calling 'updateStudent'. For add (option 3), it collects new name and surname and calls 'addStudents'. For delete (option 4), it shows all students, asks for

NOTES

Further study materials Lesson 3

Summary

an ID, and calls 'deleteStudent'. Invalid inputs display an error.

Running the Program and Conclusion

The program is wrapped in a 'baslangic' function that displays the menu, processes the user's choice, and then recursively calls itself to allow multiple operations. The final code is executed by running the Python script. The project demonstrates practical use of Python functions and SQL commands (UPDATE, INSERT, DELETE) on a database. It highlights that there is more to learn, but this milestone is achieved. The output includes screenshots of the running application.

NOTES

Further study materials Lesson 3

Transcript

TASK 8: Developing a Project on DB Using UPDATE, INSERT and DELETE Functions in SQLite with Python

In this project, we will develop a few processes by connecting to an SQL database with Python. First, I will mention SQLite and provide the auxiliary codes for installation on Kali Linux.

DB Browser for SQLite (DB4S) is an open-source tool to create, design, and edit database files compatible with SQLite. DB4S uses a familiar spreadsheet-like interface.

```
sudo apt install snapd #Snap Package Manager
(Snappy/Snapcraft)
sudo systemctl enable snapd.socket snapd apparmor
sudo systemctl start snapd.socket snapd apparmor

snap install sqlitebrowser
```

There is a simple student database in our example. We will search, update, add, and delete student data on this database.

At the beginning of the project, we will ask the user what action they want to take in the student list and list the options.

```
print ("Merhaba, öğrenci veri tabanında ne işlem yapmak
istersiniz."+"
"+"[1] Öğrenci Ara"+"
"+"[2] Öğrenci güncelleştir"+"
"+"[3] Öğrenci ekle"+"
"+"[4] Öğrenci sil")

islem = input ("Yapmak istediğiniz seçeneği giriniz: ")
```

Instead of writing the desired operations many times, we will create a functional structure once with the `def` command and perform as many processes as needed.

The `def` keyword is used to define a function (method) in Python. A function is a reusable block of code that performs a specific task or set of instructions.

```
import sqlite3

Connection =
sqlite3.connect("/home/kali/Desktop/DB4S/TESTDB.db")
Cursor = Connection.cursor() #"Cursor" receives commands
and makes connections with "Connection".

class Database:
def getColumnNames():
Cursor.execute("PRAGMA table_info(STUDENTS)")
Columns = Cursor.fetchall() # fetchall: means fetch all
data.
return Columns
```

NOTES

Further study materials Lesson 3

Transcript

```
def getAllStudents():
Cursor.execute("SELECT * FROM STUDENTS ORDER BY
studentName ASC;") #studentName column is used to sort
from ASC: A to Z. This operation is done for viewing, it
does not change the DB.
Students = Cursor.fetchall()
return Students
print(Database.getAllStudents())

def searchStudent(value): #Search for Student
value = '%' + value + '%'
Cursor.execute("SELECT * FROM STUDENTS WHERE studentName
LIKE ? OR studentSurname LIKE ?;", (value, value,))
Students = Cursor.fetchall()
return Students

def searchStudentID(value): #Search for StudentID
Cursor.execute("SELECT * FROM STUDENTS WHERE studentId=
?;", (value,))
Students = Cursor.fetchall()
return Students

def updateStudent(columnName, newValue, where): #Update
Student
Cursor.execute("UPDATE STUDENTS SET "+columnName+" = ?
WHERE studentId = ?;", (newValue, where,))
print("Veri güncellendi.")
print(Database.getAllStudents())
Students = Cursor.fetchall()
Connection.commit() #COMMIT command is the transactional
command used to save changes invoked by a transaction to
the database.
return Students

def addStudents(newName, newSurname): #Add Student's Data
Cursor.execute("INSERT INTO STUDENTS (studentName,
studentSurname) VALUES (?, ?);", (newName, newSurname,))
print("Veri eklendi.")
print(Database.getAllStudents())
Students = Cursor.fetchall()
Connection.commit()
return Students

def deleteStudent(delValue):
Cursor.execute("DELETE FROM STUDENTS WHERE studentId =
?;", (delValue,))
print("Veri silindi.")
print(Database.getAllStudents())
Students = Cursor.fetchall()
Connection.commit()
return Students
```

If the user wants to search the student list, the following code should be used for this process.

NOTES

Further study materials Lesson 3

Transcript

```
if islem == "1":
ara = input ("Aramak istediğiniz öğrenci ismini yazınız:
")
veri = Database.searchStudent(ara)
if veri == (Database.searchStudent(ara)):
print (veri)
else:
print ("Aradığınız öğrenci sistemde yoktur.")
```

If the user wants to update the student list, the following code must be used for this process.

```
elif islem == "2":
print (Database.getAllStudents())
guncelleme = input ("Güncellemek istediğiniz öğrencinin
ID bilgisini giriniz: ")
ad = Database.searchStudentID(guncelleme)

print (ad[0][0],"adlı öğrencinin, hangi bilgisini
güncelleştirmek istiyorsunuz?"+"
"+"[1] Öğrenci Adı"+"
"+"[2] Öğrenci Soyadı") # ad[0][0] : print the first
element inside the first element
guncelleme2 = input ("Güncellemek istediğiniz bilgiyi
giriniz: ")

if guncelleme2 == "1":
guncelAd= input ("Güncel öğrenci adını giriniz: ")
Database.updateStudent("studentName", guncelAd,
guncelleme)

elif guncelleme2 == "2":
guncelSoyad= input ("Güncel öğrenci soyadını giriniz: ")
Database.updateStudent("studentSurname", guncelSoyad,
guncelleme)

else:
print ("Geçersiz işlem yaptınız.")
```

If the user wants to add new data to the student list, the following code should be used.

```
elif islem == "3":
print ("Eklemek istediğiniz öğrencinin bilgilerini
yazınız")

yeniAd = input("Öğrenci adını giriniz: ")
yeniSoyad = input("Öğrenci soyadını giriniz: ")
yeniEkleme= Database.addStudents(yeniAd, yeniSoyad)

print (yeniEkleme)
```

If the user wants to delete data in the student list, the following code should be used.

```
elif islem == "4":
print (Database.getAllStudents())
```

NOTES

Further study materials Lesson 3

Transcript

```
silID = input ("Silmek istediğiniz öğrencinin ID
bilgisini giriniz: ")
veriSil= Database.deleteStudent(silID)

else:
print("Geçersiz işlem!")
```

We will need to use the structure below to include the process that the user wants to perform in the student list in an infinite loop.

```
def baslangic():

print ("Merhaba, öğrenci veri tabanında ne işlem yapmak
istersiniz."+"
"+"[1] Öğrenci Ara"+"
"+"[2] Öğrenci güncelleştir"+"
"+"[3] Öğrenci ekle"+"
"+"[4] Öğrenci sil")

islem = input ("Yapmak istediğiniz seçeneği giriniz: ")
if islem == ...
...
...
...
else:
print("Geçersiz işlem!")

return baslangic()

print (baslangic())
```

If we want to view the outputs of the process and the changes on the DB side:

```
python3 SQLite.py
```

*[Image: https://miro.medium.com/1*hjAuZhwm9tLS5hQq97ihJw.png]*

*[Image: https://miro.medium.com/1*m_M201njLuaOLxHhpthllw.png]*

*[Image: https://miro.medium.com/1*gl_WtpEjuF27bFvAxvEfoQ.png]*

*[Image: https://miro.medium.com/1*Z2phjqB3sPWhMKscbND8lw.png]*

*[Image: https://miro.medium.com/1*KUuDfQ3Sst3SZ3a1V9w9FA.png]*

In this project, we demonstrated in practice that we can change both the **def** structure and many SQL functions in the database with Python.

We still have a lot to learn. ?
We got past something. ?

UNIQUESEC Student Club | | Linkedin:)

NOTES

Further study materials Lesson 3

Content type: Text

How to Create Tables, Insert Data, Query, and Execute Multiple SQL Statements in SQLite

Available at https://medium.com/@nutanbhogendrasharma/sqlite-relational-database-management-with-python-7d9ca4fc2ae7

Full Content

Summary

SQLite and sqlite3 Module

SQLite is a C library that provides a small, fast, self-contained, serverless SQL database engine. Unlike other databases, it does not need a separate server process and reads and writes directly to ordinary disk files. A complete database is stored in a single disk file, which makes it cross-platform and very popular. It is built into all mobile phones and most computers. The sqlite3 module in Python allows us to work with SQLite databases. To use it, we create a Connection object that represents the database.

Creating Database and Person Table

To create a database file, we use the sqlite3.connect() function. First, we check if the file already exists using os.path.exists(). If it does not exist, the connect() call creates a new database file. After connecting, we define a SQL query to create a table named 'person' with columns: id (INTEGER primary key), firstname (text), lastname (text), and age (INTEGER). We then execute this query using a cursor object with the execute() method. After execution, we commit the changes and close the connection. This sets up the database for data operations.

Inserting Records into Person Table

After creating the person table, we insert multiple records using the executemany() method. We prepare a list of tuples, each containing values for id, firstname, lastname, and age. For example, we have records for Martha, John, Noah, Henry, and another John. Then we connect to the database, create a cursor, and call executemany() with an INSERT SQL statement that uses placeholders (?). After execution, we commit the transaction to save the data and close the connection. This method efficiently inserts several rows at once.

Retrieving Records from Person Table

To retrieve records, we use SELECT queries. After a SELECT, we can call fetchall() to get all rows as a list of tuples, fetchone() to get a single row, or use the cursor as an iterator. We demonstrate ordering by firstname using ORDER BY, filtering by firstname with

NOTES

Further study materials Lesson 3

Summary

named parameters (e.g., :firstname) or positional parameters (?), and using LIKE to find names containing 'j'. Each method returns the matching records, which we print. This shows various ways to query data from the person table.

Update, Delete, and Execute Multiple SQL

This part covers updating and deleting records. To update, we use an UPDATE SET statement with a WHERE condition to change specific rows. For example, we update the record with id 1 to change firstname to Maria, lastname to Lucy, and age to 55. To delete, we use DELETE FROM with a WHERE condition, removing the record with id 5. Both operations require commit to save changes. We also learn about executescript() for running multiple SQL statements at once, such as creating a 'book' table and inserting three rows. This method first commits any pending transaction. After executing the script, we retrieve and display the books. These operations are essential for managing database content.

NOTES

Further study materials Lesson 3

Transcript

SQLite Relational Database Management With Python

In this blog, we will create an SQLite database and tables. We will also insert, update, and delete records.

[Image: Created by Nutan]

What Is SQLite?

SQLite is a C-language library that implements a small, fast, self-contained, high-reliability, full-featured, SQL database engine. SQLite is an in-process library that implements a self-contained, serverless, zero-configuration, transactional SQL database engine.

SQLite is an embedded SQL database engine. Unlike most other SQL databases, SQLite does not have a separate server process. SQLite reads and writes directly to ordinary disk files. A complete SQL database with multiple tables, indices, triggers, and views, is contained in a single disk file. The database file format is cross-platform - we can freely copy a database between 32-bit and 64-bit systems. These features make SQLite a popular choice as an Application File Format. It is the most used database engine in the world. SQLite is built into all mobile phones and most computers and comes bundled inside countless other applications that people use every day.

What Is the sqlite3 Module?

The sqlite3 module was written by Gerhard Häring. It provides a SQL interface compliant with the DB-API 2.0 specification described by PEP 249, and requires SQLite 3.7.15 or newer.

To use the module, we need to create a Connection object that represents the database.

Create a Database

```
import os
import sqlite3
```

connection

This read-only attribute provides the SQLite database Connection used by the Cursor object. A Cursor object created by calling con.cursor() will have a connection attribute.

```
my_db = 'input/test.db'

db_exists = os.path.exists(my_db)

conn = sqlite3.connect(my_db)

if db_exists:
print('Database exists...')
else:
```

NOTES

Further study materials Lesson 3

Transcript

```
print('Database doesnot exists, creats a new database.')

conn.close()
```

Output: Database does not exist, creates a new database.

There was no database called test.db. So it created a new database called test.db.

Create a Schema

We will create a schema called person, which will have id, firstname, lastname and age column.

```
Column Type
id INTEGER
firstname text
lastname text
age INTEGER
```

Write a query

```
query = '''create table person (
id INTEGER primary key,
firstname text,
lastname text,
age INTEGER
)'''
```

Cursor Objects

class sqlite3.Cursor

A Cursor instance has the following attributes and methods.

execute(sql[, parameters]) - Executes an SQL statement. Values may be bound to the statement using placeholders.

execute() will only execute a single SQL statement. If you try to execute more than one statement with it, it will raise a Warning. Use executescript() if you want to execute multiple SQL statements with one call.

executemany(sql, seq_of_parameters) - Executes a parameterized SQL command against all parameter sequences or mappings found in the sequence seq_of_parameters. The sqlite3 module also allows using an iterator yielding parameters instead of a sequence.

```
my_db = 'input/test.db'

db_exists = os.path.exists(my_db)

conn = sqlite3.connect(my_db)

if db_exists:
cur = conn.cursor()
cur.execute(query)
conn.commit()
```

NOTES

Further study materials Lesson 3

Transcript

```
print("Person table created successfully...")
else:
print("Database doesnot exists, creats a new database.")

conn.close()
```

Output: Person table created successfully...

Insert Data in Person Table

We will insert multiple records in person table. For that we will use executemany() function.

```
person_list = [
(1, "Martha", "Lucy", 45),
(2, "John", "Bronze", 70),
(3, "Noah", "Charlotte", 20),
(4, "Henry", "Sophia", 55),
(5, "John", "Sophia", 45)
]
```

...

```
my_db = 'input/test.db'

db_exists = os.path.exists(my_db)

conn = sqlite3.connect(my_db)

if db_exists:
cur = conn.cursor()
cur.executemany("insert into person values (?, ?, ?, ?)",
person_list)
conn.commit()
print("Record inserted successfully...")

else:
print("Database doesnot exists, creats a new database.")

conn.close()
```

Output: Record inserted successfully...

Retrieve Records After Insert

Select Person Table and Display All Records

To retrieve data after executing a SELECT statement, either treat the cursor as an iterator, call the cursor's fetchone() method to retrieve a single matching row, or call fetchall() to get a list of the matching rows.

```
my_db = 'input/test.db'

db_exists = os.path.exists(my_db)

conn = sqlite3.connect(my_db)

if db_exists:
cur = conn.cursor()
```

NOTES

Further study materials Lesson 3

Transcript

```
cur.execute("select * from person")
records = cur.fetchall()

print("Fetching all records from person table")
print("
", records)
else:
print("Database doesnot exists, creats a new database.")

conn.close()
```

Output:

*[Image: https://miro.medium.com/1*1bkaFGvPiZKoWM-_Ox8y3g.png]*

View Records by Order

```
my_db = 'input/test.db'

db_exists = os.path.exists(my_db)

conn = sqlite3.connect(my_db)

if db_exists:
cur = conn.cursor()
cur.execute("select * from person order by firstname")
records = cur.fetchall()

print("Fetching all records from person table and order
by firstname")
print("
", records)
else:
print("Database doesnot exists, creats a new database.")

conn.close()
```

Output:

*[Image:
https://miro.medium.com/1*NPYSCwa5aztymzwnmVM_HQ.png]*

Fetch Only One Record

```
my_db = 'input/test.db'

db_exists = os.path.exists(my_db)

conn = sqlite3.connect(my_db)

if db_exists:
cur = conn.cursor()
cur.execute("select * from person")
record = cur.fetchone()

print("Fetching only one record from person table")
print("
", record)
else:
print("Database doesnot exists, creats a new database.")
```

NOTES

Further study materials Lesson 3

Transcript

```
conn.close()
```

Output:

*[Image: https://miro.medium.com/1*CKP5eBt4kRXhRr1jrDW_qQ.png]*

Retrieve Records by Named Style

```
my_db = 'input/test.db'

db_exists = os.path.exists(my_db)

conn = sqlite3.connect(my_db)

if db_exists:
cur = conn.cursor()
cur.execute("select * from person where
firstname=:firstname order by age", {"firstname":
"John"})
records = cur.fetchall()

print("Fetching all records from person table, whose
firstname is John and order by age")
print("
", records)
else:
print("Database doesnot exists, creats a new database.")

conn.close()
```

Output:

*[Image: https://miro.medium.com/1*3TOwj9_1c2Q8Yqn_aOoS9g.png]*

Retrieve Records by Positional Parameters

A question mark (?) denotes a positional argument, we have to pass to execute() function as a member of a tuple.

```
my_db = 'input/test.db'

db_exists = os.path.exists(my_db)

conn = sqlite3.connect(my_db)

if db_exists:
cur = conn.cursor()
cur.execute("select * from person where firstname = ?
order by age", ("John",))
records = cur.fetchall()

print("Fetching all records from person table, whose
firstname is John and order by age")
print("
", records)
else:
print("Database doesnot exists, creats a new database.")

conn.close()
```

NOTES

Further study materials Lesson 3

Transcript

Output:

*[Image: https://miro.medium.com/1*5mXXfGV-YCJiHjEe3uRTxQ.png]*

Retrieve Records by Like

```
my_db = 'input/test.db'

db_exists = os.path.exists(my_db)

conn = sqlite3.connect(my_db)

if db_exists:
cur = conn.cursor()
cur.execute("select * from person where firstname like
'%j%'")
records = cur.fetchall()

print("Fetching all records from person table, whose
firstname starts with J")
print("
", records)
else:
print("Database doesnot exists, creats a new database.")

conn.close()
```

Output:

[Image: https://miro.medium.com/1-4xtbOWonKYQG3qTzu5RCg.png]*

Update Record

```
my_db = 'input/test.db'

db_exists = os.path.exists(my_db)

conn = sqlite3.connect(my_db)

if db_exists:
cur = conn.cursor()
cur.execute("update person set firstname = ?, lastname =
?, age = ? where id = ?", ('Maria', 'Lucy', 55, 1,))
conn.commit()

cur.execute("select * from person")
records = cur.fetchall()

print("Fetching all records from person table after
update")
print("
", records)

else:
print("Database doesnot exists, creats a new database.")

conn.close()
```

Output:

NOTES

Further study materials Lesson 3

Transcript

[Image: https://miro.medium.com/1-cugu37FZueSt8boun7eYg.png]*

We can see first record updated successfully.

Delete Record

In this, we are deleting one person record whose id is 5.

```
my_db = 'input/test.db'

db_exists = os.path.exists(my_db)

conn = sqlite3.connect(my_db)

if db_exists:
cur = conn.cursor()
cur.execute("delete from person where id = ?", (5,))
conn.commit()

cur.execute("select * from person")
records = cur.fetchall()

print("Fetching all records from person table after
delete")
print("
", records)

else:
print("Database doesnot exists, creats a new database.")

conn.close()
```

Output:

*[Image: https://miro.medium.com/1*X5kYHKrk54fFibP8OCvo7w.png]*

Execute Multiple SQL Statements

If we want to execute multiple SQL statements with one call, we can use executescript() method.

executescript(sql_script)

This is a nonstandard convenience method for executing multiple SQL statements at once. It issues a COMMIT statement first, then executes the SQL script it gets as a parameter. This method disregards isolation_level; any transaction control must be added to sql_script.

sql_script can be an instance of str.

```
my_db = 'input/test.db'

db_exists = os.path.exists(my_db)

conn = sqlite3.connect(my_db)

if db_exists:
cur = conn.cursor()
```

NOTES

Further study materials Lesson 3

Transcript

```
cur.executescript("""
create table book(
title,
author,
published
);

insert into book(title, author, published)
values (
'Dirk Gently''s Holistic Detective Agency',
'Douglas Adams',
1987
);

insert into book(title, author, published)
values (
'Joe Biden: American Dreamer',
'Evan Osnos',
2021
);

insert into book(title, author, published)
values (
'Artificial Intelligence and the Future of Power: 5
Battlegrounds',
'Rajiv Malhotra',
2021
);

""")
else:
print("Database doesnot exists, creats a new database.")

conn.close()
```

Above script will create table book and insert record in book table. Let's retrieve and see records.

Retrieve Book Table

```
my_db = 'input/test.db'

db_exists = os.path.exists(my_db)

conn = sqlite3.connect(my_db)

if db_exists:
cur = conn.cursor()
cur.execute("select * from book")
records = cur.fetchall()

print("Fetching all records from book table")
print("
", records)
else:
print("Database doesnot exists, creats a new database.")

conn.close()
```

NOTES

Further study materials Lesson 3

Transcript

Output:

*[Image: https://miro.medium.com/1*E1odan8riRM8ayIPuQLBZQ.png]*

We have successfully executed multiple SQL scripts with executescript() method.

Thanks for reading.

NOTES

Exercises • Lesson 3

EXERCISE 1 • True or false

CONTEXT

When working with SQLite in a Flet app, managing the database connection is important for performance and reliability.

QUESTION • Indicate whether the information is true or false

It is best to open a new SQLite connection for each database query and close it immediately after.

◯ TRUE ◯ FALSE

NOTES

Exercises • Lesson 3

EXERCISE 2 • Multiple choice

CONTEXT

The lesson discusses safe database interactions in Flet apps, emphasizing the use of parameterized queries to prevent SQL injection.

QUESTION • Select the correct option

Which method should be used to insert user input into an SQL query to prevent SQL injection?

A. Use the % operator for string formatting with the variable.

B. Use ? placeholders in the SQL string and pass the values as a separate tuple to the execute() method.

C. Use f-strings to embed the variable directly into the SQL string.

D. Use string concatenation to build the SQL string with the variable.

NOTES

Exercises • Lesson 3

EXERCISE 3 • True or false

CONTEXT

The sqlite3 module in Python allows you to create and manipulate SQLite databases. Proper handling of database connections is essential for data integrity.

QUESTION • Indicate whether the information is true or false

It is necessary to call the commit() method on the connection object before closing it to persist any changes made to the database.

◯ TRUE ◯ FALSE

NOTES

Exercises • Lesson 3

EXERCISE 4 • Multiple choice

CONTEXT

After establishing a connection to an SQLite database in Python, you must properly manage resources to ensure data integrity and avoid memory leaks.

QUESTION • Select the correct option

What is the correct sequence of steps to safely insert data into an SQLite database using the sqlite3 module?

A. Create cursor, connect, execute insert, close, commit.

B. Connect, commit, create cursor, execute insert, close.

C. Connect, create cursor, execute insert, commit, close.

D. Connect, execute insert, commit, close, create cursor.

NOTES

Exercises • Lesson 3

EXERCISE 5 • True or false

CONTEXT

In SQL, there are several fundamental operations that allow you to interact with a database. These operations are often abbreviated as CRUD, which stands for the core actions you can perform on data.

QUESTION • Indicate whether the information is true or false

The four basic database operations are Create, Read, Update, and Delete.

◯ TRUE ◯ FALSE

NOTES

Exercises • Lesson 3

EXERCISE 6 • Multiple choice

CONTEXT

In SQL, the UPDATE statement is used to modify existing records in a table.

QUESTION • Select the correct option

What is the effect of executing an UPDATE statement without specifying a WHERE clause?

A) It updates only the first row.

B) It causes an error because WHERE is required for UPDATE.

C) It updates only the last row.

D) It updates all rows in the table.

NOTES

Exercises • Lesson 3

EXERCISE 7 • True or false

CONTEXT

In the student database project, a search function is implemented to find students by name or surname.

QUESTION • Indicate whether the information is true or false

The searchStudent function in the Database class searches for an exact match of the student name.

◯ TRUE ◯ FALSE

NOTES

Exercises • Lesson 3

EXERCISE 8 • Multiple choice

CONTEXT

In the Python SQLite project, the Database class contains several functions for managing student records. One of these functions handles adding new students to the database.

QUESTION • Select the correct option

What does the addStudents function in the Database class do?

A) It deletes a student from the database.

B) It searches for a student by ID.

C) It updates an existing student's name or surname.

D) It adds a new student with only name and surname to the database.

NOTES

Exercises • Lesson 3

EXERCISE 9 • True or false

CONTEXT

SQLite is a popular embedded SQL database engine used in many applications and devices.

QUESTION • Indicate whether the information is true or false

SQLite requires a separate server process to manage the database.

◯ TRUE ◯ FALSE

NOTES

Exercises • Lesson 3

EXERCISE 10 • Multiple choice

CONTEXT

In the lesson about SQLite, we learn that it is a self-contained, serverless database engine that stores data in a unique way.

QUESTION • Select the correct option

How does SQLite store its entire database?

A) In a single disk file

B) Entirely in memory

C) Using a database server process

D) In multiple separate files

NOTES

Exercise Answers

LESSON 3

EX.01 • True or false

False.

The lesson advises maintaining a single connection per session rather than opening and closing for each query. This avoids overhead and potential locking issues.

I got it right I got it wrong

EX.02 • Multiple choice

Option B — "Use ? placeholders in the SQL string and pass the values as a separate tuple to the execute() method."

The lesson explicitly states that using ? placeholders and passing values as a tuple prevents SQL injection, unlike direct string formatting.

I got it right I got it wrong

EX.03 • True or false

True.

According to the lesson, you must commit changes using conn.commit() before closing the connection; otherwise, any uncommitted changes will be lost. This ensures data persistence.

I got it right I got it wrong

EX.04 • Multiple choice

Option C — "Connect, create cursor, execute insert, commit, close."

This sequence ensures the connection is established, a cursor is obtained to execute the SQL, the insert is performed, changes are committed to persist, and finally the connection is closed to free resources.

I got it right I got it wrong

EX.05 • True or false

True.

The lesson explicitly states that we covered four basic operations: Create (CREATE TABLE, INSERT), Read (SELECT), Update (UPDATE), and Delete (DELETE). These are collectively known as CRUD.

I got it right I got it wrong

EX.06 • Multiple choice

Option D — "It updates all rows in the table."

The lesson explains that without a WHERE clause, the UPDATE statement changes every row in the table. For example, updating Charles to Chuck affected only that row because a WHERE clause was used, but without it, all rows would be updated.

I got it right I got it wrong

Exercise Answers

LESSON 3

EX.07 • True or false

False.

The searchStudent function uses the LIKE operator with wildcards (%) around the search value, allowing partial matches rather than exact matches.

I got it right I got it wrong

EX.08 • Multiple choice

Option D — "It adds a new student with only name and surname to the database."

The addStudents function uses an INSERT statement that adds values only for studentName and studentSurname, leaving studentId to be auto-generated.

I got it right I got it wrong

EX.09 • True or false

False.

Unlike most SQL databases, SQLite is serverless and does not have a separate server process. It reads and writes directly to disk files, making it embedded and self-contained.

I got it right I got it wrong

EX.10 • Multiple choice

Option A — "In a single disk file"

The lesson states that SQLite reads and writes directly to ordinary disk files, and a complete SQL database is contained in a single disk file.

I got it right I got it wrong

Chapter 06 • Async Programming and Data Integration

Lesson 04 • Async Handlers in Flet

13 pages • 18 min

elevify

Lesson 04 • Async Handlers in Flet

18 min • 13 pages

◎ What you will learn

Explains how Flet supports async event handlers natively and when to use them. Async handlers are the correct pattern for any I/O-bound operation.

Summary	8 min	**139**
Transcript	8 min	**140**
Further study materials		
Text • Async Python: Concurrency with asyncio and Avoiding Common Pitfalls	10 min	**141**
Exercises		**147**
Answer key		**151**

Async Handlers in Flet

Summary

Introduction to Async Handlers in Flet

This is Lesson 3 of the Flet Python Course. It teaches async event handlers that keep your UI responsive while the app fetches data, reads files, or waits on anything that takes time. The themed illustration shows coroutines and event arrows flowing through a UI window without blocking. The lesson begins now, focusing on a key concept for building smooth applications.

The Problem and Solution: Async Event System

The problem is that when a button click triggers a regular function doing something slow like a network request or file read, the UI freezes until it finishes. Async handlers fix this by letting slow work run in the background while the UI stays responsive. You will learn how this solves frozen screens and improves user experience by keeping the interface interactive during long operations.

How Flet's Event System Works

Flet's event system works by calling the handler you assigned when a user interacts with a control. By default it runs synchronously, but Flet also supports async handlers. Just use async def instead of def, and Flet automatically picks it up. The change is minimal: add async before def and await before any call that returns a coroutine. The UI thread never blocks, and everything else works the same, giving non-blocking performance with minimal changes.

Writing Async Handlers in Practice

Inside an async handler you can freely await any coroutine such as httpx for fetching data, reading a file, or querying a database. Once the await completes, you update controls normally. A critical rule is that await is only valid inside an async function; forgetting async def causes a syntax error. A real pattern shows opening an async HTTP client, awaiting GET, reading JSON, and setting a text control. Use page.update_async() for cleaner code in async contexts.

When to Use Async vs Sync

Not every handler needs to be async. Sync handlers work great for instant actions like toggling a checkbox, clearing a form, or updating a counter. Save async for operations that wait on I/O: HTTP requests, file reads, database queries, or asyncio.sleep. In a real Flet app you will mix both types, using sync for quick UI updates and async for any slow background work. This keeps your app responsive and efficient.

NOTES

Async Handlers in Flet

Transcript

Introduction

This is Lesson 3 of the Flet Python Course: Async Handlers in Flet. You'll write async event handlers that keep your UI responsive while the app fetches data, reads files, or waits on anything that takes time. The themed illustration shows coroutines and event arrows flowing through a UI window without blocking. Let's begin.

The Problem and Solution: Flet's Async Event System

Here's the problem. When a button click triggers a regular function that does something slow, like a network request or a file read, the UI freezes until it finishes. The user sees a frozen screen, can't click anything, and has no clue what's happening. Async handlers fix that by letting the slow work run in the background while the UI stays responsive. That's what we'll learn.

Here's how Flet's event system works. When a user interacts with a control, Flet calls the handler you assigned. By default, that handler runs synchronously. But Flet also supports async handlers — just use `async def` instead of `def`, and Flet picks up on it automatically. No extra setup needed.

The change is minimal but powerful. You add `async` before `def`, and add `await` before any call that returns a coroutine. Flet sees that your handler is a coroutine function and schedules it on the event loop instead of calling it directly. The UI thread never blocks. Everything else — reading the event object, updating controls, calling `page.update` — works exactly the same. So you get non-blocking performance with minimal changes.

Writing Async Handlers in Practice

Inside an async handler, you can freely `await` any coroutine. For instance, you might fetch data with `httpx`, read a file, or query a database. Once the await completes, you have the result available, and you update your controls the normal way. There's one critical rule: `await` is only valid inside an async function. If you forget to write `async def`, Python will raise a syntax error as soon as it encounters that `await` keyword.

Here's a real working pattern. The handler opens an async HTTP client, awaits the GET request, reads JSON, then sets a text control's value. Notice `page.update_async()` at the end — Flet provides that async version specifically for async handlers. You can still use regular `page.update`, but `update_async` is the cleaner choice when you're already in an async context.

You don't need to make every handler async. Sync handlers work great for instant actions — toggling a checkbox, clearing a form, updating a counter. Save async for anything that waits on I/O: HTTP requests, file reads, database queries, or `asyncio.sleep`. In a real Flet app, you'll

NOTES

Further study materials Lesson 4

Content type: Text

Async Python: Concurrency with asyncio and Avoiding Common Pitfalls

Full Content

Available at https://medium.com/the-pythonworld/async-await-in-python-a-beginner-friendly-guide-with-real-examples-4db9d5dffaa9

Summary

Why Async Matters in Python

Many Python developers find async confusing, but it greatly improves I/O-bound programs. Instead of blocking while waiting, async lets your program do something else in the same thread. This guide breaks down async/await step by step with real examples and practical insights to make it easy to understand.

Sync vs Async: The Restaurant Metaphor

Synchronous code blocks until each task finishes, like standing in line. Async is like getting a buzzer at a restaurant: you wait without blocking, doing other tasks. This non-linear control flow allows multiple operations to run concurrently, saving time when your program spends a lot of time waiting for external responses.

Key Concepts: async def, await, Event Loop

Core async concepts: async def defines a coroutine (pausable function), await pauses it until another completes, and the event loop manages tasks. Only use await inside async def. Start the loop with asyncio.run(). Coroutines don't run until awaited – they return objects that need explicit execution.

Real Example: Concurrent API Calls

Making five API calls sequentially takes ~10 seconds; async with aiohttp and asyncio.gather takes ~2 seconds by running them concurrently. The async version uses async with for sessions, await for responses, and gathers tasks. This example shows async's power for I/O-bound work like web scraping or API calls.

When to Use Async and Common Mistakes

Use async for waiting tasks (HTTP, disk, DB) – saves time. Avoid for CPU-bound work (use multiprocessing). Common pitfalls: mixing sync/async, blocking the event loop with time.sleep() (use asyncio.sleep()), and forgetting to await. Async improves speed, scalability, and responsiveness with less memory and threads.

NOTES

Further study materials Lesson 4

Transcript

[Image: Photo by Heidi Fin on Unsplash]

Async doesn't have to be intimidating - once you "get it," it's a game changer.

Async/Await in Python: A Beginner-Friendly Guide with Real Examples

Master Python's async and await with simple explanations, hands-on examples, and practical use cases that finally make it all click.

Introduction: "Why Does Everyone Keep Talking About Async?"

If you've been coding in Python for a while, chances are you've run into the words *async*, *await*, or *coroutines* - and maybe quickly backed away.

You're not alone.

Many developers (even experienced ones) find Python's async model confusing at first glance. The syntax is different, the control flow feels "non-linear," and the concept of coroutines sounds like something out of a computer science thesis.

But here's the truth: *async/await in Python isn't that scary - and it can drastically improve how you write I/O-bound programs.*

This article is your guide to demystifying async in Python. We'll break it down step by step, with real-world examples and practical insights that'll make you say, "Ah, *now* I get it."

What Is Async in Python, Really?

Let's start with the problem.

Imagine you're writing a script that makes multiple HTTP requests:

```
import requests

def fetch_data():
response = requests.get("https://api.example.com/data")
return response.json()
```

If you call this function multiple times in a loop, each request blocks until the previous one finishes.

That's fine for one or two calls. But what if you need to fetch hundreds of URLs? Your script will crawl.

Async I/O to the rescue.

Instead of blocking while waiting for a response, async allows your

NOTES

Further study materials Lesson 4

Transcript

program to *do something else* while waiting - all within the same thread.

Synchronous vs Asynchronous: A Visual Metaphor

Here's a metaphor:

- **Synchronous Python** is like standing in line at the DMV. Each person is served one at a time. You can't move until the person ahead of you is done.
- **Asynchronous Python** is like ordering food at a restaurant with a buzzer. You place your order and sit down. When your food's ready, they buzz you. Meanwhile, you can chat, scroll your phone, or help others.

Async lets you *wait without waiting.*

Key Concepts: `async def`, `await`, and the Event Loop

Let's break down the three most important concepts:

1. `async def` - Defining a Coroutine

In async Python, functions defined with `async def` are **coroutines**. They're special functions that can be paused and resumed.

```
async def say_hello():
print("Hello")
```

Calling `say_hello()` doesn't run it immediately - it returns a coroutine object. You need to *run* or *await* it.

2. `await` - Pausing Until Something's Done

The `await` keyword pauses your coroutine until another coroutine finishes.

```
import asyncio

async def main():
await say_hello()
```

You can only use `await` **inside** an `async def` function.

3. The Event Loop - Async's Secret Sauce

The event loop is like a task manager. It runs in the background, monitoring tasks and resuming them when they're ready.

You can start it using:

```
asyncio.run(main())
```

A Real Example: Making Multiple API Calls

NOTES

Further study materials Lesson 4

Transcript

Concurrently

Here's where async really shines. Let's fetch multiple URLs:

Without Async (Slow & Blocking):

```
import requests

urls = ["https://httpbin.org/delay/2"] * 5

for url in urls:
response = requests.get(url)
print(response.status_code)
```

This takes **~10 seconds** (2s per request × 5).

With Async (Much Faster!):

```
import asyncio
import aiohttp

async def fetch(session, url):
async with session.get(url) as response:
print(response.status)
return await response.text()

async def main():
urls = ["https://httpbin.org/delay/2"] * 5
async with aiohttp.ClientSession() as session:
tasks = [fetch(session, url) for url in urls]
await asyncio.gather(*tasks)

asyncio.run(main())
```

This takes **~2 seconds total**, because all five requests happen *concurrently*.

When Should You Use Async in Python?

Async isn't always the right tool. Use it when your program:

Spends time **waiting** - like making HTTP requests, reading from disk, or querying a database.

Does a lot of **CPU-bound work** - like crunching numbers or processing large images. For that, use multiprocessing or threads.

Good use cases:

- Web scraping multiple sites
- Building APIs (e.g., FastAPI)
- Real-time bots or chat apps
- File uploads/downloads
- Periodic background tasks

Common Gotchas for Beginners

Even though async is powerful, it's easy to trip up. Here are some

NOTES

Further study materials Lesson 4

Transcript

common pitfalls:

- **Mixing sync and async functions:** You can't just `await` a regular function.
- **Blocking the event loop:** Avoid using `time.sleep()` inside async code. Use `await asyncio.sleep()`.
- **Forgetting to `await`:** Calling an async function without `await` returns a coroutine object - it won't run until awaited.

Bonus: Async Sleep vs Sync Sleep

Let's compare two versions of "sleeping":

Synchronous Sleep

```
import time

def blocking():
time.sleep(2)
print("Done")
```

This **blocks** the entire thread.

Asynchronous Sleep

```
import asyncio

async def non_blocking():
await asyncio.sleep(2)
print("Done")
```

This **lets other tasks run** while sleeping - way more efficient in async programs.

How I Personally Use Async in My Projects

In one of my recent side projects - a dashboard that monitors multiple APIs - async helped me reduce a 20-second data refresh down to just 3 seconds.

Instead of waiting for each API to respond one by one, I used `aiohttp` to fire them all at once. The result? A snappy UI and much happier users.

Final Thoughts: Async Isn't Magic - But It Feels Like It

Once you understand the core concepts of async/await in Python, it opens up a new world of possibilities.

You'll write faster scripts, smoother APIs, and more responsive apps - all with fewer threads, less memory, and better scalability.

So don't fear the event loop. Embrace it.

And the next time someone says "just use async," you'll smile - because now, you *actually* know what that means.

NOTES

Further study materials Lesson 4

Transcript

Mastering async is like upgrading your Python brain - once you see the speed boost, there's no going back.

[Image: Photo by Javier Allegue Barros on Unsplash]

NOTES

Exercises • Lesson 4

EXERCISE 1 • True or false

CONTEXT

In Flet, event handlers can be written either synchronously or asynchronously. Async handlers are useful for tasks that involve waiting, such as network requests or file operations.

QUESTION • Indicate whether the information is true or false

When you define an event handler with 'async def', Flet automatically schedules it on the event loop instead of calling it directly.

◯ TRUE ◯ FALSE

NOTES

Exercises • Lesson 4

EXERCISE 2 • Multiple choice

CONTEXT

In Flet, event handlers can be written as synchronous or asynchronous functions to handle user interactions.

QUESTION • Select the correct option

What is the recommended way to handle a slow network request in a Flet button click event without blocking the UI?

A. Define the handler with def and use time.sleep to pause execution.

B. Define the handler with async def but don't use await, and call page.update_async() to trigger the request.

C. Define the handler with def and call asyncio.run() to execute the network request.

D. Define the handler with async def and use await for the network request.

NOTES

Exercises • Lesson 4

EXERCISE 3 • True or false

CONTEXT

In Python, asynchronous programming uses coroutines defined with 'async def' and the 'await' keyword to pause execution until a task completes.

QUESTION • Indicate whether the information is true or false

The 'await' keyword can be used inside any Python function, including those defined with 'def'.

◯ TRUE ◯ FALSE

NOTES

Exercises • Lesson 4

EXERCISE 4 • Multiple choice

CONTEXT

When writing async code, it's important to avoid blocking the event loop.

QUESTION • Select the correct option

What is the effect of using time.sleep() inside an async function?

A. It blocks the event loop and prevents other coroutines from running.

B. It is equivalent to await asyncio.sleep().

C. It raises an exception because time.sleep cannot be used in async functions.

D. It pauses only the current coroutine without blocking others.

NOTES

Exercise Answers

LESSON 4

EX.01 • True or false

True.

Flet detects that the handler is a coroutine function (async def) and schedules it on the event loop, ensuring the UI remains responsive while the handler executes.

I got it right I got it wrong

EX.02 • Multiple choice

Option D — "Define the handler with async def and use await for the network request."

Using async def makes the handler a coroutine, and await allows the slow operation to run without blocking the UI thread, keeping the app responsive.

I got it right I got it wrong

EX.03 • True or false

False.

The 'await' keyword can only be used inside 'async def' functions, not regular 'def' functions. Using 'await' outside an async context results in a syntax error.

I got it right I got it wrong

EX.04 • Multiple choice

Option A — "It blocks the event loop and prevents other coroutines from running."

time.sleep() is a synchronous blocking call. Inside an async function, it blocks the entire event loop, stopping other coroutines from executing until the sleep completes.

I got it right I got it wrong

Chapter 06 • Async Programming and Data Integration

Lesson 05 • Displaying Dynamic Lists from APIs

24 pages • 36 min

elevify

Lesson 05 • Displaying Dynamic Lists from APIs

36 min • 24 pages

◎ What you will learn

Renders API response data into ListView and DataTable controls. Connects data-fetching skills to the layout and state skills from earlier chapters.

Summary	8 min	**154**
Transcript	8 min	**155**
Further study materials		
Text • Mastering RESTful API Pagination, Filtering, and Sorting Best Practices	10 min	**158**
Text • Cursor-Based Pagination: Solving Offset Problems with Stable, Duplicate-Free Scrolling	17 min	**163**
Exercises		**171**
Answer key		**177**

Displaying Dynamic Lists from APIs

Summary

Mapping API Data to Controls

The core problem is turning API data into visible controls. Loop over the list of dictionaries, create a ListTile for each, gather them into a list, assign to ListView.controls, and update the page. Key ListView properties: expand for height, spacing for gaps, item_extent for performance. Always display a loading indicator: initialize with ProgressRing, then replace with real items. This pattern covers most list rendering scenarios.

Using DataTable for Structured Data

When API returns structured records with multiple fields, DataTable provides a clear layout. Define columns from field names once, then generate rows dynamically. Each field becomes a DataCell, which can hold any Flet control. Workflow: fetch data, define columns, use list comprehension to create DataRow per item with DataCells, assign rows to DataTable, and update page. That's about ten lines of Python for flexible interactive data displays.

Pagination Patterns for Loading More

Real APIs paginate data. Two common patterns: offset-based pagination and load-more button. For offset, track current_page, pass as query parameter, append new controls to existing list instead of replacing. Load-more button: place as last item, on tap swap for ProgressRing, await next page, append new items, put button back unless end. Detect end via has_next flag, null next URL, or empty response. Remove button when no more data to avoid empty fetches.

Pull-to-Refresh with RefreshIndicator

Users expect to drag down for fresh content. Flet provides RefreshIndicator control. Wrap your ListView in it, supply async on_refresh callback. Inside, reset page counter, clear existing controls, fetch first page again, populate list, update page. The indicator dismisses automatically when coroutine finishes. Simple implementation that gives your app a professional feel.

Combining Patterns and Avoiding Mistakes

Combine refresh and load-more by keeping paths separate: refresh resets to page one and rebuilds list; load-more increments and appends. Common mistakes: forgetting page.update, replacing list instead of extending on load-more, skipping loading state, ignoring end-of-data. Avoid these. Recap: ListView for fast lists, DataTable for structured data, pagination and pull-to-refresh for professional apps. These patterns are the foundation for data-driven screens.

NOTES

Displaying Dynamic Lists from APIs

Transcript

Overview

Static lists are fine for demos, but real apps need live data from an API. In this lesson you'll learn to pull that data and render it inside Flet controls. You'll build a scrollable list with pagination and pull-to-refresh — users load more on scroll and refresh on demand. By the end you'll have a solid reusable pattern you can drop straight into any Flet project.

Here are the three concrete things you'll build in this lesson. First, a dynamic ListView that fetches and renders items from an API—like cards in a scrollable list. Second, a DataTable that maps JSON fields into rows and columns, just like a spreadsheet. Third, the interaction patterns: pagination to load more results and pull-to-refresh to reload everything. Each one builds on the last.

Core Concept: Mapping API Data to Controls

Let me be clear. Here's the core problem: an API call returns a list of dictionaries, and your job is to turn each one into a visible control. That's it. That's the essential mapping step. Everything else you'll encounter in this lesson — pagination, refresh, and tables — is simply and solely a variation on that same one and only core idea.

Let's start simple. Your async fetch returns a list of dictionaries. Loop over that list. For each item, create a ListTile using the relevant fields. Gather those tiles into a Python list, assign it to ListView.controls, and call page.update. That single pattern handles about ninety percent of list rendering scenarios.

Three ListView properties you'll actually use. Setting expand to True is almost mandatory — without it the list collapses to zero height inside a Column. Spacing adds breathing room between items; start with 8 or 10 pixels. And item_extent is a performance trick: if every item is the same height, setting this value lets Flet skip per-item layout and render thousands of rows with zero lag.

Always put a loading indicator on screen before the data shows up. Here's the pattern: initialize your ListView with a single ProgressRing control. Await the API call. Once the data arrives, replace the controls list with the real items, then call page.update. Users get immediate feedback instead of a blank screen, and the whole transition feels instant. It's a simple trick that makes the app feel snappy.

Using DataTable for Structured Data

So far we've used ListView with cards for single items. But when your API returns structured records with multiple fields, a DataTable is often a much clearer layout. Let's walk through building one dynamically from an API response. In this section, we'll see how to make your data easy to read in a table. It's a

NOTES

Displaying Dynamic Lists from APIs

Transcript

straightforward process.

DataTable has two moving parts: columns and rows. You define the columns once, usually from the keys of your API response. Rows are generated dynamically, one per item. Inside each row, every field becomes a DataCell. Because DataCell accepts any Flet control, you can put text, icons, or even buttons inside a cell. That makes DataTable surprisingly flexible for interactive data displays.

It's a four-step workflow. First, fetch the data. Then define your columns from the field names — you do this once at build time. Next, use a list comprehension to generate a DataRow for each item, with each value wrapped in a DataCell. Finally, assign the rows to your DataTable and call page.update. That's about ten lines of Python.

Pagination Patterns

The reality: most real APIs do not return all records in one response. They paginate. So in the next slide, we'll look at two patterns for dealing with that: offset-based pagination and a load-more button. These are the two most common approaches you'll encounter when working with paginated APIs. Let's see how they work.

With offset pagination you keep track of the page you're on. Store a current_page variable, pass it as a query parameter with every request. When the data comes back, add the new controls to your existing list instead of replacing them. That's what sets it apart from the initial load — you extend, not replace. Then bump the page counter and update the UI.

The load-more button is the most user-friendly pagination pattern for mobile-style lists. You start by placing the button as the last item in your controls list. When the user taps it, swap it for a ProgressRing and await the next page of data. Once the fetch completes, append the new items to the existing list, then put the button back at the end. If the API signals there are no more pages, remove the button entirely.

How do you know when to stop? Some APIs give you an explicit has_next flag, or a next URL that becomes null. Others simply return an empty list when you've gone past the last page. You need to check for both signals. When the flag is false or null, remove it entirely. Same for an empty response — remove it completely. Otherwise, users will keep tapping and get nothing back.

Pull-to-Refresh

When people want fresh content, their first instinct is to drag down from the top of the list. That gesture—pull-to-refresh—is so common that users expect it everywhere. Flet makes it straightforward with a dedicated control. You don't need to write

NOTES

Displaying Dynamic Lists from APIs

Transcript

gesture recognizers or manage state; just wrap your list and you're done.

Just wrap your ListView in a RefreshIndicator. It handles the pull gesture and spinner for you. Supply an async on_refresh callback. Inside it, reset your page counter, clear the existing controls, fetch the first page of data again, populate the list, and call page.update. Once your coroutine finishes, the indicator dismisses itself automatically.

Combining Patterns and Common Mistakes

When you combine both patterns, separate the two paths to keep the logic clean. Refresh always resets to page one and rebuilds the list from scratch. Load-more always increments and appends. They share the same fetch function, but differ in whether they clear or extend the controls list. Keep that distinction explicit, and both features will work together without conflicts.

Four mistakes crop up repeatedly. Forgetting page.update is the most common – nothing repaints without it. Replacing the controls list on load-more instead of extending it wipes out everything the user already loaded. Skipping a loading state leaves users staring at a blank screen. And ignoring end-of-data makes your app keep firing requests that return nothing. Avoid these four and your list screens will behave correctly.

Recap

Alright, so those are the three patterns we covered. ListView gives you a fast, smooth-scrolling list from any API response. DataTable handles structured multi-field data with very little code. Pagination and pull-to-refresh give your app that professional, production-ready feel. Combine them, and you have the foundation for building data-driven screens that sit at the heart of almost any real-world application. That's the recap for this lesson.

NOTES

Further study materials Lesson 5

Content type: Text

Mastering RESTful API Pagination, Filtering, and Sorting Best Practices

Full Content

Available at https://thiagolima.blog.br/parte-5-pagina%C3%A7%C3%A3o-ordena%C3%A7%C3%A3o-e-filtros-em-apis-restful-3045d88b4114

Summary

Why Pagination, Sorting, and Filters Matter

Pagination, sorting, and filters are essential for making RESTful APIs efficient. When an API has many records, like over 10,000 wines, returning all data takes too long. These techniques break data into smaller pieces, allow ordering, and let clients request only what they need. This improves performance and scalability.

Client Experience and Infrastructure Benefits

Implementing pagination, sorting, and filters improves client experience by reducing response size and making data transfer faster. It also enables smarter infrastructure use, as data is consumed on demand. This helps with vertical and horizontal scalability, using server resources more efficiently and handling more requests without overloading the system.

Types of Pagination: Cursor, Page, Offset

Pagination has three common methods: cursor-based, page and page size, and offset and limit. Cursor-based uses unique keys like since_id and max_id to navigate records. Page and page size use page and page_size parameters to jump to a specific page. Offset and limit use offset to start from a record and limit to set the number of records returned.

Filters and Sorting in APIs

Filters and sorting are implemented via query string parameters. For example, filtering by country, state, price range, and status. Aliases like 'disponiveis' can simplify common filters. Sorting uses a sort parameter with field names and direction (asc or desc), separated by commas. This gives clients power to request exactly the data they need.

Best Practices for Pagination and Filters

Best practices include using offset and limit over page-based pagination to avoid recalculation issues. Always set default values, like a default limit of 500 and maximum of 1000, to protect performance. Avoid overcomplicating query strings and making parameters mandatory when not needed. Always design based on use cases for a better developer experience.

NOTES

Further study materials Lesson 5

Transcript

*[Image: https://miro.medium.com/1*ot0wOytPs4bqhOY6Bztcdg.jpeg]*

(Part 5) Pagination, Sorting, and Filters in RESTful APIs

Hey friends *hackers*, we've arrived at another article about APIs.

And today's topic is **pagination, sorting, and filters**, something essential in API design.

Remember that API with gastronomy data from the first articles of the series:

- [1] The anatomy of a RESTful API
- [2] What you need to know about HTTP methods and status codes
- [3] Security in RESTful APIs
- [4] Versioning RESTful APIs

So let's go, imagine that now this API already has a lot of data, and that when doing a *GET* of wines, it returns more than 10 thousand records, and it is taking too long to return to the clients consuming this resource.

That is exactly what pagination, sorting, and filters solve...

In summary, it's the implementation that makes your API work more efficiently with data in client-server communication, only transferring what is necessary!

To make it easier, I'll list here the main reasons for implementing these concepts:

Client Experience

Since you'll be fragmenting the result on the server, the response message transferred is much smaller, and therefore the client gets the request result much faster, creating a pleasant performance experience for the developer consuming the API.

Intelligent use of infrastructure

Since data consumption is on demand, this greatly helps with vertical and horizontal scalability of an API. In practice, this model uses infrastructure resources more intelligently.

Enough theory, now let's analyze the main types used in the market.

Pagination

Cursor

Cursor-based pagination works by a unique and sequential key that indicates from which record the data will be returned.

NOTES

Further study materials Lesson 5

Transcript

In other words, imagine you want the records from wine **ID 156** to **ID 200**. Then, you would make the following request:

```
HTTP GET
https://api.minhagastronomia/vinhos?since_id=156&max_id=200
```

Note that the parameters **since_id** and **max_id** were added, which in this case are the navigation cursors within the API, and they delimit the results to be returned by the request.

Typically, APIs with cursor-based pagination return in the request result the parameters for navigating to the previous page and also the next one.

Note: The name of cursor parameters may change depending on the API implementation.

Page and PageSize

Pagination based on **page** and **pagesize**, as the name itself says, is used through the parameters of the page number to be navigated and its respective size (in number of records).

Let's suppose you want data of 10 wines from the third page, then you would make the following request:

```
HTTP GET
https://api.minhagastronomia/vinhos?page=3&page_size=10
```

Note: The name of parameters may change depending on the API implementation.

Offset and Limit

Pagination based on **offset** and **limit** is used from an offset of records.

In other words, you specify in the offset from which record you want the data, and in the limit you specify the limit of records to be returned.

Now let's imagine you want 30 wines starting from the tenth record, then you would make the following request:

```
HTTP GET
https://api.minhagastronomia/vinhos?offset=10&limit=30
```

Filters and Sorting

About filters and sorting, there's not much secret, via query string parameters you can give more intelligence to your API, besides giving more capability to the client to work with data on demand.

Practical example of **filter**:

```
HTTP GET
```

NOTES

Further study materials Lesson 5

Transcript

```
https://api.minhagastronomia/vinhos?pais=Brasil&estado=RS
&de_preco=100&ate_preco=200&status=disponivel
```

In the example above, I am requesting all wines from Brazil, from the state of Rio Grande do Sul, with a price between 100 and 200 reais and that are available for purchase.

We also have the possibility of using **alias** for frequently used filters, for example:

```
HTTP GET
https://api.minhagastronomia/vinhos/disponiveis?estado=RS
&de_preco=100&ate_preco=200
```

In this case, I use the sub-resource "**disponiveis**" (available) that contains the pre-structured filter, making it easier for the developer to consume.

Practical example of **sorting**:

```
HTTP GET
https://api.minhagastronomia/vinhos?sort=pais:asc,estado
```

In the example above, note that I included the parameter **"sort"** which contains the sorting key field followed by **":"** to specify whether I want the data in ascending or descending order, in addition to the comma separating the fields.

This is not the only pattern for this technique, but I particularly find it quite ***clean***.

Best Practices

Offset and limit with "range" filters

My recommendation is to use the **offset** and **limit** technique for pagination, as it is not only the most used method lately, but also provides the best experience for the API consumer.

Navigation by page and page size requires page recalculations when changing the page size, and this ends up creating additional complexity that harms the developer experience.

And if you are going to use page and page size, please specify in your documentation at which page the data search starts, because I have seen countless cases where page 0 and 1 return the same records!

Have default values

Have default values in pagination parameters, for example, a default **limit** of 500 objects, and a maximum **limit** of 1000 objects, this way, besides making consumption easier when the client does not send the parameter, you also protect the performance and scalability of your API.

NOTES

Further study materials Lesson 5

Transcript

Don't overdo Query String parameters

My recommendation on this subject is not to overdo query string parameters, that is, analyze what really makes sense, otherwise you will be polluting the URL unnecessarily, making everything more complex for the developer consuming the API.

Another important point is not to make parameters mandatory in cases that don't make sense, for example:

```
HTTP GET
https://api.minhagastronomia/vinhos?pais=Brasil&estado=RS
&de_preco=100&ate_preco=200&status=disponivel
```

Imagine that in this request above the **status** parameter were mandatory, that would make no sense, because if I wanted all wines regardless of status, I would have to concatenate several requests.

In other words, by making a parameter mandatory, I created a bad experience in consuming my API.

And lastly, always think about use cases of your API when creating a new resource, this will greatly facilitate the definition of parameters and metadata.

That's it folks... We've reached the end of another article in our series on APIs.

And stay tuned on social media, I'll be back soon with more content!

NOTES

Further study materials Lesson 5

Content type: Text

Cursor-Based Pagination: Solving Offset Problems with Stable, Duplicate-Free Scrolling

Available at https://latteandcode.medium.com/paginaci%C3%B3n-todo-lo-que-necesitas-saber-209f1dadfb1

Full Content

Summary

Limit-Offset Pagination Issues

Many developers quickly implement pagination using LIMIT and OFFSET in SQL queries. However, this method can cause problems in real-time applications with infinite scroll or frequent data changes. For example, if new items are added or deleted between requests, users may see duplicate or missing items. This issue is less severe for simple blogs but becomes critical for messaging apps or social media feeds. Therefore, it is important to evaluate the risk before choosing this pagination method.

Example of Duplicate Photo Problem

Consider a photo list ordered by creation date. When a user scrolls to load more photos, a query with LIMIT and OFFSET is used. If another user adds a new photo at the same time, the original user may see a duplicate because the new photo shifts the position of earlier items. Similarly, if a photo is deleted, the next page may skip an item. This instability makes offset-based pagination unsuitable for environments with concurrent data modifications, as it can break the user's experience.

Stable Pagination with Cursor

To avoid the instability of offset pagination, use a stable value like the creation date as a cursor. Instead of page numbers, pass the last seen value to the query: SELECT * FROM photo WHERE createdAt < previousDate LIMIT 10. This ensures that the user always sees the next set of items after the last one, regardless of additions. However, if two items share the same timestamp, duplicates can occur. Solutions include using an incremental ID or filtering duplicates with <= condition.

Implementing Cursor-Based Pagination

Cursor-based pagination uses a stable identifier (cursor) to mark a position in the list. Requests include parameters like after (the cursor) and first (limit). For example, a query might be: SELECT * FROM posts WHERE created_at < $after ORDER BY created_at LIMIT $page_size. The response contains a cursor for the next page. Twitter's API uses since_id and max_id for similar purpose. This method prevents duplicate or missing items even when data

NOTES

Further study materials Lesson 5

Summary

changes in real time.

Relay Cursor Connections and Conclusion

Relay Cursor Connections is a GraphQL specification that standardizes cursor pagination. It uses pageInfo to indicate hasNextPage, startCursor, and endCursor. Results are returned as edges with a node and its cursor. This spec can also be applied to REST APIs. In conclusion, offset pagination is easier to implement and allows jumping between pages, but cursor pagination is better for real-time data. Choose based on project complexity and need for stability.

NOTES

Further study materials Lesson 5

Transcript

Pagination: everything you need to know

List of techniques to implement pagination of items in your applications

*[Image: https://miro.medium.com/1*jF6aZuh5Yi5AMNpbD9pbug.jpeg]*

When in a project we are asked to paginate the items in a list, we usually don't take long, right? The first thing we do is write a query like this:

```
SELECT .... LIMIT limit, offset
```

And then, we design a UX that everyone will recognize:

[Image: Numbered pages]

However, I have bad news: this type of pagination is not as valid in a world plagued by infinite scrolls and real-time activity. Therefore, throughout this article I will explain why sometimes this method can bring us problems and the alternative systems we can use to avoid them.

Paginating with limit, offset: perhaps a bad idea

Suppose we have a list of photos (Instagram style) that we want to display in a paginated manner with the most recent first and using *infinite scroll*. Surely the solution that comes to mind to display the first page involves a query like this:

```
SELECT * FROM photo ORDER BY createdAt DESC LIMIT 0, 10
```

Which will return the 10 most recent images. So far so good.

Now suppose the user scrolls to the bottom so we need to load the next 10 images. Our next query will be:

```
SELECT * FROM photo ORDER BY createdAt DESC LIMIT 10, 10
```

Obtaining the next 10 photos.

What happens if in the meantime user B adds a new photo? Well, the initial user will see a duplicate image because the new photo added by user B will push the tenth photo to the eleventh position, so it will become part of the second page:

*[Image: https://miro.medium.com/1*QD5n1-H6Evx5DmeXkUCAnw.png]*

In fact, the situation would be even worse if items can be deleted from the list. If user B deletes an item from the first page and user A then requests the second page, the eleventh item will not be visible because the pagination would skip it. Imagine this happens in a messaging app: disaster strikes.

So, when we are tempted to use this type of pagination, we must

NOTES

Further study materials Lesson 5

Transcript

evaluate both the probability of such cases occurring and the severity for our application if they do. For example, perhaps for a basic blog it is very unlikely to happen and if it did it wouldn't be very dramatic, so we can benefit from the speed at which this system is implemented.

However, if we cannot accept these drawbacks, we must implement some stable pagination system.

Stable pagination

The previous problem is due to the fact that the previous pagination method depends on the volatile *limit* and *offset* values, which makes it **unstable** in cases like the one above. A possible solution would be to choose a value that is stable over time and use it to paginate through. In our photo example, it could be the creation date of the photo.

From there, when we want to go through the pages, instead of passing to the query the page number we want to extract, we pass the date of the last returned photo:

```
SELECT * FROM photo WHERE createdAt < previousDate LIMIT
10
```

This way we ensure that the user will always see the set of photos after the last one they saw...

Unless two photos have the same date.

One way to solve this problem would be to modify the query to get all photos with a date *equal to or earlier than the one we receive* and then filter out duplicates (a quick fix and you know it).

```
SELECT * FROM photo WHERE createdAt <= previousDate LIMIT
10
```

And the other would be to use the photo's `id` (as long as it is incremental) as the element to paginate on, which would ensure we don't have repeated items after the query.

> *" Note. In case you haven't noticed, one of the drawbacks of this method (as well as the one I will describe below) is that we won't be able to have paginators like this:*
>
> *" [<<] [<] [1] 2 [3] ... [>][>>]*
>
> *" However, I believe it is acceptable to give up the ability to jump between pages, especially if we are moving towards pagination by infinite scroll where the user doesn't have that option, and which also seems to be the trend, especially on mobile.*

This type of pagination is called **cursor-based pagination** and it is what the rest of the article will focus on.

NOTES

Further study materials Lesson 5

Transcript

Cursor-based pagination

If you observe this diagram:

*[Image: https://miro.medium.com/1*QD5n1-H6Evx5DmeXkUCAnw.png]*

The problem of showing duplicate/losing items when items are added/deleted would be solved if we could directly specify **from which element we want to get the next results**. This way, it wouldn't matter how many items were added to the beginning of the list, because we would have a **stable cursor** pointing exactly to the place from which we want to continue getting results. This type of pagination is based on an element called a **cursor**, which is a kind of identifier that represents a location within a paginated list.

Thus, when we want to fetch data, we need two parameters:

- The cursor from which to start
- And the number of data items we want to retrieve.

For example:

```
// Front page
https://myapi.com/

// Next...
https://myapi.com/?after=t3_49i88b&first=30

// Next...
https://myapi.com/?after=t3_49kej9&first=30
```

In this case, the `after` parameter indicates the cursor from which we want to continue fetching data, and `first` acts as a `LIMIT` —that is, the number of results we want to obtain.

Implementing cursor-based pagination

As we saw in the introduction to stable pagination, the simplest way to implement this method would be to choose a cursor that remains stable over time and extract the data using queries like this:

```
SELECT * FROM posts
WHERE created_at < $after
ORDER BY created_at LIMIT $page_size;
```

Subsequently returning the results in this way (for example, if it were a REST API):

```
{
cursors: {
after: 23492834 // cursor that "represents" the last
element
},
posts: [ ... ]
}
```

NOTES

Further study materials Lesson 5

Transcript

Here, the `post` property would contain the array of posts, while the `cursors` property would contain a property called `after` that specifies the cursor value we must pass to the API in the next call to get the next batch of results.

Example of pagination on Twitter

If you have ever browsed Twitter (who hasn't...), the feed fills up as the people we follow post their tweets, so it is clear that they must use a pagination system similar to cursors. In fact, to search tweets through their API, the calls have this form:

```
https://api.twitter.com/1.1/search/tweets.json?q=php&sinc
e_id=24012619984051000&max_id=250126199840518145&result_t
ype=recent&count=10
```

That is, we use the `count` parameter to limit the number of results we want and the `since_id` parameter to specify from which tweet we want to get results. For that query, the returned result includes the `search_metadata` field with the following information:

```
"search_metadata": {
"completed_in": 0.047,
"max_id": 1125490788736032770,
"max_id_str": "1125490788736032770",
"next_results":
"?max_id=1124690280777699327&q=from%3Atwitterdev&count=2&
include_entities=1&result_type=mixed",
"query": "from%3Atwitterdev",
"refresh_url":
"?since_id=1125490788736032770&q=from%3Atwitterdev&result
_type=mixed&include_entities=1",
"count": 2,
"since_id": 0,
"since_id_str": "0"
}
```

In which we have the URL to get the next page (`next_results`), as well as the `max_id` of the returned results. You can see the full call at this link.

Relay cursor connections

With the arrival of GraphQL, a generic specification emerged for performing this work with cursors, implemented by Facebook's Relay library: the **Relay Cursor Connections:**

> *"Relay Cursor Connections Specification*

This specification is truly useful because it generalizes the concept of cursor and establishes a standard for making paginated requests and formatting responses. To give you an idea of the appearance proposed by this specification, here is what a GraphQL query looks like:

```
{
```

NOTES

Further study materials Lesson 5

Transcript

```
allNews(first: 3, after: "YXJyYXljb25uZWN0aW9uOjI=") {
pageInfo {
startCursor
endCursor
hasNextPage
}
edges {
node {
id
title
body
}
cursor
}
}
}
```

The main features of this format are as follows:

- Use of the `pageInfo` key to get information about the current page, such as whether there are more pages. However, the total number of elements is not returned because theoretically Relay clients do not need that information.
- Use of the `edges` key to get the array of results, known as nodes.
- Each result has on one side the node information and on the other its own cursor—that is, each element is returned along with its own cursor to facilitate page navigation (for example, we could make the next query starting from the middle of the one we have because we know the cursor value of the element in the center).

Finally, although this specification arose for working with GraphQL, we do not necessarily have to stick to this language, so we can easily port it to a REST API. That is:

> *" Relay simply generalizes the concept of cursor and presents a contract between client and server about how results are paginated.*

Conclusions

After analyzing the different pagination methods, my opinion on their use is that we should always choose the one that best fits the application or API we are developing. The main differences, in my opinion, would be:

- In offset-based pagination, sorting can be done by any column, while in cursor-based pagination, results are always sorted based on the field used to create the cursor.
- Offset-based pagination includes both page numbers and links to the previous and next pages, whereas in cursor-based pagination, it is not possible to jump between pages due to the

NOTES

Further study materials Lesson 5

Transcript

dynamic nature of the data to be returned.

• Offset-based pagination allows navigation in both directions (the direction is the same ?), while cursor-based navigation is most commonly used to go forward (although it can also be implemented to allow going backward).

So for starting any simple project, adopting the page number system (`LIMIT` and `OFFSET`) might be sufficient, since the speed of implementation is a clear point in its favor. However, for more complex applications involving a lot of real-time data, I think cursor-based pagination is a better choice—not for nothing is it used by applications like Twitter and Facebook.

Do you want to receive more articles like this?

Subscribe to our newsletter:

" ***Latte and code***

NOTES

Exercises • Lesson 5

EXERCISE 1 • True or false

CONTEXT

When building a dynamic list that loads more data on demand, managing the controls list correctly is crucial for a smooth user experience.

QUESTION • Indicate whether the information is true or false

In Flet's load-more pagination pattern, you should extend the existing controls list with new items rather than replacing it.

◯ TRUE ◯ FALSE

NOTES

Exercises • Lesson 5

EXERCISE 2 • Multiple choice

CONTEXT

When working with large lists in Flet, performance can become an issue. There is a property that can help if all items have the same height.

QUESTION • Select the correct option

Which ListView property can be set to skip per-item layout and render thousands of rows with zero lag?

- A controls
- B spacing
- C item_extent
- D expand

NOTES

Exercises • Lesson 5

EXERCISE 3 • True or false

CONTEXT

Pagination is an important technique for efficiently managing large datasets in RESTful APIs. Different pagination strategies exist, such as cursor-based pagination.

QUESTION • Indicate whether the information is true or false

Cursor-based pagination uses parameters such as since_id and max_id to navigate through records.

◯ TRUE ◯ FALSE

NOTES

Exercises • Lesson 5

EXERCISE 4 • Multiple choice

CONTEXT

In RESTful API design, pagination is crucial for efficiently handling large datasets. Different techniques exist, and the lesson discusses their pros and cons.

QUESTION • Select the correct option

Which pagination technique does the lesson recommend as the best practice for API consumers?

- A Page and page size pagination
- B Cursor-based pagination
- C Offset and limit pagination
- D Range-based pagination using filters

NOTES

Exercises • Lesson 5

EXERCISE 5 • True or false

CONTEXT

When implementing pagination, developers choose between offset-based (using LIMIT and OFFSET) and cursor-based methods. Each method has different characteristics regarding stability and navigation.

QUESTION • Indicate whether the information is true or false

In cursor-based pagination, users can directly navigate to a specific page number, just like in offset-based pagination.

◯ TRUE ◯ FALSE

NOTES

Exercises • Lesson 5

EXERCISE 6 • Multiple choice

CONTEXT

The lesson discusses two main pagination methods: offset-based and cursor-based. Each has its advantages and disadvantages.

QUESTION • Select the correct option

Which of the following is a key feature of cursor-based pagination?

A. It allows users to jump to any page number directly.

B. It uses a stable cursor (e.g., an ID or timestamp) to fetch the next set of results after a specific item.

C. It sorts results by a random field each time.

D. It guarantees that the total number of pages is known upfront.

NOTES

Exercise Answers

LESSON 5

EX.01 • True or false

True.

Extending the controls list preserves previously loaded items, allowing users to see their scroll history. Replacing would wipe out all data loaded so far, causing a poor user experience.

I got it right I got it wrong

EX.02 • Multiple choice

Option C — "item_extent"

Setting item_extent when every item has the same height lets Flet skip per-item layout calculations, enabling smooth scrolling for thousands of rows.

I got it right I got it wrong

EX.03 • True or false

True.

The lesson states that cursor-based pagination works by a unique and sequential key, and gives the example of using since_id and max_id parameters to delimit results.

I got it right I got it wrong

EX.04 • Multiple choice

Option C — "Offset and limit pagination"

The lesson explicitly states that offset and limit is the most used method lately and provides the best experience for API consumers, recommending it over other methods.

I got it right I got it wrong

EX.05 • True or false

False.

Cursor-based pagination does not support page-number navigation because the cursor represents a position in the list, and the data can change dynamically, making page numbers unstable. Instead, it uses cursors to fetch the next or previous set of results.

I got it right I got it wrong

EX.06 • Multiple choice

Option B — "It uses a stable cursor (e.g., an ID or timestamp) to fetch the next set of results after a specific item."

Cursor-based pagination uses a stable value like an ID or creation date to continue from exactly where the previous page ended, avoiding duplicates and skips even if data changes.

I got it right I got it wrong

Chapter 06 • Async Programming and Data Integration

Practical project of the chapter

15 pages

elevify

Practical project of the chapter

15 pages

Context and Provocation	**180**
Development Guide	**184**
Self-Assessment Rubric	**184**
Model Response & Assessment Reference	**185**
Notes	**191**

Async Programming and Data Integration

Driving Question

¿Cuál es la captura de pantalla (.png) que muestra tu aplicación obteniendo datos de la API y el diálogo de guardado de archivo?

Production Format

Argumentative essay — 800 to 1500 words.

Context and Provocation

Objective

Construir una pequeña aplicación Flet que demuestre el uso de async/await para obtener datos desde una API pública (TheMealDB) y permita guardar la receta en un archivo local usando FilePicker. Este proyecto ejercita las competencias de programación asíncrona, integración de datos externos y manejo de archivos, preparando el camino para la funcionalidad de importación/exportación del proyecto final MiRecetario.

Creative Brief

Marta, tu amiga cocinera, ha descubierto que hay muchas recetas interesantes en internet. Quiere que su recetario digital pueda importar recetas desde la web para no tener que escribirlas a mano. Te pide un prototipo rápido: una app que, con un solo clic, obtenga una receta aleatoria desde una API culinaria y la muestre en pantalla. Además, debe permitir guardar esa receta en un archivo local para usarla después. ¡Este es el primer paso para que MiRecetario se conecte con el mundo!

Objective

Construir una pequeña aplicación Flet que demuestre el uso de async/await para obtener datos desde una API pública (TheMealDB) y permita guardar la receta en un archivo local usando FilePicker. Este proyecto ejercita las competencias de programación asíncrona, integración de datos externos y manejo de archivos, preparando el camino para la funcionalidad de importación/exportación del proyecto final MiRecetario.

Creative Brief

Marta, tu amiga cocinera, ha descubierto que hay muchas recetas interesantes en internet. Quiere que su recetario digital pueda importar recetas desde la web para no tener que escribirlas a mano. Te pide un prototipo rápido: una app que, con un solo clic, obtenga una receta aleatoria desde una API culinaria y la muestre en pantalla. Además, debe permitir guardar esa receta en un archivo local para usarla después. ¡Este es el primer paso para que MiRecetario se conecte con el mundo!

Materials and Resources

- Computadora con Python 3.8+ y Flet instalado
- Editor de código (VS Code, PyCharm, o tu preferido)
- Conexión a internet (para consumir la API)
- Bibliotecas: flet, httpx (instalar con pip install httpx)
- Opcional: Postman o navegador para probar la API manualmente

Execution Steps

1. Configurar el proyecto y crear el archivo principal

• Crea una carpeta para el proyecto y dentro un archivo 'importador_recetas.py'.

• Importa las bibliotecas necesarias: import flet as ft, import httpx, import json, import asyncio.

• Profesional insider tip: Usa 'ft.app(target=main)' como punto de entrada y define una función 'main' asíncrona con 'async def main(page: ft.Page):'.

• Common mistake to avoid: Olvidar añadir 'await' en las llamadas a httpx; sin él la solicitud no se ejecutará correctamente.

2. Diseñar la interfaz de usuario

• Agrega un botón con texto "Obtener receta aleatoria" y un ListView vacío (ft.ListView) para mostrar los detalles de la receta.

• Coloca un segundo botón "Guardar receta" inicialmente deshabilitado (disabled=True).

• Profesional insider tip: Usa 'ft.ListView(expand=True, spacing=5)' para que ocupe todo el espacio vertical y los elementos se vean ordenados.

• Common mistake to avoid: Poner el ListView dentro de un Container sin expand, lo que puede hacer que no se vea correctamente.

3. Implementar el manejador asíncrono para obtener datos

• Define un 'async def obtener_receta(e)' que se ejecute al hacer clic en el primer botón.

• Dentro, usa 'async with httpx.AsyncClient() as client:' para hacer una solicitud GET a 'https://www.themealdb.com/api/json/v1/1/random.php'.

• Procesa la respuesta JSON para extraer el nombre, categoría, ingredientes e instrucciones.

• Actualiza el ListView con los datos (por ejemplo, creando ft.Text para cada campo).

• Habilita el botón "Guardar receta".

• Profesional insider tip: Usa 'page.update()' después de modificar los controles para que los cambios se reflejen en la UI.

• Common mistake to avoid: No manejar excepciones; si la API falla, la app se congela. Envuelve la solicitud en try/except y muestra un SnackBar con el error.

4. Agregar la funcionalidad de guardado con FilePicker

• Crea un FilePicker ('ft.FilePicker()') y agrégalo a la página (page.overlay.append).

• Define un 'async def guardar_receta(e)' que llame al FilePicker para elegir ubicación y nombre de archivo (extensión .json).

• En el evento 'on_result' del FilePicker, si se selecciona un archivo, escribe el JSON de la receta (usando json.dump) en la ruta elegida.

• Muestra un SnackBar de éxito.

• Profesional insider tip: Usa 'ft.FilePickerSaveFile' con 'allowed_extensions=["json"]' para filtrar archivos.

• Common mistake to avoid: Confundir FilePicker con el diálogo de archivos de sistema; asegúrate de manejar correctamente el evento 'on_result'.

5. Probar y depurar

• Ejecuta la app con 'flet run importador_recetas.py'.

• Haz clic en "Obtener receta aleatoria" y verifica que aparezcan los datos.

• Haz clic en "Guardar receta" y elige una ubicación; comprueba que el archivo se haya creado con el contenido correcto.

• Profesional insider tip: Usa 'print()' en la terminal para depurar si los datos llegan correctamente.

• Common mistake to avoid: No cerrar el cliente httpx; usar 'async with' asegura que se cierre automáticamente.

Self-Assessment Checklist

• [ ] La aplicación se ejecuta sin errores y muestra la interfaz con los dos botones y el ListView.

• [ ] Al hacer clic en "Obtener receta aleatoria", se muestra el nombre, ingredientes e instrucciones de una receta.

• [ ] Mientras carga, se muestra algún indicador (puede ser un Text "Cargando...") o el botón se deshabilita momentáneamente.

• [ ] Si la API falla, se muestra un mensaje de error (SnackBar o AlertDialog).

• [ ] El botón "Guardar receta" se habilita solo después de obtener una receta.

• [ ] Al hacer clic en "Guardar receta", se abre el diálogo de FilePicker para guardar un archivo .json.

• [ ] El archivo guardado contiene la receta en formato JSON válido.

• [ ] Todo el código utiliza funciones asíncronas con async/await correctamente.

Bonus Challenge

Agrega un campo de texto que permita buscar recetas por nombre usando el endpoint 'https://www.themealdb.com/api/json/v1/1/search.php?s={nombre}'. Muestra los resultados en el ListView y permite guardar cualquiera de ellos.

Submission Format

Accepted formats:

- image (.jpg/.png)

Minimum delivery:

- Una captura de pantalla (.png) que muestre la aplicación con una receta cargada en el ListView y el diálogo de FilePicker abierto para guardar el archivo. La captura debe evidenciar que la API respondió correctamente.

Development Guide

1. Research Phase
Look for examples of small apartment designs on sites like ArchDaily or Dezeen. Read news articles on micro-apartments to see both praise and criticism. Note how different designs handle the space-saving vs. comfort tradeoff. Use the course chapters on apartment planning and 3D modeling as your main guides.
2. Organization Phase
Start by stating your main argument — your position on balancing efficiency and comfort. Then outline three to four key points that support your view, using the mandatory references. For each point, plan what evidence or examples you'll use. Also think about one counterargument and how you'll respond to it.
3. Writing Phase
Write an introduction that presents the problem and your thesis. In the body paragraphs, each should focus on one key point — explaining a course concept and applying it to your design example. Use simple language: imagine explaining to a friend. Include specific examples, like how you'd place a bed or choose lighting. End with a conclusion that summarizes your argument and suggests practical tips for designers.
4. Review Phase
Check that your essay clearly states your position. Verify that you've used all four mandatory references correctly. Read it aloud to see if the logic flows smoothly. Ask yourself: would a beginner understand this? Remove any jargon or complex terms without explanation.

Self-Assessment Rubric

1. 1. Presencia de elementos UI (botones, ListView, FilePicker dialog)

- Insuficiente — Falta al menos un elemento obligatorio (botón o ListView) o el FilePicker no se ve.
- Adecuado — Los elementos están presentes pero con algún pequeño desalineamiento (botones no centrados, ListView no expandido).
- Excelente — Todos los elementos están visibles y correctamente ubicados. El ListView ocupa todo el espacio.

2. 2. Visualización correcta de la receta (nombre, ingredientes, instrucciones)

- Insuficiente — No se ve la receta o solo se ve un texto de carga sin datos reales.
- Adecuado — Se muestra nombre y al menos algunos ingredientes, pero falta categoría o instrucciones parciales.
- Excelente — Se muestra nombre, categoría, lista completa de ingredientes (al menos 5) e instrucciones legibles.

3. 3. Estado del botón "Guardar receta"

- Insuficiente — Botón deshabilitado (gris) a pesar de tener receta cargada.
- Adecuado — Botón habilitado pero no hay diferencia visual con el deshabilitado.
- Excelente — Botón claramente habilitado (no gris) y se distingue del botón de obtener.

4. 4. FilePicker dialog visible y con filtro .json

- Insuficiente — No hay ningún diálogo visible; solo se ve la ventana principal.
- Adecuado — El diálogo está abierto pero no se ve el filtro claramente (puede estar oculto).
- Excelente — El diálogo de guardado está abierto, mostrando el campo de nombre y el filtro de tipo .json (o "JSON Files").

5. 5. Sin mensajes de error

- Insuficiente — Hay un mensaje de error destacado que impide ver la receta o indica fallo.
- Adecuado — Aparece un mensaje de error menor (por ejemplo, de conexión) pero la receta aún se muestra.
- Excelente — No hay mensajes de error, ni SnackBar ni AlertDialog.

Model Response & Assessment Reference

> *This guide presents possible paths, not single answers. An excellent essay may defend any of the theses below — or an original thesis not anticipated here — as long as it demonstrates argumentative rigor, correct use of sources, and autonomous critical reflection.*

1. Expected Thesis Position(s)

The central question admits multiple legitimate positions. Below are argumentative paths representing robust and distinct directions.

> ***Author note:*** *The recommended range is 2 to 4 theses. If the assignment has only one defensible answer, keep just Thesis A and adapt the wording. A third thesis often works best as a synthetic or dialectical position.*

1.1. Thesis A — {Short label}

{Full thesis statement, in one or two sentences. It should take a clear position on the central question.}

Main argumentative line: {One paragraph describing how this thesis sustains itself: the central claim, supporting reasoning, implicit framework, and what makes it defensible.}

Concepts mobilized: {Course concept 1} ({brief connection}); {Course concept 2} ({brief connection}); {Course concept 3} ({brief connection}).

1.2. Thesis B — {Short label}

{Full thesis statement — a meaningfully different position from Thesis A, not just a softer variant.}

Main argumentative line: {One paragraph describing the central claim, supporting reasoning, implicit framework, and what makes it defensible.}

Concepts mobilized: {Course concept 1} ({brief connection}); {Course concept 2} ({brief connection}); {Course concept 3} ({brief connection}).

1.3. Thesis C — {Synthetic/dialectical position, optional}

{Full thesis statement — a meaningfully different position from Thesis A, not just a softer variant.}

Main argumentative line: {Show explicitly how this position produces a new insight that neither A nor B alone can reach.}

Concepts mobilized: {Course concept 1} ({brief connection}); {Course concept 2} ({brief connection}); {Course concept 3} ({brief connection}).

2. Key Concepts and Their Correct Use

> ***Author note:*** *Include 3 to 5 key concepts per project. For each concept: precise definition + example of correct application + common pitfall. The pitfall section is critical — it tells the evaluator what to watch for.*

2.1. A — {Concept Name}

How to define/explain: {One paragraph defining the concept precisely. Include its theoretical origin or author when relevant. Define its scope and limits.}

Correct use (example): {A verbatim example paragraph showing the concept doing actual argumentative work — not just being named, but actively grounding a claim.}

Common pitfall: {The most frequent way students misuse this concept: over-application, decorative citation, missing nuance, ignoring critiques, or conflating related concepts. Be concrete.}

2.2. B — {Concept Name}

How to define/explain: {One paragraph defining the concept precisely. Include its theoretical origin or author when relevant. Define its scope and limits.}

Correct use (example): {A verbatim example paragraph showing the concept doing actual argumentative work and grounding a claim.}

Concepts mobilized: {Course concept 1} ({brief connection}); {Course concept 2} ({brief connection}); {Course concept 3} ({brief connection}).

2.3. C — {Concept Name}

How to define/explain: {One paragraph defining the concept precisely. Include its theoretical origin or author when relevant. Define its scope and limits.}

Correct use (example): {A verbatim example paragraph showing the concept doing actual argumentative work and grounding a claim.}

Concepts mobilized: {Course concept 1} ({brief connection}); {Course concept 2} ({brief connection}); {Course concept 3} ({brief connection}).

3. Model Argumentative Structure

The outline below illustrates the structure of an excellent-level essay, regardless of which thesis the student chooses.

> ***Author note:*** *Word ranges are suggestions calibrated for a {total length}-word essay. Adjust proportionally if the assignment is shorter or longer.*

3.1. Introduction ({X}–{Y} words)

Contemporary hook: {Describe what kind of opening works for this project — a concrete example, striking data point, scene from culture, or current event. Adapt to the project's domain.}

Bridge to subject: {How the hook should pivot into the project's central question.}

Explicit thesis: The author's position in one or two clear, specific sentences, positioned directly against the central question.

3.2. Development ({X}–{Y} words)

Suggested: 2–3 paragraphs, each with a central argument.

Paragraph 1: {Function and how it advances the thesis.}

Paragraph 2: {Function and how it advances the thesis.}

Paragraph 3: {Function and how it advances the thesis.}

3.3. Counterargument ({X}–{Y} words)

An excellent essay does not merely mention an objection: it presents the strongest counterargument with intellectual honesty and responds consistently.

> ***Author note:*** *List the best counterargument for each thesis option, with a suggested response. The student should not feel that confronting the objection weakens their position — done well, it strengthens it.*

If thesis is A: {Strongest objection and possible response.}

If thesis is B: {Strongest objection and possible response.}

If thesis is C: {Strongest objection and possible response.}

3.4. Conclusion ({X}–{Y} words)

Restatement of thesis: Reaffirm the position in light of the full argumentation, without repeating the introduction verbatim.

Implications: {What this reflection reveals about something larger — the field, how we think, or a contemporary issue.}

Opening (no new information): A provocative question or forward-looking gesture that broadens the horizon without introducing arguments not developed in the essay.

3.5. Evidence and Data Types to Mobilize

- Specific passages from primary sources.
- Theoretical concepts with citation to the original work.
- Documented contemporary examples: news, official reports, data, or cultural productions.
- References to academic debates covered in the course.

4. Rubric — Excellent Level

Detailed description of what the evaluator should observe in an essay rated “Excellent” on each criterion.

> ***Author note:*** *The criteria below cover the standard dimensions of argumentative essays. Remove or adapt criteria that do not apply to the specific project.*

4.1. Thesis clarity

What is observed at the EXCELLENT level: The thesis appears in the introduction unequivocally. It is specific and directly engages the project's central tension. The entire essay is organized around it.

4.2. Use of course concepts

What is observed at the EXCELLENT level: Required references are defined precisely before application. Concepts are integrated organically into the argument, and their limits are recognized.

4.3. Quality of argumentation

What is observed at the EXCELLENT level: Each paragraph opens with a clear claim, sustains it with evidence, and connects back to the thesis. Logical progression is clear and the student analyzes rather than merely describes.

4.4. Counterarguments

What is observed at the EXCELLENT level: The student presents the strongest objection to their position and responds with a substantial argument. The thesis emerges stronger from confronting the critique.

4.5. Originality of analysis

What is observed at the EXCELLENT level: The essay goes beyond reproducing course materials. The student offers original connections, asks new questions, or proposes a novel interpretation from familiar evidence.

4.6. Connection to the contemporary

What is observed at the EXCELLENT level: The dialogue between subject and present is specific and grounded. Concrete contemporary examples illuminate both sides.

4.7. Structure and academic norms

What is observed at the EXCELLENT level: Cohesive structure with fluid transitions. Citations follow the established academic pattern, and references are complete and correctly formatted.

5. Common Argumentative Errors

The errors below are the most frequent in this type of project. The evaluator should watch for them.

> ***Author note:*** *Errors 2, 3, and 4 apply to almost any argumentative project. Errors 1 and 5 are project-specific slots where particular risks may be described.*

5.1. {Project-specific error}

How it appears in the text: {Verbatim-style example showing the error in action.}

How to correct: {Concrete, actionable advice on what to do differently.}

5.2. Strawman fallacy

How it appears in the text: The student presents the counterargument in its weakest, easiest-to-refute form instead of confronting the strongest version.

How to correct: Reformulate the counterargument as an intelligent interlocutor would present it, then respond to that reason.

5.3. Decorative use of concepts

How it appears in the text: The student names a concept but never applies it — it appears as a loose citation, disconnected from the argument.

How to correct: After citing a concept, explain exactly how it grounds the specific argument of that paragraph.

5.4. False equivalence

How it appears in the text: The student treats as equivalent two phenomena that operate in radically different contexts or functions.

How to correct: Make differences explicit before drawing parallels. Use precise comparative language. Comparison is not equation.

5.5. {Project-specific error}

How it appears in the text: {Example}

How to correct: {Correction}

> ***Final reminder:*** *The quality of reasoning is always more important than the position defended. An essay that defends an unpopular thesis with rigor outperforms one that defends a consensus thesis superficially.*